

ROMÂNIA
MINISTERUL APĂRĂRII NAȚIONALE
ACADEMIA TEHNICĂ MILITARĂ "FERDINAND I"

FACULTATEA DE SISTEME INFORMATICE ȘI SECURITATE CIBERNETICĂ
Calculatoare și Sisteme Informatice Pentru Apărare și Securitate Națională



***FROM LIKES TO LOVE: Înțelegerea Emoțiilor în Social Media
prin Analiza Sentimentală***

CONDUCĂTOR ȘTIINȚIFIC:
Lect. dr. ing. Stelian SPÎNU

ABSOLVET:
Sd. sg. maj. Bianca-Andreea GUCEANU

Conține: _____ file
Inventariat sub nr. _____
Poziția din indicator: _____
Termen de păstrare: _____

BUCUREȘTI
2025

Temă aprobată de Comandantul Academiei Tehnice Militare cu nr. __ din

NECLASIFICAT

Referat conducator stiintific (3 file)

NECLASIFICAT
1 din x

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

Declarație plagiat (1 filă)

NECLASIFICAT

Abstract

The objective of this paper is to conduct a detailed study on the performance of various artificial intelligence models applied to the detection and prevention of harassment on social networks. Additionally, the study will evaluate the effectiveness of integrating these models into web applications dedicated to analyzing and monitoring user interactions. The main objective is to ascertain the most efficacious methods and techniques that facilitate expeditious and precise intervention in the digital domain, thereby contributing to the establishment of a more secure and user-friendly online environment.

The study is organized into two primary sections. The initial section is dedicated to a meticulous examination of several algorithms employed in natural language processing, in addition to an exploration of the stages involved in processing and preparing datasets. This encompasses an evaluation of language processing techniques, as well as a meticulous analysis of available databases to identify the most pertinent and illustrative examples for training and testing models. The methods employed to maximize the value of the extracted information are also investigated. These methods include data enrichment techniques and the careful selection of relevant features. These techniques and features contribute to improving the accuracy and resilience of models in the face of the complexity and diversity of language used in the social media environment.

The second part of the study focuses on the design and development of an intuitive and accessible web application that ensures an easy user experience. The interface is designed to enable efficient data collection through automatic extraction techniques, as well as real-time data flow management using dedicated technologies. This section explores the optimal methods and strategies for integrating the necessary technologies into a robust web application capable of supporting fast and secure data processing and highlighting the results of the analyses.

The study's conclusions highlight the real potential of using artificial intelligence to create a functional platform that supports the detection and prevention of online harassment, contributing to the construction of a safer and more responsible virtual environment.

NECLASIFICAT

(pagina albă)

Rezumat

Această lucrare își propune să realizeze un studiu detaliat privind performanța diferitelor modele de inteligență artificială aplicate în detectarea și prevenirea hărțuirii pe rețelele de socializare, precum și evaluarea eficienței integrării acestor modele în cadrul aplicațiilor web dedicate analizei și monitorizării interacțiunilor utilizatorilor. Scopul principal este identificarea celor mai eficiente metode și tehnici care să permită o intervenție rapidă și precisă în mediul digital, contribuind astfel la crearea unui spațiu online mai sigur și mai prietenos.

Studiul este structurat în două secțiuni principale. Prima secțiune se concentrează pe o analiză detaliată a mai multor algoritmi utilizați în procesarea limbajului natural, precum și pe explorarea etapelor de prelucrare și pregătire a seturilor de date. Aceasta include o evaluare a tehnicilor de prelucrare a limbajului, dar și o analiză atentă a bazelor de date disponibile, pentru identificarea celor mai relevante și reprezentative exemple pentru antrenarea și testarea modelelor. De asemenea, sunt investigate metode pentru valorificarea cât mai completă a informațiilor extrase, prin tehnici de îmbogățire a datelor și selecția atentă a caracteristicilor relevante, toate acestea contribuind la îmbunătățirea preciziei și rezistenței modelelor în fața complexității și diversității limbajului utilizat în mediul online.

A doua parte a studiului se concentrează pe proiectarea și dezvoltarea unei aplicații web intuitive și accesibile, care să asigure o experiență facilă pentru utilizatori. Interfața este gândită astfel încât să permită colectarea eficientă a datelor prin tehnici de extragere automată, precum și gestionarea fluxurilor de date în timp real folosind tehnologii dedicate. În această secțiune, se explorează metodele și strategiile optime pentru integrarea tehnologiilor necesare în cadrul unei aplicații web robuste, capabile să susțină procesarea rapidă și sigură a datelor și să pună în valoare rezultatele analizelor.

Concluziile studiului evidențiază potențialul real al utilizării inteligenței artificiale în crearea unei platforme funcționale care să sprijine detectarea și prevenirea hărțuirii online, contribuind la construirea unui mediu virtual mai sigur și mai responsabil.

NECLASIFICAT

(pagină albă)

Cuprins

Listă de abrevieri	16
Listă de figuri	18
Listă de tabele.....	21
1. Introducere.....	23
1.1. Context	23
1.2. Importanța temei.....	24
1.3. Obiectivele lucrării	24
1.4. Etică.....	25
2. Noțiuni teoretice	25
2.1. Inteligența artificială	25
2.1.1. Provocările actuale în domeniul inteligenței artificiale	26
2.1.2. Învățarea automată	27
2.1.3. Învățarea profundă.....	28
2.1.4. Învățarea supervizată.....	31
2.1.5. Învățarea nesupervizată	32
2.2. Procesarea limbajului natural.....	32
2.2.1. Tehnici de preprocesare a textului	33
2.3. Modele de învățare automată utilizate	35
2.3.1. Logistic Regression	35
2.3.2. Random Forest	35
2.3.3. Multinomial Naive Bayes.....	35
2.3.4. Support Vector Machine	36
2.3.5. K-Nearest Neighbors	36
2.3.6. Bidirectional Encoder Representations from Transformers.....	36
2.3.7. Robustly Optimized BERT Approach	36
2.4. Evaluarea performanței modelelor	37

2.4.1. Matricea de confuzie	37
2.4.2. Acuratețe, precizie, regăsire, Scorul F1	38
2.4.3. Validarea modelelor	38
2.5. Noțiuni teoretice în dezvoltarea aplicației web	39
2.5.1. Fundamente React	39
2.5.2. Arhitectura REST și FastAPI	39
2.5.3. Apache Kafka în procesarea fluxului de date	40
2.5.4. Containerizare și orchestrare cu Docker și Docker Compose	41
2.5.5. Gestionarea datelor nerelaționale cu MongoDB	41
3. Studii și cercetări relevante în dezvoltarea proiectului	41
4.1. Analiza bazelor de date utilizate	43
4.1.1. Cyberbullying Classification	43
4.1.2. Offensive Language Identification Dataset	47
4.1.3. Loyola Univeristy Cyberbullying Dataset	50
4.2. Etapa de pregătire a seturilor de date	53
4.2.1. Transformări lingvistice preliminare ale datelor	53
4.2.2. Reprezentarea numerică a textului	55
4.3. Algoritmi folosiți pentru detectarea hărțuirii în mediul online	58
4.3.2. Random Forest	63
4.3.3. Multinomial Naive Bayes	68
4.3.4. Support Vector Machine	71
4.3.5. K-Nearest Neighbors	75
4.3.6. RoBERTa	77
4.3.7. BERT	79
4.3.8. Justificarea alegerii modelului final	80
5. Implementarea aplicației likes2love	80
5.1. Definirea cerințelor software ale sistemului	81

5.1.1. Cerințe funcționale	81
5.1.2. Cerințe nefuncționale	83
5.2. Arhitectura sistemului	83
5.2.1. Arhitectura clientului	84
5.2.2. Arhitectura serverului	85
5.3. Analiza Undefined Modeling Language	92
5.3.1. Diagrama cazurilor de utilizare	93
5.3.2. Diagrama de secvență	94
5.4. Procesul de testare a sistemului	95
5.5. Scenariu de utilizare	97
5.6. Dificultăți întâmpinate în procesul de implementare	105
6. Concluzii și direcții viitoare	107
Bibliografie	110

NECLASIFICAT

(pagina albă)

Listă de abrevieri

Nr. Crt.	Abreviere	Formă Completă
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	DL	Deep Learning
4	NLP	Natural Language Processing
5	LR	Logistic Regression
6	RF	Random Forest
7	NB	Naive Bayes
8	KNN	K-Nearest Neighbours
9	SVM	Support Vector Machine
10	BERT	Bidirectional Encoder Representations from Transformers
11	RoBERTa	Robustly Optimized BERT Approach
12	API	Application Programming Interface
13	UUID	Universally Unique Identifier
14	GPU	Graphic Processing Unit
15	CPU	Central Processing Unit

NECLASIFICAT

(pagina albă)

Listă de figuri

<i>Figura 2.1. Componentele inteligenței artificiale și relațiile dintre acestea</i>	26
<i>Figura 2.4.1 – Matrice de confuzie 2x2</i>	37
<i>Fig. 2.5.3-1 Apache Kafka.....</i>	40
<i>Figură 4.1.1-1 Distribuția claselor în setul de date</i>	44
<i>Figură 4.1.1-2 Distribuția numărului de cuvinte în comentarii</i>	44
<i>Figură 4.1.1-3 Nor de cuvinte</i>	45
<i>Figură 4.1.1-4 Frecvența cuvintelor per clasă</i>	46
<i>Figură 4.1.2-1 Distribuția claselor</i>	47
<i>Figură 4.1.2-2 Distribuția binară a claselor</i>	48
<i>Figură 4.1.2-3 Nor de cuvinte</i>	48
<i>Figură 4.1.2-4 Frecvența cuvintelor per clasă</i>	49
<i>Figură 4.1.3-1 Distribuția claselor</i>	50
<i>Figură 4.1.3-3 Matricea de corelație a claselor</i>	51
<i>Figură 4.1.3-4 Distribuția asocierii claselor</i>	52
<i>Figură 4.1.3-5 Nor de cuvinte</i>	52
<i>Fig 4.3.1 –1 Raport de clasificare pentru Logistic Regression TF-IDF</i>	59
<i>Fig 4.3.2 –3 Curba ROC pentru Logistic Regression TF-IDF</i>	60
<i>Fig 4.3.1–4 Raport de clasificare pentru Logistic Regression GloVe</i>	60
<i>Fig 4.3.1 –5 Matricea de confuzie pentru Logistic Regression GloVe</i>	61
<i>Fig 4.3.2 –6 Curba ROC pentru Logistic Regression GloVe.....</i>	61
<i>Fig 4.3.1–7 Raport de clasificare pentru Logistic Regression Word2Vec</i>	62
<i>Fig 4.3.1 –8 Matricea de confuzie pentru Logistic Regression Word2Vec</i>	62
<i>Fig 4.3.2 –1 Arbore de decizie</i>	64
<i>Fig 4.3.2 –3 Matricea de confuzie pentru Random Forest TF-IDF.....</i>	65
<i>Fig 4.3.2 –4 Curba ROC pentru Random Forest TF-IDF</i>	65
<i>Fig 4.3.2 –5 Raport de clasificare pentru Random Forest GloVe</i>	66
<i>Fig 4.3.2 –6 Matricea de confuzie pentru Random Forest GloVe</i>	66
<i>Fig 4.3.2 –7 Curba ROC pentru Random Forest GloVe.....</i>	67
<i>Fig 4.3.2 –8 Raport de clasificare pentru Random Forest Word2Vec</i>	67
<i>Fig 4.3.2 –9 Matricea de confuzie pentru Random Forest Word2Vec</i>	68
<i>Fig 4.3.2 –10 Curba ROC pentru Random Forest Word2Vec.....</i>	68
<i>Fig 4.3.3 –1 Raport de clasificare Multinomial Naive Bayes TF-IDF</i>	69
<i>Fig 4.3.3 –2 Raport de clasificare Multinomial Naive Bayes CountVectorizer</i>	70
<i>Fig 4.3.3 –3 Matricea de confuzie Multinomial Naive Bayes TF-IDF.....</i>	71
<i>Fig 4.3.3 –4 Matricea de confuzie Multinomial Naive Bayes CountVectorizer</i>	71
<i>Fig 4.3.4 –1 Raport de clasificare SVM TF-IDF</i>	72

<i>Fig 4.3.4 –2 Matricea de confuzie SVM TF-IDF</i>	72
<i>Fig 4.3.4 –3 Raport de clasificare SVM GloVe.....</i>	73
<i>Fig 4.3.4 –4 Matricea de confuzie SVM GloVe</i>	73
<i>Fig 4.3.4 –5 Raport de clasificare SVM Word2Vec</i>	74
<i>Fig 4.3.4 –6 Matricea de confuzie SVM Word2Vec</i>	74
<i>Fig 4.3.4 –1 Raport de clasificare KNN TF-IDF</i>	75
<i>Fig 4.3.4 –1 Raport de clasificare KNN GloVe</i>	76
<i>Fig 4.3.4 –1 Raport de clasificare KNN Word2Vec</i>	76
<i>Fig 4.3.6 –1 Raport de clasificare RoBERTa</i>	77
<i>Fig 4.3.6 –2 Curbe de învățare model RoBERTa.....</i>	78
<i>Fig 4.3.6 –3 Matricea de confuzie RoBERTa</i>	78
<i>Fig 4.3.7 –1 Raport de clasificare BERT-base-uncased.....</i>	79
<i>Fig 4.3.7 –2 Matricea de confuzie BERT-base-uncased.....</i>	79
<i>Fig 4.3.7 –1 Raport de clasificare GroNLP/HateBERT.....</i>	Error! Bookmark not defined.
<i>Fig 4.3.7 –2 Matricea de confuzie GroNLP/HateBERT.....</i>	Error! Bookmark not defined.
<i>Figură 5.2-1 Arhitectura aplicației</i>	84
<i>Fig. 5.2.2-1 Structura bazei de date</i>	91
<i>Fig 5.3.1-1 Diagrama cazurilor de utilizare</i>	93
<i>Fig 5.3.1-1 Diagrama de secvență</i>	94
<i>Fig 5.4-1 Planul de testare a aplicației.....</i>	96
<i>Fig 5.5-1 Pagina principală a aplicației</i>	97
<i>Fig 5.2-2 Pagină /find-more</i>	98
<i>Fig 5.5-3 Pagina destinată analizei</i>	99
<i>Fig. 5.5-4 Container informativ pentru utilizatori</i>	99
<i>Fig 5.5-6 Istoric în variantă extinsă</i>	100
<i>Fig 5.5-7 Vizualizare raport generat</i>	101
<i>Fig 5.5-8 Grafic destinat primului model.....</i>	101
<i>Fig 5.5-9 Piechart distribuție comentarii ofensive.....</i>	102
<i>Fig 5.5-10 Grafic clasificator principal.....</i>	103
<i>Fig 5.5-12 Grafic engagement pe zile</i>	104
<i>Fig 5.5-13 Grafic engagement pe ore</i>	104

Listă de tabele

Tabel 1 Logistic Regression	63
Tabel 2 Random Forest.....	68
Tabel 3 Multinomial Naive Bayes.....	71
Tabel 4 Support Vector Machine	74
Tabel 5 K-Nearest Neighbours.....	76
Tabel 6 RoBERTa-base.....	78
Tabel 7 BERT	80

NECLASIFICAT

(pagina albă)

1. Introducere

1.1. Context

Rețelele de socializare reprezintă, în contextul actual, principalele platforme pentru comunicare și informare. Acestea găzduiesc volume mari de date brute încărcate de utilizatori sub formă de text, imagini, videoclipuri și conținut audio, care pot fi analizate pentru a extrage informații valoroase despre comportamentele și impactul acestor platforme asupra utilizatorilor.

TikTok se remarcă printr-o creștere rapidă a popularității, în special în rândul adolescenților din România, cu peste 84% de adolescenți români utilizând platforma zilnic, iar mai bine de jumătate accesând platforma de mai mult de 4 ori pe parcursul acesteia. În medie, tinerii petrec aproximativ două ore jumătate în cadrul aplicației, iar vârsta medie la care aceștia își creează un cont este de 13 ani¹.

Hărțuirea online reprezintă utilizarea tehnologiilor digitale pentru a umili, răni sau provoca suferință unei alte persoane. Aceasta se caracterizează prin comportamente intenționate: adresarea jignirilor, amenințarea, distribuirea de informații private sau jignitoare în spațiul public, care adesea pot afecta negativ sănătatea mintală a victimei, inclusiv prin instaurarea depresiei, anxietății sau chiar izolării sociale².

Întrucât volumul masiv de conținut expus pe rețelele de socializare face imposibilă moderarea exclusiv umană, algoritmi de inteligență artificială sunt utilizați tot mai frecvent pentru analiza automată textelor, imaginilor și videoclipurilor în vederea identificării conținutului problematic. Modelul hibrid, adoptat de platforme precum Meta și Google, implică moderarea proactivă susținută de inteligența artificială, cu revizuire ulterioară umană, și raportează un procent semnificativ de cazuri soluționate înainte de a fi semnalate de utilizatori³.

¹ <https://cursdeguvernare.ro/romania-tiktok-generation-8-out-of-10-teenagers-access-it-daily-average-age-upon-creating-an-account-13-years-old-time-spent-on-the-platform.html>

² <https://www.apa.org/topics/bullying/cyberbullying-online-social-media>

³ <https://pmc.ncbi.nlm.nih.gov/articles/PMC8872854/>

Cu toate acestea, limitările tehnologice rămân considerabile. Eficiența acestor sisteme este limitată de cantitatea, calitatea și diversitatea datelor de antrenament, iar algoritmiile oferă performanțe net superioare în cazul limbilor de circulație internațională, pentru care există resurse lingvistice și modele contextuale bine dezvoltate, capabile să surprindă nuanțele culturale sau dialectele. În aceste condiții, acuratețea procesului de moderare automată variază semnificativ în funcție de limba și contextul cultural analizat.

1.2. Importanța temei

Analiza comportamentului și interacțiunii utilizatorilor pe rețelele de socializare reprezintă un element esențial în detectarea și prevenirea hărțuirii online. Lucrarea de față își propune să sprijine comunitatea științifică prin analiza algoritmilor de învățare automată pentru identificarea acestui tip de interacțiune între utilizatori, realizând comparații pe baza mai multor tehnici de procesare, antrenare și vizualizare a întregului proces.

Aplicația finală reprezintă un instrument facil pentru utilizatorii din mediul online, fiind destinată identificării limbajului instigator la ură și a comportamentelor agresive. Soluția propusă urmărește consolidarea unui mediu digital mai sigur și civilizat, încurajând la interacțiuni respectuase și constructive.

1.3. Obiectivele lucrării

Această lucrare are ca obiectiv proiectarea și implementarea unei platforme web destinată analizei comentariilor postărilor de pe platforma de socializare TikTok. Sistemul informatic se concentrează asupra interpretării reacțiilor și comportamentelor utilizatorilor prin intermediul comentariilor, și nu asupra conținutului postării în sine. Procesul presupune extragerea textului și a metadatelor aferente fiecărui comentariu: data posării, numărul de aprecieri și de răspunsuri la acesta.

În etapa de pregătire a datelor, sistemul se concentrează pe traducerea conținutului în limba engleză, eliminarea zgomotului din conținut, prin curățarea textului și prin conversia emoji-urilor și a emoticoanelor în forma lor text, pentru a captura cât mai fidel mesajul transmis.

Prima clasificare va fi din punct de vedere al sentimentului general pe cele trei categorii: pozitiv, negativ și neutru, precum și calcului ratei medii a fiecăria dintre acestea, ceea ce permite identificarea impactului general al reacțiilor utilizatorilor. În paralel, sistemul utilizează clasificatori specializați în detectarea conținutului ofensator și a sarcasmului, pentru a identifica comentariile ce pot semnala un comportament agresiv.

În final, comentariile care semnalează comportament nepotrivit sunt analizate cu clasificatorul principal al sistemului, care le încadrează în una dintre cele cinci categorii specifice: : limbaj ofensator general (care nu reprezintă neapărat hărțuire), hărțuire pe bază de religie, etnie, gen sau vârstă. Astfel, utilizatorul primește informații clare despre natura comentariilor și are o perspectivă mai clară asupra efectelor lor.

1.4. Etică

Această lucrare nu asigură o detectare 100% precisă a hărțuirii în comentariile de pe rețelele de socializare. De asemenea, sistemul analizează exclusiv postările publice, iar predicțiile generate nu trebuie considerate drept concluzii definitive. Se recomandă ca orice caz identificat automat să fie supus și unei evaluări umane înainte de a se lua măsuri.

2. Noțiuni teoretice

Capitolul descrie conceptele teoretice fundamentale care stau la baza cercetării și dezvoltării aplicației web propuse. Prima parte este dedicată este inteligenței artificiale, cu accent pe metodele de învățare automată și procesarea limbajului natural, în special în contextul analizei sentimentale. A doua parte evidențiază tehnologiile software utilizate pentru implementarea aplicației, incluzând arhitectura bazată pe microservicii, comunicația prin Apache Kafka, containerizarea, precum și framework-urile și bibliotecile utilizate pentru dezvoltarea sistemului informatic.

2.1. Inteligența artificială

Inteligența artificială reprezintă, prin definiție, capacitatea sistemelor computaționale de a realiza sarcini care, în mod tradițional, sunt asociate cu inteligența umană, cum ar fi învățarea, raționamentul, rezolvarea problemelor, percepția și luarea deciziilor [4]. Acest domeniu de cercetare informatică dezvoltă și

studiază algoritmi și modele care permit agenților inteligenți să își perceapă mediul și să efectueze acțiuni menite să maximizeze șansele de a-și atinge cu succes obiectivele. Acești algoritmi folosesc volume mari de date și tehnici avansate, cum ar fi învățarea automată, învățarea profundă și prelucrarea limbajului natural.

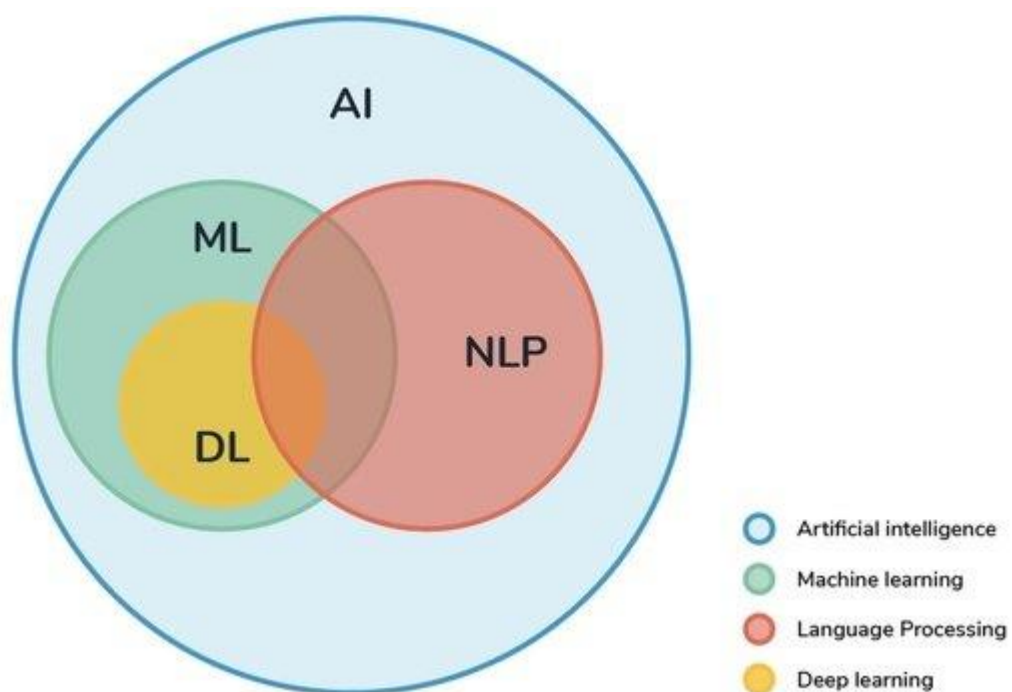


Fig 2.1. Componentele inteligenței artificiale și relațiile dintre acestea

2.1.1. Provocările actuale în domeniul inteligenței artificiale

Provocările actuale în domeniul inteligenței artificiale sunt multiple și complexe, cele mai frecvent menționate în contextul actual fiind protecția datelor personale și confidențialitatea, transparența deciziilor algoritmice, precum și gestionarea tendințelor care poate conduce la rezultate discriminatorii. De asemenea, interpretabilitatea și explicabilitatea modelelor devin esențiale pentru creșterea încrederii utilizatorilor și pentru asigurarea siguranței în utilizarea sistemelor AI. Modelele necesită volume mari de date diverse, aspect care ridică provocări legate de colectarea, preprocesarea și calitatea acestora, mai ales în cazul limbilor sau domeniilor cu resurse limitate.

În domeniul procesării limbajului natural, în special analiza sentimentelor, complexitatea limbajului uman adaugă o nouă dimensiune provocărilor. Modelul

trebuie să interpreteze corect sentimente exprimate subiectiv, sub diverse forme și limbi, fiecare cu particularitățile sale. De altfel, exprimarea și stilul uman sunt în continuă schimbare, iar modelele necesită actualizate constant pentru a reflecta aceste transformări.

Aspectul multilingv reprezintă în continuare una dintre provocări, resursele și date utilizate în antrenările modelelor fiind în principal în limba engleză. Traducerea automată a textelor din alte limbi reprezintă opțiunea cea mai rapidă, însă poate duce la pierderea semnificației originale a sentimentelor exprimate în aceste limbi [6]. De asemenea, valența și interpretarea emoji-urilor, care au devenit un mijloc comun de exprimare a sentimentelor în comunicarea digitală, necesită atenție specială. Includerea și interpretarea acestora pot aduce un plus de exprimare în contextul analizei, dar de asemenea pot foarte ușor conduce la predicții eronate.

Antrenarea modelelor pe seturi diverse de date este crucială pentru a asigura o acuratețe ridicată a analizei sentimentelor. Totuși, utilizarea unor seturi de date care conțin limbaje sau stiluri de comunicare specifice anumitor grupuri demografice, domenii specifice sau subiecte tematice pot introduce subiectivitate în analiza finală și pot afecta obiectivitatea predicțiilor și pot conduce la discriminare în interpretarea sentimentelor [5].

În plus, ambiguitatea cuvintelor, negația polisemia și sarcasmul reprezintă provocări semnificative. Un cuvânt poate avea mai multe înțelesuri în funcție de context, iar modelele de analiză a sentimentelor trebuie să fie capabile să discearnă între aceste sensuri pentru a determina corect polaritatea sentimentului exprimat.

Toate aceste provocări subliniază situația actuală în domeniul inteligenței artificiale și accentuează necesitatea dezvoltării unor modele de analiză a sentimentelor care să integreze în mod eficient contextul lingvistic, cultural și emoțional al mesajelor, pentru a asigura o predicție cât mai corectă a sentimentelor exprimate în comunicarea digitală.

2.1.2. Învățarea automată

Învățarea automată este o ramură fundamentală a inteligenței artificiale care se concentrează pe proiectarea de modele capabile să învețe automat din date, să detecteze tipare și să extragă informații utile fără a fi programate în mod explicit pentru fiecare sarcină specifică. Prin analizarea unor seturi de date mari și diverse,

aceste modele dobândesc capacitatea de a generaliza, în scopul de a oferi predicții precise, de a lua de decizii informate și de a recunoaște tipare în situații noi, contribuind astfel la rezolvarea unor probleme complexe în diverse domenii.

Considerăm un set de date:

$$D = \{(x_i, y_i)\}_{i=1}^n \quad (1)$$

unde $x_i \in X$ reprezintă vectorul de caracteristici și $y_i \in Y$ reprezintă valoarea țintă, eticheta asociată. Scopul învățării automate este de a găsi o funcție:

$$f: X \rightarrow Y \quad (2)$$

care să aproximeze relația dintre vectorii de caracteristici, datele de intrare, și valorile țintă, datele de ieșire, astfel încât, pentru un nou vector x , să poată prezice corect valoarea y .

Procesul de învățare constă în optimizarea funcției de pierdere:

$$L(y, f(x)) \quad (3)$$

care măsoară diferența dintre predicțiile modelului și valorile reale. Această optimizare se exprimă ca:

$$\hat{f} = \arg \min_f \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) \quad (4)$$

Unde $\hat{f}$ este funcția optimă care minimizează eroarea medie pe întregul set de date de antrenament.

2.1.3. Învățarea profundă

Învățarea profundă este o subramură a învățării automate care utilizează rețele neuronale capabile să învețe reprezentări ierarhice și abstracte ale datelor de intrare. Aceste rețele sunt construite prin suprapunerea mai multor straturi de neuroni interconectați. Fiecare strat preia rezultatul stratului anterior și aplică o transformare non-liniară, ceea ce îi permite rețelei să capteze relații complexe și să genereze predicții precise pe volume mari de date .

Un strat este alcătuit dintr-o succesiune de neuroni, fiecare neuron din stratul curent fiind conectat la toți neuronii din stratul precedent și, implicit, influențând neuronii din stratul următor. Fiecare neuron primește un vector de intrări și efectuează combinație liniară ajustată cu coeficienți numiți ponderi, peste care se aplică o funcție de activare pentru a introduce non-liniaritate în model.

Ponderile θ sunt valori numerice ajustate continuu pe durata antrenării rețelei, în scopul de a minimiza eroarea dintre predicțiile modelului și valorile reale. Această optimizare se realizează prin minimizarea funcției de pierdere:

$$L(y, f(x; \theta)) \quad (5)$$

Funcția rețelei, notată ca $f(x; \theta)$, are forma:

$$f(x; \theta) = f^{(D)}(\dots f^{(2)}(f^{(1)}(x)) \dots) \quad (6)$$

unde D reprezintă numărul de straturi ale rețelei.

Astfel, învățarea profundă constă în ajustarea progresivă a parametrilor rețelei pentru a îmbunătăți performanța modelului pe datele de antrenament, permițând generalizarea pe date noi.

Complexitatea unui model de învățare profundă este determinată de numărul de straturi neuronale pe care le conține. Primul strat, numit strat de intrare, preia datele brute, iar ultimul strat oferă rezultatul, corespunzător sarcinii de învățare. Între aceste două niveluri se află mai multe straturi ascunse, care pot avea diverse roluri, printre care:

- *Stratul de încorporare (Embedding Layer)*: transformă datele de tip text în reprezentări vectoriale, capabile să surprindă semnificații și relații între cuvinte.
- *Stratul recurent (Recurrent Layer)*: procesează date secvențiale și reține informații din pașii anteriori pentru modelarea contextului.
- *Stratul de atenție (Attention Layer)*: atribuie ponderi diferențiate părților relevante ale datelor de intrare pentru generarea datelor de ieșire.

- *Stratul de abandon (Dropout Layer)*: utilizează o tehnică de regularizare prin dezactivarea aleatorie a unui procent de neuroni în timpul antrenării, pentru a preveni supraînvăţarea.
- *Stratul convoluţional (Convolutional Layer)*: extrage trăsături locale prin aplicarea de filtre pe date şi este folosit frecvent în procesarea imaginilor.
- *Max pooling*: O operaţiune ce reduce dimensiunea datelor prin selectarea valorii maxime într-o regiune locală, păstrând astfel cele mai importante caracteristici.

Printre funcţiile de activare comune aplicate în fiecare strat se numără:

- *Funcţia treaptă unitate*: este cea mai simplă funcţie, dar rar folosită în reţele moderne.

$$f(x) = \begin{cases} 0, & \text{dacă } x < 0 \\ 1, & \text{altfel} \end{cases} \quad (7)$$

- *Funcţia liniară*: direct proporţionalitate între ieşire şi intrare

$$f(x) = x \quad (8)$$

- *Funcţia sigmoid (logistică)*: utilizată în clasificarea binară, restrânge ieşirea în intervalul (0,1)

$$f(x) = \frac{1}{1 + e^{-x}} \quad (9)$$

- *Funcţia ReLU (Rectified Linear Unit)*: transformă valorile negative în zero şi păstrează valorile pozitive neschimbate, ceea ce ajută reţeaua să înveţe mai eficient.

$$f(x) = \max(0, x) \quad (10)$$

- *Funcţia softmax*: normalizează un vector de valori reale în distribuţie de probabilităţi, cu suma tuturor ieşirilor egală cu 1. Folosită frecvent în ultimul strat pentru clasificare multi-clasă.

$$f(x_i) = \frac{e^{x_i}}{\sum_k x^k} \quad (11)$$

2.1.4. Învățarea supervizată

Învățarea supervizată este o tehnică fundamentală de învățare automată, în care un model este antrenat pe un set de date etichetate. Fiecare exemplu de antrenament este format din perechea (x_i, y_i) , unde x_i este intrarea și reprezintă vectorul de caracteristici, iar y_i este ieșirea și respectiv, eticheta reală a acestui vector. Acest proces permite modelului să învețe o funcție

$$f: X \rightarrow Y \quad (12)$$

pe baza relațiilor din datele de antrenament.

Datele etichetate oferă modelului un punct de referință clar pentru a evalua performanță în sarcina specifică și îi permite să ajusteze parametrii pentru a optimiza rezultatele conform obiectivelor definite de etichete. Astfel, modelul învață să asocieze corect caracteristicile de intrare cu rezultatele așteptate, facilitând o generalizare mai bună pe date noi.

Învățarea supervizată se împarte în două tipuri de probleme, în funcție de natura variabilei țintă:

- Clasificarea: este o problemă de învățare supervizată în care scopul este de a învăța o funcție:

$$f: X \rightarrow C \quad (13)$$

unde C reprezintă un set finit de clase discrete. Modelul primește un vector de caracteristici x_i și atribuie o clasă $\hat{y}_i \in C$, unde $\hat{y}_i$ reprezintă eticheta prezisă de model pentru respectivul vector de caracteristici. Procesul constă în găsirea unor limite clare între clase, astfel încât să se reducă cât mai mult erorile de clasificare.

- Regresia: este o problemă în care variabila țintă y_i este numerică și continuă. Modelul învață o funcție:

$$f: X \rightarrow \mathbb{R} \quad (14)$$

menită să prezică valori numerice pe baza datelor de intrare.

2.1.5. Învățarea nesupervizată

Învățarea nesupervizată este, de asemenea, o tehnică de învățare automată în care modelul primește un set de date format doar din vectori de caracteristici $x_i \{x_i\}_{i=1}^n$, fără a avea asociate etichete sau valori țintă. Prin acest proces, modelul învață să grupeze datele în funcție de legăturile pe care le identifică între ele.

Aceste modele pot fi considerate funcții de tip

$$g: X \rightarrow Z \quad (15)$$

unde Z reprezintă un spațiu de caracteristici de dimensiune redusă sau un spațiu latent, în care datele sunt reprezentate astfel încât relațiile și structurile interne ale acestora să fie mai ușor de identificat și analizat. Funcția g are ca scop să transforme datele într-o reprezentare în Z prin reducerea complexității spațiului original, păstrând informația esențială și eliminând zgomotul și redundanța acestora.

Învățarea nesupervizată se împarte în patru tipuri de probleme, în funcție de cerințe:

- Clusterizarea: gruparea datelor pe baza similarităților și diferențelor dintre acestea.
- Reducerea dimensionalității: transformarea datelor prin micșorarea numărului de atribute, păstrând informația relevantă, fără a pierde integritatea acestora.
- Asocierea: identificarea relațiilor între variabilele datelor.
- Detectarea anomaliilor: identificarea datelor care diferă semnificativ față de restul setului de date.

2.2. Procesarea limbajului natural

Procesarea limbajului natural este un domeniu al inteligenței artificiale care se concentrează pe dezvoltarea de algoritmi și modele ce permit sistemelor informatice să înțeleagă și să proceseze limbajul uman, fie în formă scrisă sau vocală. Prin combinarea tehnicilor de lingvistică computațională cu învățarea automată și rețele neuronale, NLP facilitează interpretarea datelor lingvistice și extragerea de informații valoroase din surse nestructurate.

Această disciplină joacă un rol crucial în contextul digitalizării masive, permițând, printre altele, analiza sentimentului, clasificarea textului, recunoașterea entităților și construirea modelelor care stau la baza aplicațiilor moderne de inteligență artificială, cum ar fi asistenții virtuali și chatbotii.

2.2.1. Tehnici de preprocesare a textului

Pentru a asigura o procesare eficientă a datelor textuale și pentru a le pregăti în vederea antrenării modelelor, este necesară aplicarea unor tehnici variate de curățare și preprocesare. Acestea transformă datele din forma lor brută într-un format uniform, coerent și adecvat pentru modelele de învățare.

În funcție de cerințele specifice modelului utilizat, putem începe etapa de preprocesare cu tehnici de curățare a datelor care includ eliminarea semnelor de punctuație, a adreselor URL și a mențiunilor utilizatorilor, a caracterelor non-ASCII, precum și a cuvintelor de legătură, care nu aduc valoare semnificativă în sarcinile de clasificare. De asemenea, se pot efectua corecturi ortografice, extinderea abrevierilor, iar elementele non-textuale, cum ar fi emoji-urile, pot fi convertite în text care reflectă semnificația acestora, pentru a facilita procesarea ulterioară.

Un alt pas important în preprocesarea textului îl constituie reducerea variațiilor lexicale prin tehnici precum stemming-ul și lematizarea. Stemming-ul elimină afixe pentru a extrage rădăcina cuvintelor, fără a ține cont de contextul gramatical. În schimb, lematizarea folosește informații morfologice și contextuale pentru a transforma cuvântul în forma sa de bază, canonică. De asemenea, tokenizarea, care descompune textul în unități mai mici, numite token-uri, este o etapă esențială în procesarea limbajului natural.

Algoritmii și modelele de învățare procesează date în format numeric, prin urmare, vectorizarea este o etapă importantă. Aceasta constă în transformarea datelor din format text în reprezentări numerice, prin codificarea informațiilor semantice și sintactice din text într-un spațiu numeric. Vectorizarea nu se limitează doar la conversia într-un format numeric, ci și la capturarea înțelesului semantic al cuvintelor și permite, de asemenea, reducerea și dimensiunii datelor, sporind eficiența.

Metodele de vectorizare pot fi împărțite în două mari categorii: tehnici bazate

pe frecvența cuvintelor și tehnici bazate pe învățarea reprezentărilor distribuite.

TF-IDF (Term Frequency - Inverse Document Frequency) este o tehnică ce cuantifică importanța unui termen într-un document, raportat la corpusul întreg. Frecvența Termenului (TF) măsoară cât de des apare un cuvânt într-un document, adică importanța cuvântului:

$$TF(c, d) = \frac{\text{numărul de cuvinte } c \text{ din document}}{\text{numărul total de cuvinte din document}} \quad (16)$$

în timp ce Frecvența Inversă a Documentului (IDF) reduce greutatea cuvintelor comune, frecvent întâlnite în întreg corpusul, măsurând importanța unui cuvânt într-un document, pentru a evidenția termeni mai relevanți contextual:

$$IDF(c, d) = \ln \frac{N}{\text{numărul de apariții a cuvântului } c \text{ în corpusul de documente} + 1} \quad (17)$$

Produsul TF-IDF cuantifică importanța relativă a unui termen în cadrul unui document, penalizând termenii frecvenți în tot corpusul.

$$TF - IDF = TF * IDF \quad (18)$$

Word2Vec este o tehnică prin care se obține o reprezentare vectorială a cuvintelor, prin învățarea reprezentărilor vectoriale dense ale cuvintelor, care capturează semnificația lor pe baza contextului local, adică a cuvintelor învecinate.

GloVe (Global Vectors for Word Representation) combină avantajele modelelor bazate pe frecvența cuvintelor și ale celor bazate pe context, prin învățarea vectorilor de cuvinte pe baza matricelor de co-apariție la nivel global în corpus. Această abordare asigură că reprezentările încorporate reflectă relații statistice semnificative între cuvinte, conservând atât informații locale, cât și globale despre limbaj.

Astfel, aplicarea corectă a tehnicilor de preprocesare și vectorizare este fundamentală pentru obținerea unei reprezentări clare și relevante a datelor textuale, pregătindu-le pentru analize și modele eficiente în etapele ulterioare. Alegerea metodelor potrivite depinde de natura datelor și de obiectivele specifice ale aplicației,

iar înțelegerea aprofundată a mecanismelor acestor tehnici contribuie decisiv la succesul sistemelor bazate pe procesarea limbajului natural.

2.3. Modele de învățare automată utilizate

În procesarea limbajului natural, performanța sistemelor depinde în mare măsură de modelele de învățare automată aplicate. Pentru a identifica cele mai potrivite metode, am explorat o gamă variată de modele, de la tehnici clasice, simple și interpretabile, până la modele avansate bazate pe rețele neuronale profunde și arhitecturi transformer. Această diversitate a permis evaluarea eficienței și adaptabilității fiecărui model în funcție de specificul datelor și cerințele aplicației.

În secțiunile următoare, voi descrie pe scurt modelele alese și particularitățile fiecăruia în contextul procesării limbajului natural.

2.3.1. Logistic Regression

Regresia logistică este un algoritm de învățare automată supravegheată care determină probabilitatea ca o instanță să aparțină unei clase folosind o funcție logistică. În clasificarea multiclasă, modelul este extins prin antrenarea unui clasificator separat pentru fiecare clasă, pentru a estima probabilitățile fiecărei clase și a atribui eticheta cu probabilitatea cea mai mare.

2.3.2. Random Forest

Random Forest este un algoritm de învățare automată supravegheată bazat pe ansambluri de arbori de decizie independenți. Fiecare arbore antrenat pe un subset diferit al datelor. Pentru clasificare, rezultatele acestora sunt combinate prin vot majoritar, ceea ce crește precizia și stabilitatea predicțiilor, reducând riscul de supraînvățare.

2.3.3. Multinomial Naive Bayes

Multinomial Naive Bayes este un algoritm de clasificare supravegheată bazat pe principiul probabilităților condiționate. Acesta estimează probabilitatea ca un exemplu să aparțină unei clase, presupunând că fiecare caracteristică este independentă față de celelalte. Este foarte eficient pentru date discrete, cum ar fi frecvențele cuvintelor într-un text. Algoritmul atribuie clasa cu cea mai mare probabilitate estimată, oferind o clasificare rapidă și simplă.

2.3.4. Support Vector Machine

Support Vector Machine este un algoritm de învățare supravegheată utilizat pentru clasificare, care identifică o frontieră optimă, numită hiperplan, ce separă clasele în spațiul caracteristicilor. Acest hiperplan este ales astfel încât să maximizeze marginea, adică distanța dintre el și cele mai apropiate puncte din fiecare clasă, pentru a asigura o separare cât mai clară și o generalizare bună.

2.3.5. K-Nearest Neighbors

K-Nearest Neighbors este un algoritm de clasificare supravegheată care atribuie o clasă unei instanțe noi pe baza claselor celor k vecini cei mai apropiați în spațiul caracteristicilor, determinați printr-o măsură de distanță, de exemplu, distanța Euclidiană. Performanța algoritmului depinde de alegerea valorii k și de modul în care este măsurată distanța între puncte, fiind eficient în special pentru seturi de date bine delimitate, dar sensibil la zgomot și la date dezechilibrate.

2.3.6. Bidirectional Encoder Representations from Transformers

BERT reprezintă o arhitectură avansată de rețele neuronale, construită pe baza modelului Transformer, care utilizează mecanisme de atenție pentru a analiza contextul cuvintelor în mod bidirecțional, atât înainte, cât și după poziția curentă într-un text. Modelul este preantrenat pe volume extinse de date în mod nesupravegheat, ceea ce îi permite să înțeleagă mai profund relațiile și semnificațiile din limbajul natural. Performanțele sale superioare au revoluționat multe sarcini din procesarea limbajului natural.

2.3.7. Robustly Optimized BERT Approach

RoBERTa este o variantă îmbunătățită a modelului BERT, ce beneficiază de un antrenament extins pe seturi de date mult mai mari și de ajustări fine ale parametrilor de învățare. Această optimizare îi permite să capteze mai bine complexitățile limbajului și să ofere rezultate mai robuste în diverse sarcini de procesare a limbajului natural.

2.4. Evaluarea performanței modelelor

Evaluarea performanței modelelor de învățare automată este esențială pentru a determina cât de bine acestea rezolvă sarcina propusă și pentru a compara diferite metode. În procesarea limbajului natural, se utilizează metrici care măsoară atât corectitudinea clasificării, cât și capacitatea modelului de generalizare și îmbunătățirea acestuia în timpul antrenării.

2.4.1. Matricea de confuzie

Metricile utilizate pentru evaluarea performanței modelelor de clasificare se bazează pe matricea de confuzie. Aceasta este o reprezentare tabelară care compară valorile reale ale etichetelor cu predicțiile modelului, organizând instanțele în funcție de corectitudinea clasificării. Astfel, matricea permite evidențierea numărului de cazuri clasificate corect sau incorect pentru fiecare categorie și constituie fundamentul pentru calculul unor metrici esențiale precum acuratețea, precizia, regăsirea și scorul F1.

		<i>Clasa Prezisă</i>	
		<i>Positive</i>	<i>Negative</i>
<i>Clasa Actuală</i>	<i>Positive</i>	True Positive	False Negative
	<i>Negative</i>	False Positive	True Negative

Figura 2.4.1 – Matrice de confuzie 2x2

Fiecare coloană reprezintă:

- *True Positive (TP)*: numărul de instanțe pozitive corect prezise ca pozitive.
- *True Negative (TN)*: numărul de instanțe negative corect prezise ca negative.
- *False Positive (FP)*: numărul de instanțe negative greșit prezise ca pozitive.
- *False Negative (FN)*: numărul de instanțe pozitive greșit prezise ca negative.

2.4.2. Acuratețe, precizie, regăsire, Scorul F1

Acuratețea exprimă proporția predicțiilor corecte, atât pozitive cât și negative, raportat la totalul predicțiilor realizate de model.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (19)$$

Precizia măsoară proporția datelor clasificate corect ca fiind pozitive din totalul predicțiilor prezise ca pozitive de către model.

$$Precision = \frac{TP}{TP + FP} \quad (20)$$

Regăsirea indică proporția instanțelor pozitive identificate corect de către model, din totalul instanțelor cu etichetă pozitivă.

$$Recall = \frac{TP}{TP + FN} \quad (21)$$

Scorul F1 calculează media armonică dintre precizie și regăsire și oferă un echilibru între cele două atunci când există un compromis între ele. Este util în special în cazul unor clase dezechilibrate.

$$F1 - Score = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (22)$$

2.4.3. Validarea modelelor

Curba ROC (Receiver Operating Characteristic) este un instrument utilizat pentru evaluarea modelelor și este constituită dintr-o reprezentare grafică care ilustrează performanța unui model prin compararea ratei predicțiilor pozitive adevărate cu rata predicțiilor pozitive false la diverse praguri de decizie.

Suprafața de sub curbă, numită AUC (Area Under the Curve), oferă o măsură agregată a performanței modelului: cu cât această suprafață este mai mare, cu atât clasificarea este mai optimă.

Această reprezentare grafică oferă informații esențiale despre performanța modelului, iar analiza acestora ne permite să identificăm comportamente precum felul în care modelul învață și capacitatea sa de generalizare pe date noi:

- *Supraînvățarea (Overfitting)*: apare atunci când modelul învață prea bine datele din setul de antrenament, clasificându-le perfect, dar are performanțe slabe pe date neîntâlnite anterior. Aceasta indică o capacitate redusă de generalizare și necesită tehnici de regularizare sau ajustarea complexității modelului.
- *Subînvățarea (Underfitting)*: apare atunci când modelul este prea simplu pentru a captura relațiile dintre date, ceea ce conduce la performanțe slabe atât pe datele de antrenament, cât și pe cele noi. Modelul nu reușește să învețe suficient din date și necesită o creștere a complexității sau un timp mai lung de antrenare.

2.5. Noțiuni teoretice în dezvoltarea aplicației web

Dezvoltarea aplicațiilor web moderne implică utilizarea unor tehnologii și concepte variate, care asigură scalabilitate, eficiență și o experiență optimă pentru utilizator. În această secțiune sunt prezentate noțiunile teoretice fundamentale care stau la baza arhitecturii și funcționalității aplicației dezvoltate.

2.5.1. Fundamente React

React este un framework JavaScript utilizat pentru dezvoltarea interfețelor vizuale ale aplicațiilor web. Permite crearea de componente reutilizabile și interactive, ceea ce face dezvoltarea mai simplă și ușurința în întreținere. În plus, React este în continuă evoluție, beneficiind constant de actualizări și noi funcționalități, care îl mențin relevant și performant în peisajul dezvoltării web.

2.5.2. Arhitectura REST și FastAPI

Arhitectura REST (Representational State Transfer) reprezintă o manieră de a construi servicii web care facilitează comunicarea între client și server prin intermediul cererilor HTTP. Aceasta oferă o modalitate clară și scalabilă de a interacționa cu datele.

FastAPI¹ este un framework modern utilizat pentru crearea API-urilor REST în Python. Acesta permite dezvoltarea rapidă a serviciilor web și oferă suport pentru validarea automată a datelor și o are o performanță ridicată.

¹ <https://fastapi.tiangolo.com/release-notes/#docs>

2.5.3. Apache Kafka în procesarea fluxului de date

Apache Kafka¹ este un sistem distribuit conceput pentru procesarea și gestionarea fluxurilor de date în timp real. Este utilizat pentru a transmite mesaje între diferite componente ale unui sistem, facilitând un transfer rapid, paralel și sigur de informații. Datele sunt transmise prin intermediul topicurilor, care asigură o organizare clară a fluxurilor de mesaje.

Kafka funcționează pe baza unui model format din 3 roluri principale:

- *Broker*: este un server Kafka care stochează mesajele pe disc și se ocupă de replicarea și distribuția datelor între noduri, astfel asigurând disponibilitatea și reziliența acestora.
- *Producer*: este componenta care se ocupă de generarea și trimiterea mesajelor către unul sau mai multe topicuri din Kafka. Aceștia controlează modul în care datele sunt partiționate și publicate.
- *Consumer*: este componenta care citește mesajele primite de la topic-ul de care aparține, citind mesajele în ordinea transmisă fie individual, fie coordonat în grupuri de consumatori pentru a permite procesarea paralelă.

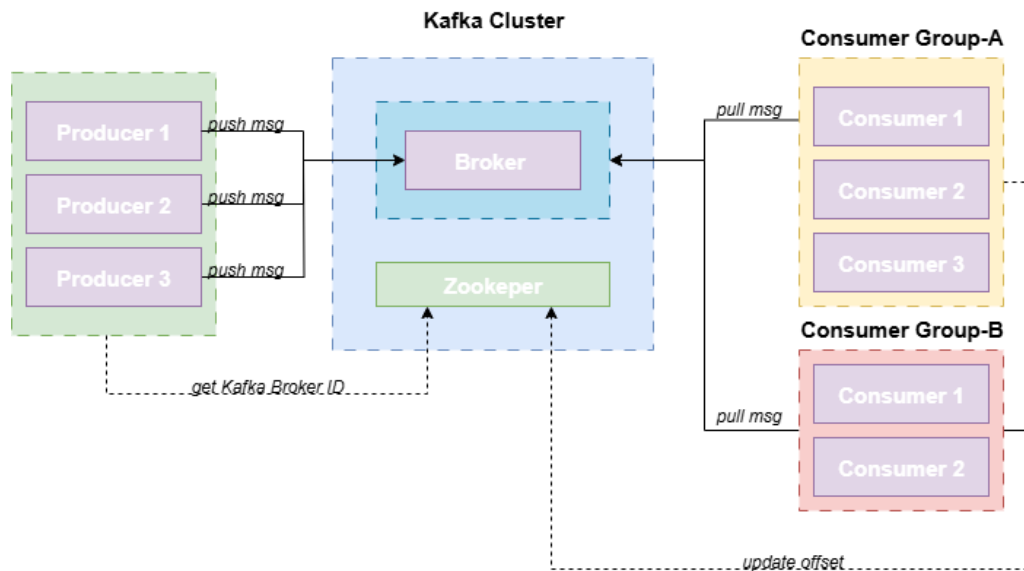


Fig. 2.5.3-1 Apache Kafka

¹ <https://kafka.apache.org/24/documentation.html>

Această arhitectură distribuie și procesează eficient fluxuri mari de date, fiind utilizată pe scară largă în sisteme de procesare în timp real.

2.5.4. Containerizare și orchestrare cu Docker și Docker Compose

Containerizarea este o metodă prin care aplicațiile sunt izolate și ambalate alături de toate bibliotecile, fișierele și resursele de care au nevoie pentru a funcționa corect. Aceasta permite rularea aplicației în mod identic pe orice sistem, oferind consistență, portabilitate și un control mai bun asupra mediului de execuție, fără a depinde de configurația sistemului gazdă.

Docker este una dintre cele mai populare platforme de containerizare, permițând dezvoltatorilor să creeze, să distribuie și să ruleze aplicații în containere rapide și eficiente din punct de vedere al resurselor.

Orchestrarea este procesul prin care mai multe containere sunt gestionate și coordonate astfel încât să colaboreze eficient, să fie pornite, oprite, scalate sau înlocuite automat în funcție de nevoie. Docker Compose este un instrument care simplifică acest proces, permițând definirea și configurarea serviciilor multiple într-un singur fișier, astfel încât acestea să poată fi pornite și administrate împreună.

2.5.5. Gestionarea datelor nerelaționale cu MongoDB

MongoDB este o bază de date non-relațională care stochează datele în documente JSON flexibile, facilitând astfel gestionarea datelor semi-structurate și scalarea orizontală. MongoDB Compass este interfața grafică oficială pentru MongoDB, permițând vizualizarea, interogarea și manipularea datelor din colecții într-un mod intuitiv, fără a scrie direct cod de interogare. Cu ajutorul MongoDB Compass, se poate analiza structura datelor, se poate optimiza performanța interogărilor și gestiona indexările într-un mod eficient.

3. Studii și cercetări relevante în dezvoltarea proiectului

Rețelele de socializare au devenit un mediu esențial pentru comunicare și interacțiune socială, având un impact semnificativ asupra comportamentului utilizatorilor, în special în rândul adolescenților. Cu toate acestea, odată cu creșterea utilizării acestor platforme, a crescut și fenomenul de hărțuire online, cunoscut sub

denumirea de cyberbullying. Studiile au demonstrat că expunerea la cyberbullying poate avea efecte negative asupra sănătății mintale, inclusiv anxietate, depresie și chiar gânduri suicidare, în special în rândul tinerilor [9]. De asemenea, în perioada pandemiei COVID-19, utilizarea rețelelor de socializare a crescut semnificativ, iar acest lucru a fost însoțit de o intensificare a cazurilor de hărțuire online [10].

Conform studiului *Cyberbullying Detection: Exploring Datasets, Technologies and Approaches on Social Media Platforms* realizat în anul 2024, modelele de inteligență artificială reprezintă un mecanism automat util pentru a preveni și monitoriza acest fenomen, dar implementarea acestora vine cu diverse provocări.

Printre cele mai frecvente provocări se numără și lipsa diversității lingvistice, dezechilibrul între clase și accesul limitat la date provenite din platforme relevante precum Facebook sau Instagram. Majoritatea seturilor de date disponibile sunt colectate de pe Twitter, datorită accesibilității API-urilor. Limba engleză este predominantă în aceste colecții, ceea ce reduce eficiența modelelor în alte limbi și contexte culturale. În plus, multe dintre aceste seturi de date sunt dezechilibrate, cu majoritatea instanțelor fiind etichetate ca non-abuzive, iar aceste inconsistențe în procesul de etichetare afectează precizia generală a clasificatorilor [6].

În analiza realizată de autorii studiului menționat anterior, metodele supervizate s-au dovedit a fi superioare celor nesupervizate în detectarea comportamentului abuziv online. Algoritmul Support Vector Machines, în combinație cu vectori de caracteristici obținuți prin metode precum TF-IDF, oferă printre cele mai bune rezultate scoruri F1 între 80% și 85%, fiind considerat unul dintre cele mai fiabile dintre abordările tradiționale. Cu toate acestea, metodele clasice întâmpină dificultăți în procesarea volumelor mari de date și tind să obțină o acuratețe mai scăzută. Modelele BERT și RoBERTa, pe de altă parte, s-au evidențiat prin acuratețe superioară în cercetările descoperite în acest studiu, atingând scoruri F1 de până la 93% [6].

În ceea ce privește setul de date Cyberbullying Classification, utilizat și în acest proiect, aceasta a fost evaluată într-un alt studiu recent, în care modelul RoBERTa a obținut cea mai ridicată acuratețe, de 92.25%. De asemenea, algoritmul

Random Forest a obținut o acuratețe de 88%, iar SVM a obținut 82%, folosind FastText [7].

În contextul limbilor cu resurse limitate, cum este, de altfel, și limba română, studiile subliniază că cele mai bune rezultate sunt obținute tot cu ajutorul modelelor mari de limbaj. Spre exemplu, modelul AraBERT a stabilit performanțe de vârf în sarcini precum analiza de sentiment, recunoașterea entităților numite și răspunsul la întrebări, în limba arabă. Totuși, acest model a fost antrenat pe date formale, precum Wikipedia și surse de știri, ceea ce limitează aplicabilitatea sa în contextul limbajului informal folosit pe rețelele sociale [11].

4. Dezvoltarea proiectului

4.1. Analiza bazelor de date utilizate

Pentru realizarea proiectului, am utilizat seturi de date preluate din diferite studii realizate pe tema hărțuirii în rețelele de socializare. Seturile de date conțin comentarii provenite din diferite platforme sociale, precum Twitter, Facebook, Instagram și YouTube. Acestea au necesitat prelucrări specifice, adaptate în funcție de caracteristicile fiecărui corpus.

Bazele de date au fost analizate cu fiecare model dezvoltat, în vederea identificării variantei cu cea mai bună aplicabilitate practică pentru aplicația finală. Preprocesarea datelor a fost realizată în două moduri: o abordare mai robustă, specifică clasificatorilor tradiționali, dar și o preprocesare simplificată, menită să păstreze esența textului, pentru modelele cu arhitecturi avansate.

4.1.1. Cyberbullying Classification

Primul set de date utilizat provine de pe Kaggle [12] și conține peste 47,000 de comentarii de pe Twitter. După analiza manuală a acestuia, am observat faptul că acesta provine din postările unei emisiuni americane, *My Kitchen Rules*, față de care utilizatorii au exprimat, în general, atât păreri negative, cât și comentarii hărțuitoare la adresa altor personae.

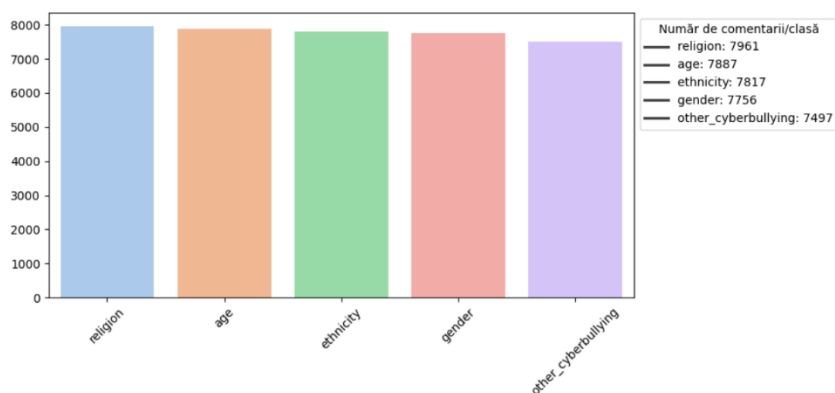
Acesta conține următoarele atribute:

- `tweet_text`
- `cyberbullying_type`

A fost necesară eliminarea clasei *not_cyberbullying* deoarece acesta crea dezechilibru semnificativ în dezvoltarea modelelor datorită calității slabe a datelor

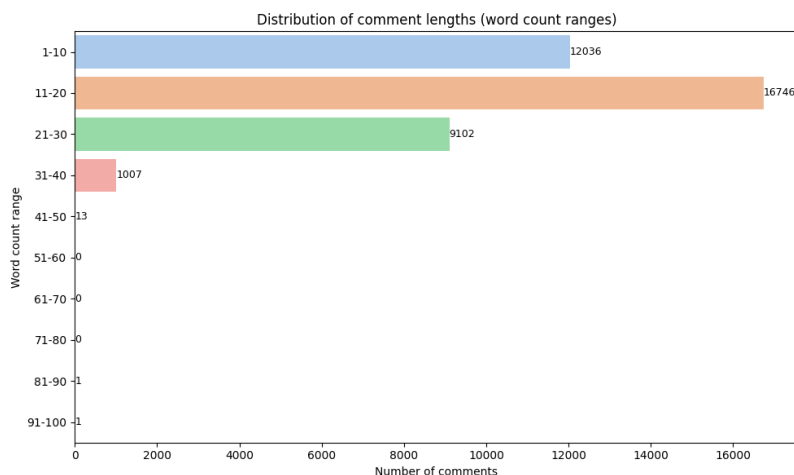
pe care această clasă le avea, iar după preprocesare a rămas corpus format din 38.918 de comentarii distribuite astfel:

- *religion*: 7961 comentarii
- *age*: 7887 comentarii
- *ethnicity*: 7817 comentarii
- *gender*: 7756 comentarii
- *other_cyberbullying*: 7497 comentarii



Figură 4.1.1-1 Distribuția claselor în setul de date

Analizând distribuția comentariilor în funcție de lungimea lor, observăm că majoritatea sunt relativ scurte. Cele mai multe comentarii au între 11 și 20 de cuvinte, iar un număr semnificativ dintre ele au chiar mai puțin de 10 cuvinte:



Figură 4.1.1-2 Distribuția numărului de cuvinte în comentarii

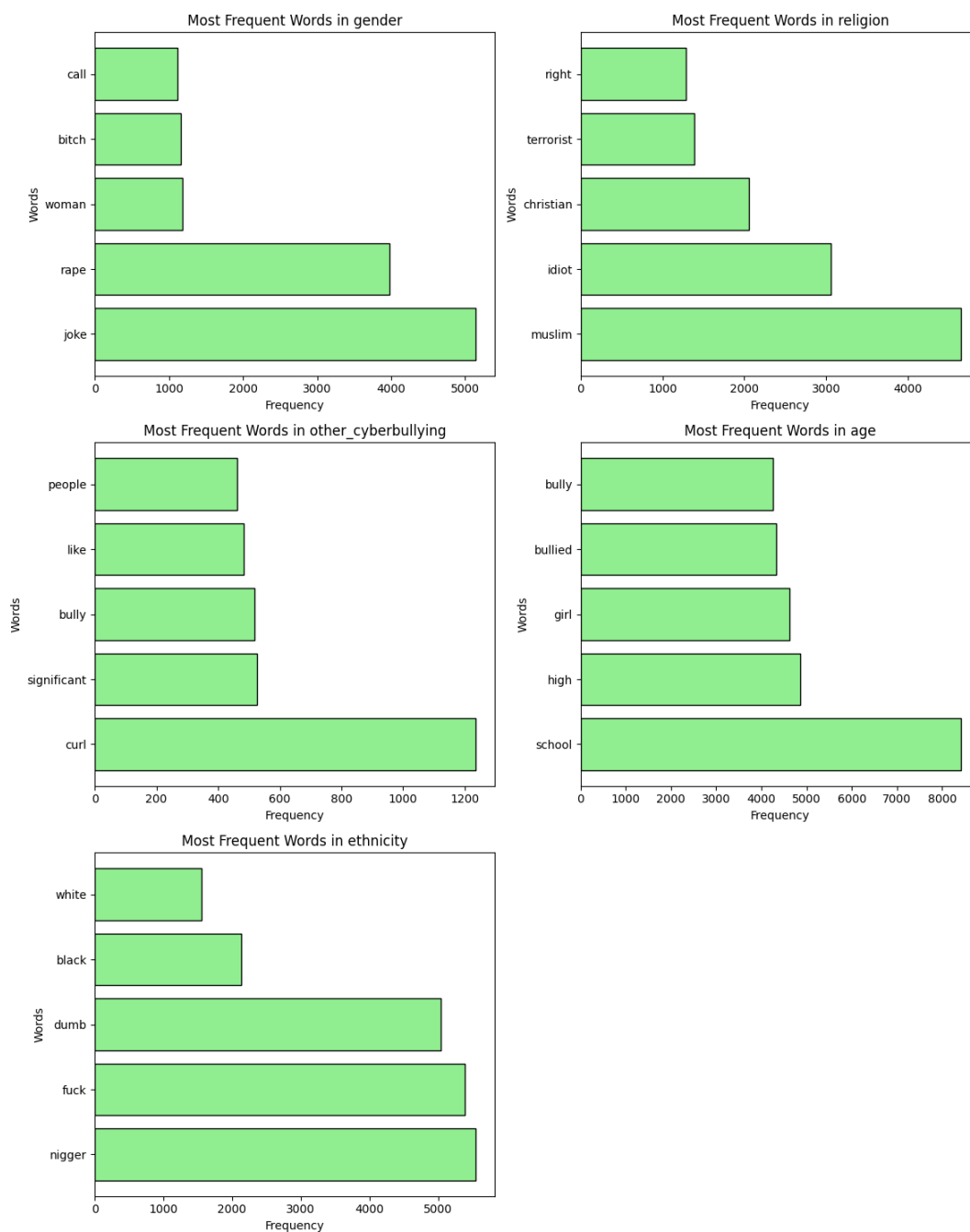
Pentru a înțelege mai bine structura limbajului, am generat un nor de cuvinte. Acesta evidențiază vizual cele mai frecvente cuvinte din setul de date, folosind dimensiuni diferite de font pentru a marca frecvența apariției: cu cât cuvântul este mai folosit, cu atât apare mai mare în reprezentare. Astfel, putem identifica rapid temele și expresiile predominante în setul de date.



Figură 4.1.1-3 Nor de cuvinte

Astfel, din analiza acestuia, putem remarca prezența cuvintelor cu caracter ofensator, precum *idiot*, *bitch* sau *dumb*, dar și o serie de cuvinte de sugerează hărțuiri pe baza etnicității (*nigger* , *black*), a religiei (*muslim*) sau orientate pe gen (*girl*, *men*).

Pentru a evidenția numeric frecvența cuvintelor ofensatoare din baza de date, am analizat distribuția acestora pe fiecare clasă, pentru a observa cum variază limbajul utilizat în funcție de tipul de hărtuire.



Figură 4.1.1-4 Frecvența cuvintelor per clasă

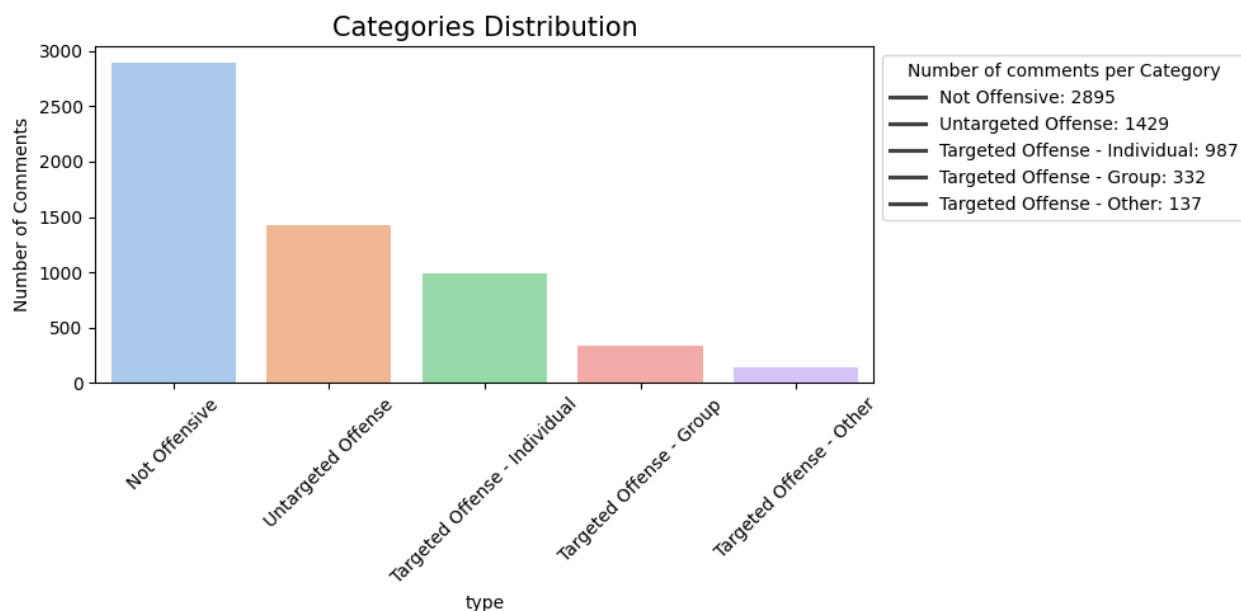
Se poate observa astfel că fiecare clasă conține cuvinte ofensatoare într-o frecvență semnificativ crescută care reflectă tipul de hărțuire pe care îl susțin. Singura clasă pentru care vocabularul nu este la fel de specific și frecvența termenilor ofensatori este considerabil mai redusă și chiar nu foarte relevantă în semnificatii este

clasa care ar trebui să conțină orice alt tip de hărțuire în afară de celelalte patru clase. Acestea din urmă, pe de altă parte, conțin atât un vocabular foarte clar, cât și o frecvență mare a cuvintelor specifice, de ordinul miilor.

Astfel, acest set de date dovedește caracteristici care îi confirmă utilitatea în construirea unor modele eficiente de detectare a hărțuirii online.

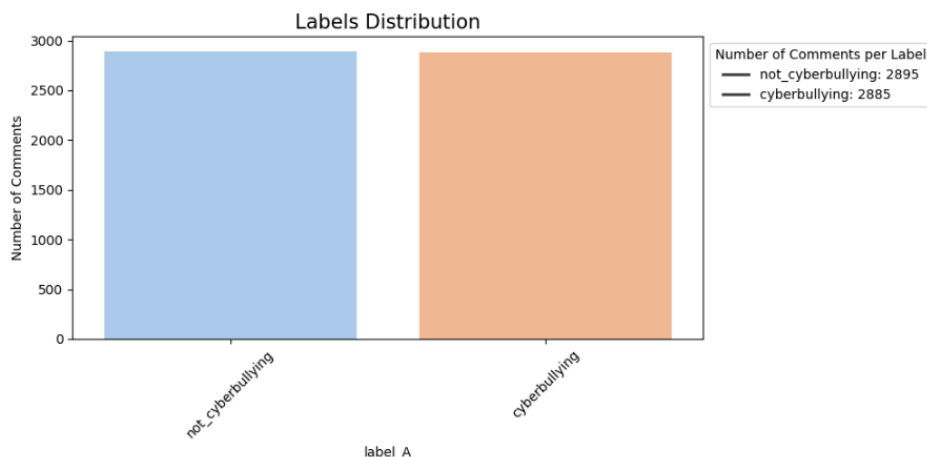
4.1.2. Offensive Language Identification Dataset

Acest set de date provine din competiția *OffensEval*, organizată la conferința *Semantic Evaluation*, care se ocupă cu detectarea limbajului ofensator în social media [13]. A fost creat de către cercetători în anul 2020 și conține aproximativ 6.000 mii de date, distribuite în cinci clase astfel:



Figură 4.1.2-1 Distribuția claselor

Setul de date este dezechilibrat în această formă, însă poate fi transformat într-o clasificare binară echilibrată prin combinarea celor patru clase care descriu diferite forme de hărțuire într-o singură categorie care indică prezența hărțuirii, indiferent de ținta acesteia:



Figură 4.1.2-2 Distribuția binară a claselor

Limbajul poate de asemenea fi evidențiat prin analiza norului de cuvinte generat. Astfel, pot fi observați termeni care subliniază un limbaj ofensator:



Figură 4.1.2-3 Nor de cuvinte

Totuși, această reprezentare generală nu surprinde pe deplin scopul setului de date, și anume evidențierea limbajului ofensator direcționat pe anumite ținte. Din

acest motiv, este necesară și o analiză a frecvenței cuvintelor în funcție de cele cinci clase distincte ale setului de date:



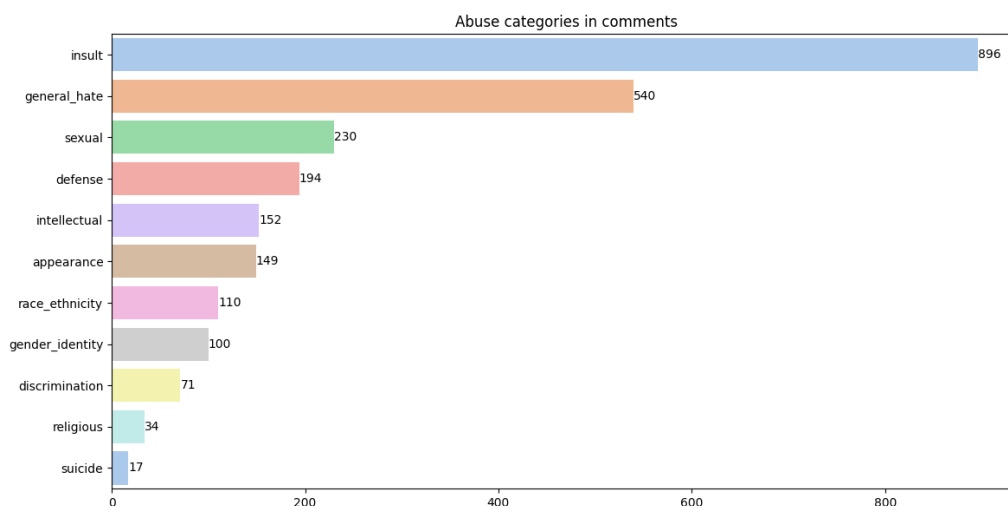
Figură 4.1.2-4 Frecvența cuvintelor per clasă

Nici această analiză nu oferă o vizualizare foarte clară pentru diferențele între clase, însă. Setul de date pare a fi bun pentru a fi folosit în detectarea limbajului ofensiv sau nu, mai mult decât pentru detectarea limbajului targetat, însă dimensiunea

mica a setului de date nu prezintă un aspect pozitiv în continuarea utilizării acestuia în antrenarea modelelor.

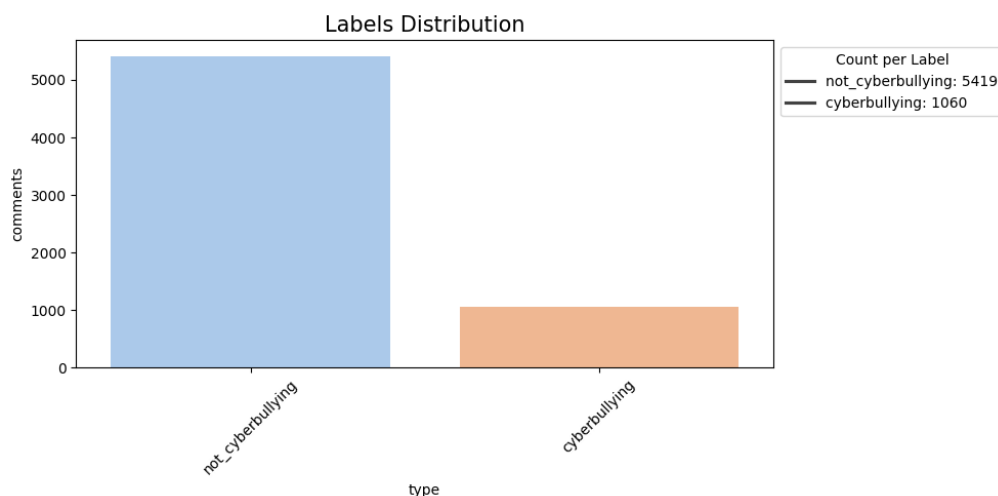
4.1.3. Loyola Univeristy Cyberbullying Dataset

Acest set de date a fost dezvoltat în cadrul unui proiect de cercetare realizat de Univeristatea Loyola din Chicago, în colaborare cu Univeristatea de Stat din Arizona [14]. Comentariile, extrase din postări de pe Instagram, au fost etichetate manual, iar setul de date include adnotări complexe și variate, aplicate în combinație. Distribuția claselor care reflectă fenomenul de hărțuire online este următoarea:



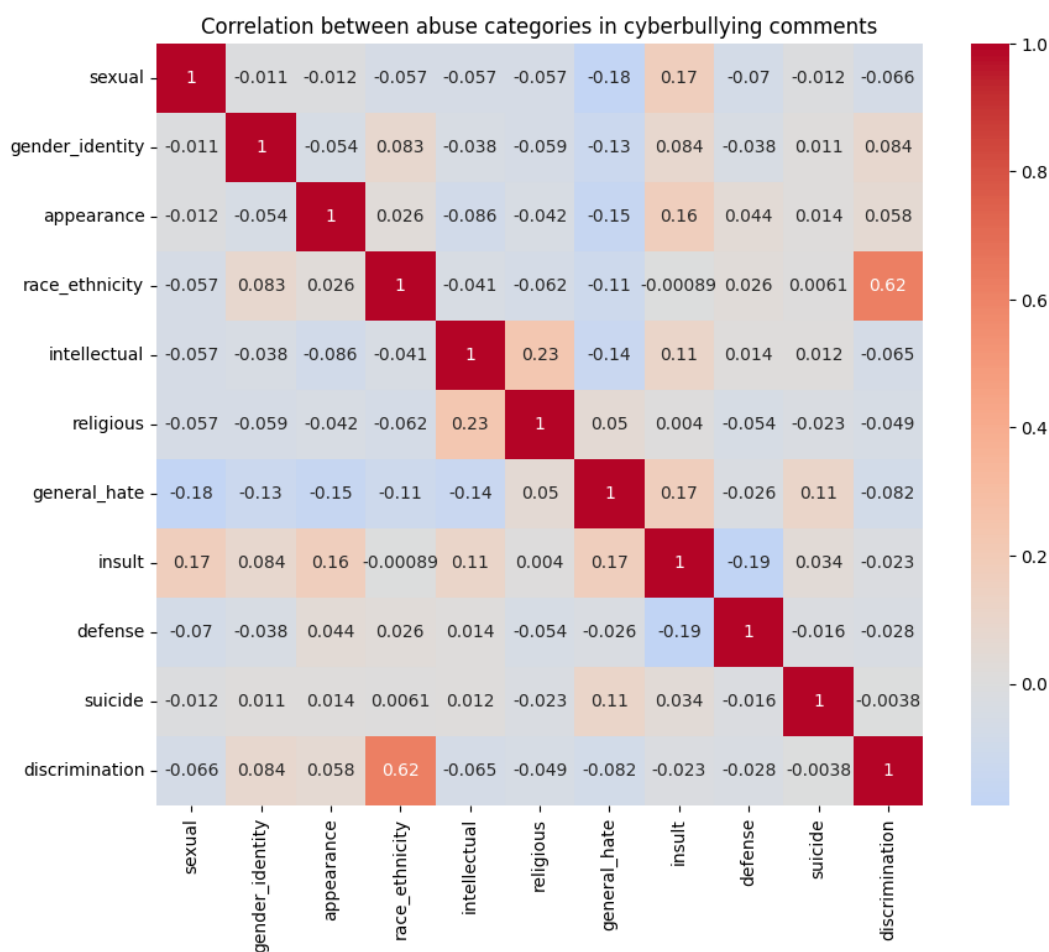
Figură 4.1.3-1 Distribuția claselor

Setul de date conține, însă, și comentarii care nu conțin elemente de hărțuire. Combinând aceste două categorii, setul de date se dovedește în continuare a fi dezechilibrat:



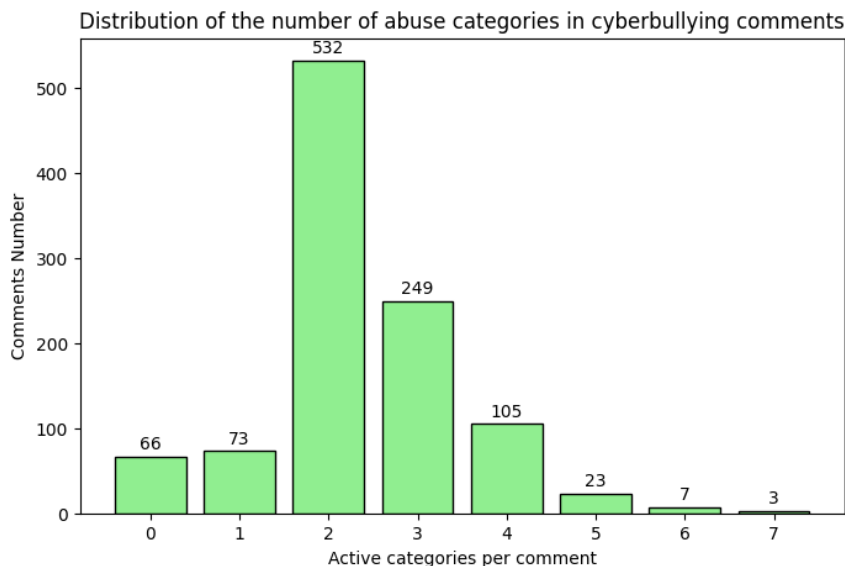
Figură 4.1.3-2 Distribuția binară a claselor

Matricea de corelație a etichetelor ne oferă o imagine clară despre care sunt etichetele utilizate cel mai frecvent împreună, astfel, putem observa că un comentariu discriminator este frecvent și un comentariu care sugerează hărțuire din punct de vedere al etnicității. De asemenea, mai sunt și alte clase asociate, însă cu o frecvență mai redusă, cum ar fi intelectualitatea cu religia, aspectul fizic cu insulta:



Figură 4.1.3-3 Matricea de corelație a claselor

Pentru a evidenția câte etichete sunt atribuite, în general, unui comentariu, am realizat următorul grafic. Acesta arată că majoritatea comentariilor sunt adnotate cu două etichete, iar un număr semnificativ mai mic ajung să primească trei. Celelalte combinații apar într-un număr redus, ceea ce sugerează o predominanță a adnotărilor simple sau moderate în acest set de date:



Figură 4.1.3-4 Distribuția asocierii claselor

Norul de cuvinte oferă o imagine mai clară despre vocabularul utilizat în aceste comentarii:



Figură 4.1.3-5 Nor de cuvinte

Însă, setul de date nu s-a dovedit potrivit pentru antrenarea și implementarea eficientă a modelelor de clasificare, din cauza dimensiunii reduse și a distribuției dezechilibrate a etichetelor, care afectează semnificativ capacitatea modelelor de a generaliza și de a învăța în mod uniform.

4.2. Etapa de pregătire a seturilor de date

4.2.1. Transformări lingvistice preliminare ale datelor

Scopul acestei etape este de a standardiza și a uniformiza textele brute, astfel încât acestea să poată fi interpretate și procesate eficient de către modelele computaționale. Acest proces este, de asemenea, adaptat în funcție de cerințele specifice ale fiecărui model.

○ Traducerea automată

Întrucât resursele NLP pe care le-am utilizat sunt preantrenate exclusiv pe corpusuri în limba engleză, prima etapă în procesarea datelor este uniformizarea lingvistică a seturilor de date. Dat fiind faptul că acestea conțineau comentarii în multiple limbi, primul pas în preprocesare a fost traducerea automată a acestora.

Pentru această etapă am utilizat API-ul *DeepL*¹, care oferă suport pentru peste 40 de limbi și este recunoscut pentru calitatea și corectitudinea traducerilor sale. Fiecare intrare din setul de date a fost mai întâi analizată cu funcția de detectare a limbii oferită de DeepL. Dacă funcția detecta că limba era diferită de engleză, textul era fost tradus automat.

Alegerea traducerii automate în limba engleză a fost justificată de faptul că majoritatea resurselor disponibile, atât la nivel de modele preantrenate, cât și de seturi de date și instrumente de preprocesare, sunt optimizate pentru această limbă. Prin urmare, traducerea a permis alinierea seturilor de date la standardele dominante din domeniul NLP, maximizând compatibilitatea și performanța modelelor utilizate.

○ Normalizarea elementelor non-textuale

Dat fiind faptul că aceste seturi de date comentarii scrise de utilizatori pe diverse rețele de socializare, ce conțin frecvent elemente neconvenționale dificil de interpretat de modelele NLP, am implementat o etapă de conversie a elementelor neconvenționale din text, cum ar fi emoji-urile, emoticoanele și abrevierile.

Pentru conversia emoji-urilor, am utilizat un dicționar predefinit extras dintr-

¹ <https://developers.deepl.com/docs/>

-un proiect open-source de pe GitHub¹, numit *EMO_UNICODE*. Acest dicționar conține identificatori Unicode ale emoji-urilor existente împreună cu semnificațiile lor text asociate. Am extins acest dicționar prin includerea identificatorilor Unicode pentru emoji-urile noi apărute în anul 2025, pentru a asigura actualizarea continuă, ajungând astfel la un total de peste 2400 de emoji-uri.

Acest proces de conversie constă într-o funcție care parcurge tot setul de date, identifică secvențele Unicode corespunzătoare emoji-urilor și le înlocuiește cu semnificația lor text, facilitând astfel interpretarea lor de către modelele NLP.

Conversia emoticoanelor respectă același principiu, dicționarul *EMOTICONS* este preluat din aceeași sursă cu dicționarul emoji-urilor și conține aproximativ 200 de elemente. Am utilizat tot o funcție care parcurge setul de date și înlocuiește emoticoanele identificate cu semnificația lor text.

Pentru abrevieri, am înlocuit termenii prescurtați cu forma lor extinsă, folosind dicționarul *INTERNET_SLANG*, construit manual și care conține peste 300 de elemente. Înlocuirea este realizată cu ajutorul expresiilor regulate, astfel încât să fie insensibilă la majuscule și să respecte limitele de cuvânt, prevenind înlocuirile parțiale sau eronate.

Scopul acestei etape a fost păstrarea și valorificarea informației semantice implicite în elementele neconvenționale ale comentariilor. Această alegere urmărește să îmbunătățească capacitatea modelelor de a capta nuanțele afective și intenționale ale mesajelor. Totuși, trebuie menționat că includerea acestor transformări poate introduce zgomot în date, mai ales în cazul în care interpretarea automată a semnificațiilor este imperfectă, dată fiind sensibilitatea anumitor modele la variații subtile de formulare.

○ **Curățarea datelor**

Această etapă are scopul de a reduce zgomotului pentru a maximiza calitatea semantică a datelor. Prelucrarea a fost adaptată în funcție de arhitectura modelelor, fiind implementată pe două ramuri distincte: una dedicată modelelor clasice de clasificare și cealaltă pentru modelele de tip transformer.

¹ <https://github.com/NeelShah18/emot/blob/master/emot/>

Pentru modelele clasice, care sunt mai sensibile la variații formale și zgomot lexical, a fost necesară o preprocesare riguroasă. Aceasta presupune atât eliminarea elementelor care nu conțin informații care pot contribui la îmbunătățirea modelelor, cum ar fi adresele URL, mențiunile de utilizator, semnele de punctuație și caracterele non-ASCII, cât și simbolurile hashtagurilor, la care am păstrat totuși textul. Repetițiile exagerate de caractere au fost și acestea reduse printr-o expresie regulată la maxim două apariții consecutive, pentru a atenua stilizările excesive frecvent întâlnite în limbajul informal al rețelelor sociale. Textele au fost apoi convertite în litere mici, iar spațierea redundantă a fost normalizată.

În continuare, am utilizat biblioteca *NLTK* pentru a elimina cuvintele fără relevanță semantică, care se află în corpusul numit *STOPWORDS* în această bibliotecă, cu excepția cuvintelor cu potențial negativ, cum ar fi “no”, “not”, “never”. De asemenea, s-a realizat lematizarea cuvintelor, iar erorile ortografice au fost corectate automat utilizând biblioteca *pyspellchecker*, care permite identificarea și corectarea cuvintelor ce nu se regăsesc în vocabularul standard.

În contrast, modelele de tip transformer necesită o preprocesare mai lejeră, întrucât dispun de o capacitate ridicată de a interpreta structuri morfosintactice variate și zgomot textual. În acest caz, s-a păstrat o parte din structura inițială a textului, fiind eliminate doar elementele strict necesare: adresele URL, mențiunile de utilizator, caracterele non-ASCII și simbolurile hashtagurilor. Totodată, s-a efectuat standardizarea spațierii pentru a asigura coerența secvenței de cuvinte.

Toate aceste transformări au fost realizate într-un cadru modular, folosind *pandas* pentru procesarea tabelară a datelor, *re* pentru expresii regulate și *unicodedata* pentru conversia caracterelor accentuate la formele lor echivalente ASCII în vederea eliminării lor. Această abordare a contribuit la obținerea unor seturi de date omogene și semantic relevante, care pot fi eficient valorificate de modelele de învățare automată utilizate în etapa următoare.

4.2.2. Reprezentarea numerică a textului

După etapa de transformare lingvistică a datelor, am utilizat mai multe metode de vectorizare a datelor pentru a observa care dintre acestea oferă cele mai bune rezultate împreună cu clasificatoarele noastre:

○ Vectorizare TF-IDF

Din biblioteca *Scikit-learn*, mai precis modulul *sklearn.feature_extraction.text*, am utilizat clasa *TfidfVectorizer* pentru a transforma textele în reprezentări numerice pe baza schemei de ponderare TF-IDF. Configurarea vectorizatorului a presupus includerea unui interval de n-grame de la 1 la 2, ceea ce înseamnă că au fost luate în calcul atât cuvintele individuale, cât și combinațiile de câte două cuvinte consecutive. Dimensionalitatea spațiului vectorial a fost limitată la 10.000 de trăsături și a fost aplicată și o scalare logarimtică asupra frecvențelor, pentru a reduce influența termenilor foarte frecvenți. Pentru a exclude termenii care apar doar o singură dată în întregul corpus, am inclus parametrul *'min_df=2'*.

Împărțirea setului de date s-a realizat în proporție de 80% pentru antrenare și 20% pentru testare, utilizând stratificarea pe baza etichetelor de clasă. Această tehnică asigură menținerea distribuției claselor în ambele subseturi, ceea ce este esențial în cazul unor clase dezechilibrate.

Pentru datele de antrenament, vectorizarea a fost aplicată prin metoda *fit_transform()*, pentru a învăța vocabularul și a-l și transforma în formă matriceală. Pentru datele de test, am avut nevoie doar de transformarea lor în formă matriceală și aplicarea vocabularului deja învățat, astfel, am aplicat doar metoda *transform()*.

Pentru algoritmi de clasificare care sunt sensibili la scala caracteristicilor, a fost aplicată o etapă suplimentară de normalizare. Această normalizare, realizată cu ajutorul clasei *Normalizer* din biblioteca *scikit-learn*, ajustează fiecare vector astfel încât suma pătratelor valorilor să fie egală cu 1. Astfel, accentul cade pe distribuția relativă a trăsăturilor din text, nu pe valoarea lor absolută.

○ Vectorizare GloVe

Pentru vectorizarea textelor, am utilizat dicționarul preantrenat GloVe, dedicat limbajului specific rețelelor sociale, denumit *'glove.twitter.27B.200d'*. Acest model a fost antrenat pe un corpus ce cuprinde aproximativ 2 miliarde de tweet-uri, însumând peste 27 de miliarde de tokeni, iar fiecare cuvânt este reprezentat printr-un vector cu dimensiunea de 200.

Am încărcat dicționarul preantrenat, atât cuvântul cât și vectorul său asociat și apoi pentru fiecare text din setul de date, am împărțit șirul de caractere în cuvinte

individuale și am extras vectorii corespunzători fiecărui cuvânt din dicționar. Ulterior, pentru a obține o singură reprezentare numerică a întregului text, am calculat media aritmetică a tuturor vectorilor cuvintelor din fiecare intrare, obținând astfel un vector mediu care sintetizează informația conținută în text.

Ulterior, am împărțit datele rezultate în seturi distincte pentru antrenament și testare, urmând aceeași metodă utilizată și în cazul vectorizării TF-IDF.

○ **Vectorizare Word2Vec**

Pentru vectorizarea textelor cu ajutorul modelului Word2Vec, am urmat un proces structurat care implică atât pregătirea datelor, cât și construirea propriu-zisă a reprezentărilor vectoriale.

În primul rând, fiecare text din setul de date a fost procesat printr-o tokenizare simplă, folosind funcția `simple_preprocess` din biblioteca Gensim. Aceasta transformă fiecare propoziție într-o listă de cuvinte, eliminând automat semnele de punctuație și standardizând textul la litere mici, ceea ce este esențial pentru obținerea unei vocabular coerent. După această etapă, am antrenat un model Word2Vec pe baza acestor secvențe de cuvinte.

Modelul a fost configurat astfel încât să genereze vectori de dimensiune 200 pentru fiecare cuvânt, similar cu modelul GloVe utilizat anterior. Am optat pentru arhitectura *skip-gram*, care este cunoscută pentru capacitatea sa de a captura relații semantice mai fine între cuvinte, mai ales în cazul cuvintelor rare. Parametrul `window=5` a fost folosit pentru a defini dimensiunea ferestrei de context, adică numărul de cuvinte din jur care contribuie la învățarea relațiilor semantice. În plus, am exclus cuvintele foarte rare prin setarea `min_count=2`, astfel încât să ne concentrăm doar pe termenii cu apariții relevante în corpus.

După antrenarea modelului, similar cu vectorizarea GloVe, fiecare text a fost reprezentat numeric printr-un vector mediu, calculat pe baza vectorilor cuvintelor existente în respectivul text. Pentru acest lucru, am definit o funcție care extrage vectorii doar pentru cuvintele regăsite în vocabularul modelului și apoi calculează media acestora.

○ **Tokenizare**

Pentru antrenarea modelelor de tip transformer, am utilizat tokenizarea predefinită, specifică fiecărui model în parte. Atât *RobertaTokenizer*, cât și *BertTokenizer*, aparțin bibliotecii *HuggingFace*, și au fost configurate să includă și trunchierea textelor pentru a ne asigura că nu depășesc lungimea maximă prestabilită.

Fiecare text este convertit într-un șir de tokeni numerici, `input_ids`, iar în același timp se generează și masca de atenție, `attention_mask`, care indică ce poziții din secvență conțin cuvinte: valoarea 1 marchează prezența unui token valid, iar valoarea 0 indică pozițiile ocupate de padding. Toate aceste liste de tokeni și măști sunt apoi convertite în tensori PyTorch, pregătindu-le pentru utilizarea în rețeaua neuronală.

4.3. Algoritmi utilizați în detectarea hărțuirii din mediul online

Pentru algoritmii și modelele prezentate în continuare, am utilizat prima bază de date menționată în studiu, *Cyberbullying Classification*, care oferă o clasificare multiclasă în 5 categorii distincte: etnicitate, religie, gen, vârstă sau alt tip de hărțurile, diferit față de cele menționate anterior. Pentru fiecare algoritm, voi detalia metoda de tokenizare și, unde este cazul, tehnica de vectorizare aplicată, evidențiind avantajele și particularitățile fiecărei abordări în contextul specific al detectării acestor tipuri de hărțuire.

Pentru implementarea algoritmilor clasici de clasificare, am pregătit datele prin aplicarea celor trei tehnici de vectorizare: *TF-IDF*, *GloVe* și *Word2Vec*, pentru a observa rezultatele oferite de algoritmi pentru fiecare tehnică în parte. Datele au fost împărțite utilizând o proporție de 80% pentru antrenare și 20% pentru testare, folosind o valoare fixă pentru parametrul `random_state` pentru a asigura reproductibilitatea rezultatelor.

4.3.1. Logistic Regression

Modelul a fost configurat să efectueze până la 1000 de iterații în procesul de optimizare, pentru a oferi suficient timp algoritmului să convergă către o soluție stabilă și precisă. Am ales metoda de rezolvare *saga*, deoarece este eficientă și recomandată în special pentru lucrul cu seturi mari de date, asigurând un echilibru bun între viteza de procesare și acuratețea rezultatului.

Modelul Logistic Regression aplicat pe datele vectorizate cu TF-IDF a obținut rezultate foarte bune în clasificarea tipurilor de hărțuire cibernetică. Acuratețea generală a modelului este de 93%, reflectând o capacitate ridicată de predicție corectă a claselor. Analizând metricele pe fiecare clasă, observăm performanțe remarcabile: clasa *age* a înregistrat un scor F1 de 98%, cu precizie și regăsire apropiate, ceea ce indică o predicție aproape perfectă. Clasele *religion* și *ethnicity* au înregistrat de asemenea scoruri F1 ridicate, de 97% și respectiv 96%, susținute de valori înalte ale preciziei și regăsirii. Pentru clasele *gender* și *other_cyberbullying*, modelul menține un echilibru bun, cu scoruri F1 de 90% și 85%, ceea ce indică o performanță solidă și pentru aceste categorii.

	precision	recall	f1-score	support
gender	0.92	0.87	0.90	1548
religion	0.99	0.95	0.97	1592
other_cyberbullying	0.80	0.91	0.85	1486
age	0.99	0.98	0.98	1577
ethnicity	0.98	0.94	0.96	1563
accuracy			0.93	7766
macro avg	0.93	0.93	0.93	7766
weighted avg	0.94	0.93	0.93	7766

Accuracy: 0.932
Precision: 0.936
Recall: 0.932
F1 Score: 0.933

Fig 4.3.1 –1 Raport de clasificare pentru Logistic Regression TF-IDF

Prin intermediul matricii de confuzie se poate observa confuzia pe care modelul o realizează, în special, între clasa *other_cyberbullying* și *gender*:

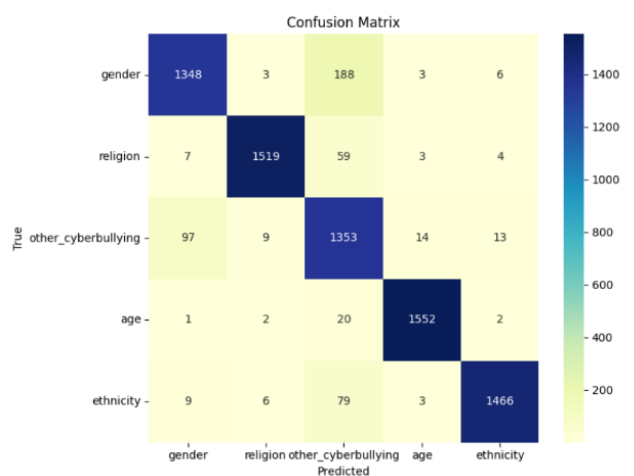


Fig 4.3.1 –2 Matricea de confuzie pentru Logistic Regression TF-IDF

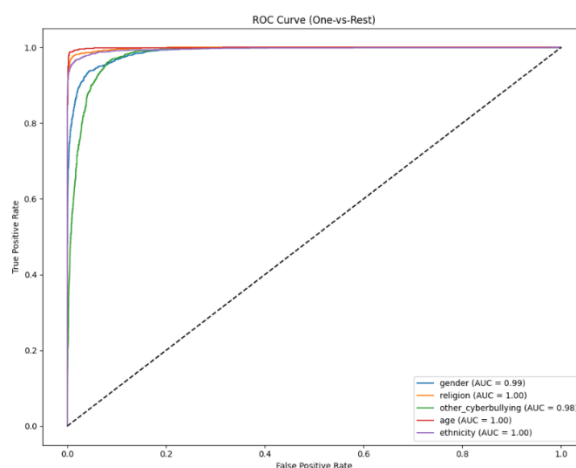


Fig 4.3.2 –3 Curba ROC pentru Logistic Regression TF-IDF

Modelul Logistic Regression aplicat pe datele vectorizate cu GloVe înregistrează rezultate bune, evidențiindu-se în special clasa *age*, pentru care scorul F1 este de 95%. Aceste rezultate arată o performanță solidă, deși ușor inferioară celei obținute cu vectorizarea TF-IDF.

	precision	recall	f1-score	support
gender	0.84	0.82	0.83	1552
religion	0.94	0.93	0.94	1592
other_cyberbullying	0.74	0.77	0.75	1500
age	0.95	0.96	0.95	1577
ethnicity	0.92	0.91	0.92	1563
accuracy			0.88	7784
macro avg	0.88	0.88	0.88	7784
weighted avg	0.88	0.88	0.88	7784

Accuracy: 0.879
Precision: 0.880
Recall: 0.879
F1 Score: 0.880

Fig 4.3.1–4 Raport de clasificare pentru Logistic Regression GloVe

Matricea de confuzie confirmă din nou o tendință observată și în cazul vectorizării TF-IDF, respectiv confuzia frecventă între clasele *gender* și *other_cyberbullying*. Această situație indică faptul că modelul întâmpină dificultăți în a distinge clar aceste două tipuri de hărțuire, ceea ce poate afecta ușor performanța pe aceste categorii specifice.

NECLASIFICAT

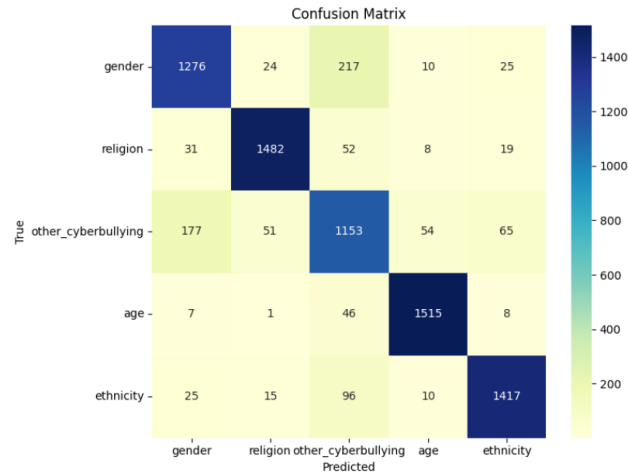


Fig 4.3.1 –5 Matricea de confuzie pentru Logistic Regression GloVe

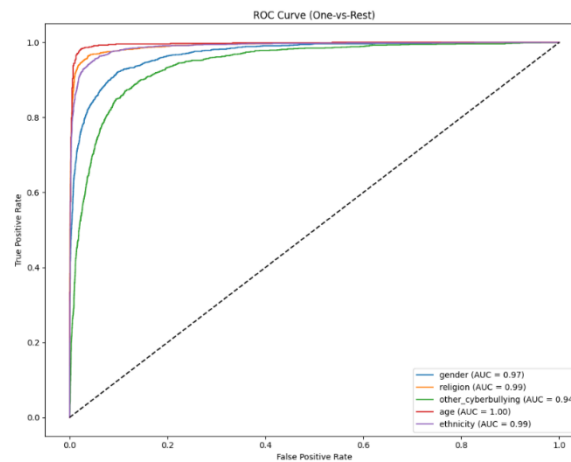


Fig 4.3.2 –6 Curba ROC pentru Logistic Regression GloVe

Modelul logistic antrenat pe date vectorizate cu Word2Vec a obținut rezultate bune, cu o acuratețe generală de aproximativ 90%. Categoria *age* continuă să se remarce prin scoruri foarte ridicate, ceea ce sugerează că această clasă este ușor de identificat pe baza reprezentărilor semantice. Și celelalte categorii înregistrează performanțe solide, menținând un echilibru consistent între metricile de evaluare.

NECLASIFICAT

	precision	recall	f1-score	support
gender	0.88	0.84	0.86	1646
religion	0.94	0.96	0.94	1582
other_cyberbullying	0.77	0.79	0.78	1536
age	0.96	0.96	0.96	1600
ethnicity	0.95	0.95	0.95	1584
accuracy			0.90	7948
macro avg	0.90	0.90	0.90	7948
weighted avg	0.90	0.90	0.90	7948

Accuracy: 0.898
Precision: 0.898
Recall: 0.898
F1 Score: 0.898

Fig 4.3.1–7 Raport de clasificare pentru Logistic Regression Word2Vec

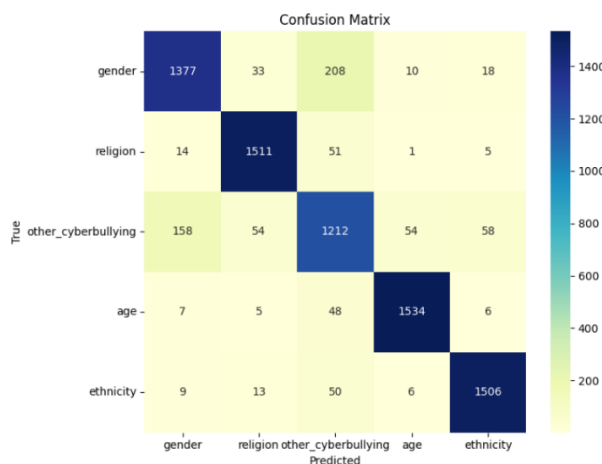


Fig 4.3.1 –8 Matricea de confuzie pentru Logistic Regression Word2Vec

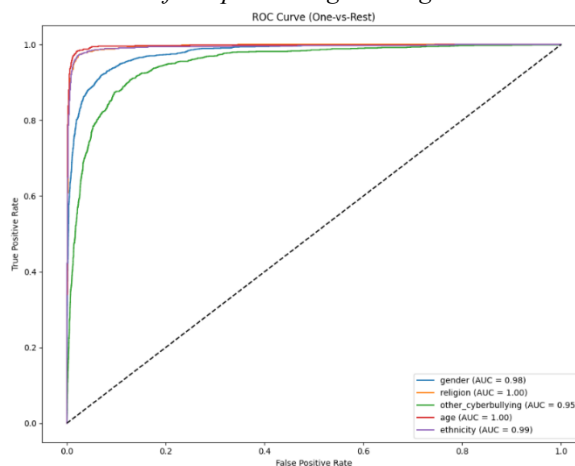


Fig 4.3.2 –8 Curba ROC pentru Logistic Regression Word2Vec

După ce am observat problemele la anumite clase, am încercat îmbunătățirea datelor prin diverse metode, inclusiv augmentarea clasei *other_cyberbullying* și înlocuirea unor exemple cu altele mai relevante. Cu toate acestea, aceste intervenții nu au dus la o îmbunătățire semnificativă a rezultatelor. Am testat, de asemenea, integrarea clasei *not_cyberbullying*, însă datele disponibile pentru această categorie s-au dovedit a fi mai puțin consistente și au afectat negativ performanța generală.

Se poate observa că rezultatele obținute cu algoritmul de regresie logistică, prezentate în Tabelul 1, evidențiază faptul că metoda TF-IDF conduce în mod clar cu cele mai bune scoruri pe toate metricele. Word2Vec urmează în clasament, în timp ce GloVe obține performanțe ușor inferioare.

Tabel 1 Logistic Regression

Nr.	Metodă vectorizare	Acuratețe	Precizie	Regăsire	Scor F1
1	TF-IDF	93%	93%	93%	93%
2	GloVe	88%	88%	87%	88%
3	Word2Vec	90%	89%	89%	89%

4.3.2. Random Forest

Pentru optimizarea performanței modelului Random Forest, am definit o listă de hiperparametri relevanți pe care i-am utilizat ulterior prin aplicarea unei căutări sistematice a celor mai bune combinații ale acestor hiperparametri folosind metoda Grid Search, prin intermediul funcției GridSearchCV din biblioteca Scikit-learn. Această metodă a evaluat toate combinațiile posibile din lista stabilită, folosind o validare încrucișată pe 3 fold-uri și ca metrică acuratețea.

Astfel, am identificat setul optim de hiperparametri care maximizează performanța modelului pe datele de antrenament. Algoritmul a fost configurat cu 200 de estimatori, iar pentru diviziunea nodurilor am selectat log2, care alege un subset de caracteristici egal cu logaritmul în baza 2 al numărului total de caracteristici. Împărțirea nodurilor este restricționată la minimum 15 eșantioane pentru a preveni supraînvățarea, iar fiecare frunză a arborelui trebuie să conțină cel puțin un eșantion, permițând astfel modelului să învețe cu o granularitate fină.

Pentru a ilustra mai clar modul în care setul optim de hiperparametri influențează performanța modelului Random Forest, voi prezenta un exemplu

concret de arbore de decizie construit folosind acești parametri. Acest exemplu evidențiază cum valorile alese pentru numărul de arbori, criteriile de diviziune și limitele pentru noduri și frunze determină structura și deciziile luate de un singur arbore în cadrul pădurii aleatoare.

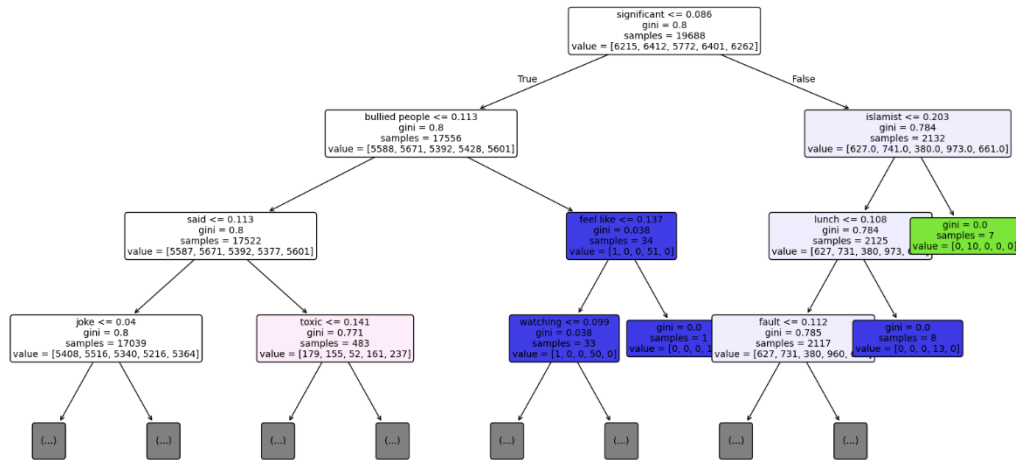


Fig 4.3.2 –1 Arbore de decizie

Predicțiile realizate pentru datele vectorizate cu TF-IDF au o performanță solidă în detectarea și clasificarea tipurilor de hărțuire. Acuratețea generală a modelului este de 93%, iar analiza pe clase evidențiază rezultate remarcabile pentru mai multe categorii. Clasa „age” înregistrează o precizie, regăsire și scor F1 de 99%, urmată de categoriile „religion” și „ethnicity”, care obțin o precizie de 99%, regăsirea de 96%, respectiv 93%, și scoruri F1 de 98% și 96%. Chiar și pentru celelalte clase, modelul atinge valori ale scorului F1 de peste 85%, demonstrând astfel un echilibru solid între precizie și regăsire.

	precision	recall	f1-score	support
gender	0.92	0.87	0.89	1548
religion	0.99	0.96	0.98	1592
other_cyberbullying	0.80	0.92	0.85	1486
age	0.99	0.99	0.99	1577
ethnicity	0.99	0.93	0.96	1563
accuracy			0.93	7766
macro avg	0.94	0.93	0.93	7766
weighted avg	0.94	0.93	0.94	7766

Accuracy: 0.934
 Precision: 0.938
 Recall: 0.934
 F1 Score: 0.935

Fig 4.3.2 –2 Raport de clasificare pentru Random Forest TF-IDF

Rezultatele foarte bune ale modelului sunt susținute, de asemenea, de matricea de confuzie și curba ROC.

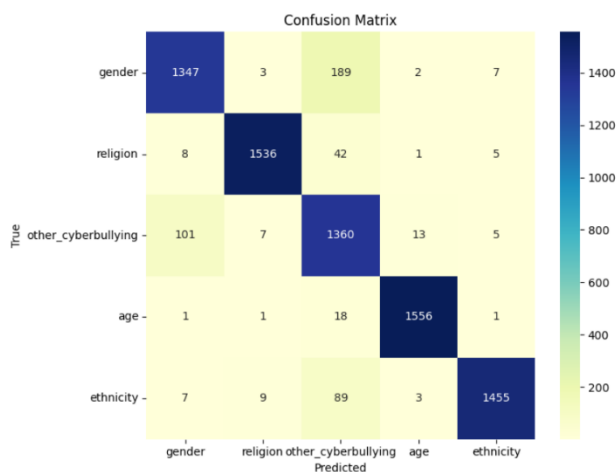


Fig 4.3.2 –3 Matricea de confuzie pentru Random Forest TF-IDF

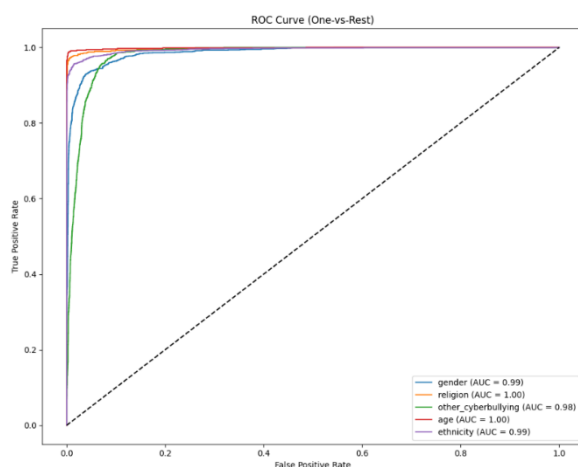


Fig 4.3.2 –4 Curba ROC pentru Random Forest TF-IDF

Pentru GloVe, scorurile modelului Random Forest sunt ușor mai scăzute comparativ cu cele obținute pe datele vectorizate cu TF-IDF, dar rămân totuși la un nivel rezonabil, menținând un echilibru satisfăcător între precizie și regăsire. Similar cu TF-IDF, clasa *age* înregistrează cele mai bune rezultate, urmată de aproape aceleași clase, *religion* și *ethnicity*, care continuă să performeze bine. Totuși, pentru clasele *gender* și *other_cyberbullying*, modelul pe GloVe are o ușoară scădere a performanței față de TF-IDF, indicând că vectorizarea prin TF-IDF oferă o reprezentare mai eficientă pentru aceste categorii.

NECLASIFICAT

	precision	recall	f1-score	support
gender	0.85	0.76	0.80	1552
religion	0.91	0.91	0.91	1592
other_cyberbullying	0.71	0.82	0.76	1500
age	0.94	0.96	0.95	1577
ethnicity	0.93	0.86	0.89	1563
accuracy			0.86	7784
macro avg	0.87	0.86	0.86	7784
weighted avg	0.87	0.86	0.86	7784

Accuracy: 0.864
Precision: 0.869
Recall: 0.864
F1 Score: 0.865

Fig 4.3.2 –5 Raport de clasificare pentru Random Forest GloVe

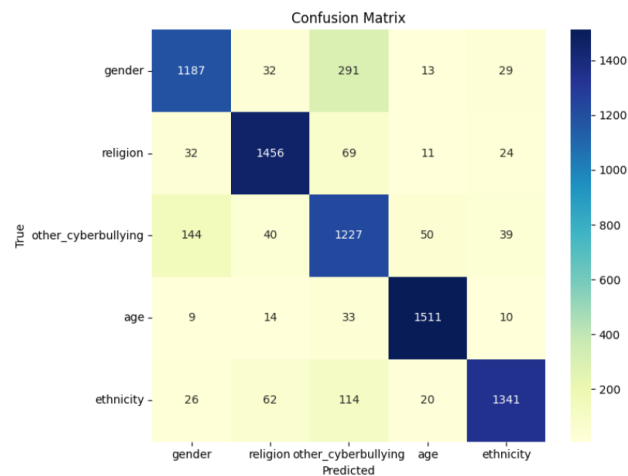


Fig 4.3.2 –6 Matricea de confuzie pentru Random Forest GloVe

Diferențele de performanță dintre TF-IDF și GloVe, în special pentru clasele *other_cyberbullying* și *gender*, sunt evidențiate clar și de curbele ROC:

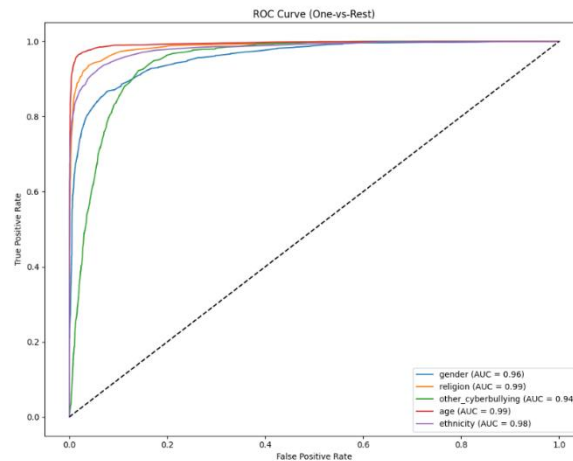


Fig 4.3.2 –7 Curba ROC pentru Random Forest GloVe

Pentru datele vectorizate cu Word2Vec, modelul afișează o performanță mai bună comparativ cu GloVe, evidențiind o precizie, recall și scor F1 mai ridicate pentru majoritatea claselor. Totuși, rezultatele rămân ușor inferioare față de cele obținute cu TF-IDF, care păstrează un ușor avantaj în ceea ce privește acuratețea și echilibrul general între precizie și regăsire. Clasa *age* continuă să înregistreze cele mai bune valori, urmată de *ethnicity* și *religion*, în timp ce clasele *gender* și *other_cyberbullying* prezintă variații mai mari între cele trei metode de vectorizare.

	precision	recall	f1-score	support
gender	0.93	0.78	0.85	1646
religion	0.90	0.95	0.92	1582
other_cyberbullying	0.73	0.84	0.79	1536
age	0.95	0.95	0.95	1600
ethnicity	0.95	0.91	0.93	1584
accuracy			0.89	7948
macro avg	0.89	0.89	0.89	7948
weighted avg	0.89	0.89	0.89	7948

Accuracy: 0.886
Precision: 0.892
Recall: 0.886
F1 Score: 0.887

Fig 4.3.2 –8 Raport de clasificare pentru Random Forest Word2Vec

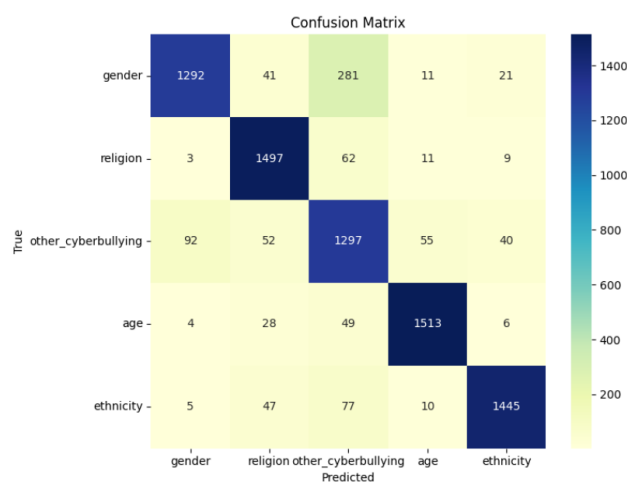


Fig 4.3.2 –9 Matricea de confuzie pentru Random Forest Word2Vec

Similaritatea dintre GloVe și Word2Vec se poate observa și prin analiza curbelor ROC, unde se observă și performanța ușor mai crescută pentru 3 dintre cele 5 clase:

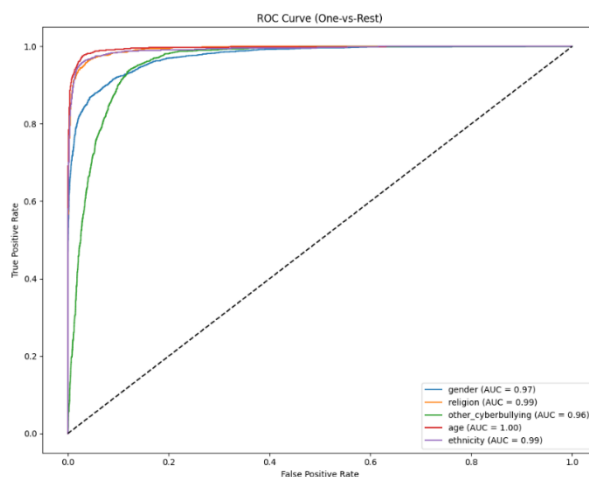


Fig 4.3.2 –10 Curba ROC pentru Random Forest Word2Vec

Analiza comparativă a performanțelor modelului Random Forest pe cele trei tipuri de vectorizare evidențiază faptul că TF-IDF oferă cele mai bune rezultate, sugerând faptul că tehnicile bazate pe frecvențe și importanța cuvintelor reușesc să capteze cel mai eficient caracteristicile relevante, comparativ cu vectorizările bazate pe modele de învățare a reprezentărilor contextuale.

Tabel 2 Random Forest

Nr.	Metodă vectorizare	Acuratețe	Precizie	Regăsire	Scor F1
1	TF-IDF	93%	93%	93%	93%
2	GloVe	86%	86%	86%	86%
3	Word2Vec	88%	89%	88%	88%

4.3.3. Multinomial Naive Bayes

Pentru antrenarea algoritmului Multinomial Naive Bayes, am optat pentru două metode de vectorizare: TF-IDF și Count Vectorizer. Alegerea acestor tehnici este justificată de faptul că modelul presupune ca atributele de intrare să fie formate din valori pozitive, reprezentând frecvențe sau importanța relativă a termenilor. În

consecință, metode de reprezentare precum GloVe sau Word2Vec, care generează vectori continui ce pot conține și valori negative, nu sunt compatibile cu ipotezele pe care se bazează acest algoritm.

Rezultatele celor două sunt, din punct de vedere al acurateței generale, foarte apropiate. Ambele abordări conduc la o performanță solidă a modelului, cu o acuratețe de 89% în cazul TF-IDF și 90% pentru CountVectorizer. Diferențele devin însă mai vizibile atunci când analizăm performanța modelului pe fiecare clasă în parte.

În cazul clasei *other_cyberbullying*, diferența este mai pronunțată: CountVectorizer atinge o regăsire mai ridicată, deci reușește să recunoască mai bine exemplele corecte, chiar dacă cu un mic compromis în ceea ce privește precizia. Acest lucru sugerează că această metodă este ceva mai sensibilă în fața unui tip de conținut care, în general, este mai dificil de identificat cu acuratețe. Clasa age continuă să fie identificată foarte bine, cu scoruri foarte ridicate în ambele scenarii, semnalând că modelul reușește constant să învețe bine caracteristicile acestei clase.

	precision	recall	f1-score	support
gender	0.89	0.85	0.87	1548
religion	0.91	0.96	0.94	1592
other_cyberbullying	0.80	0.76	0.78	1486
age	0.92	0.98	0.95	1577
ethnicity	0.92	0.90	0.91	1563
accuracy			0.89	7766
macro avg	0.89	0.89	0.89	7766
weighted avg	0.89	0.89	0.89	7766

Fig 4.3.3 –1 Raport de clasificare Multinomial Naive Bayes TF-IDF

	precision	recall	f1-score	support
gender	0.91	0.81	0.86	1548
religion	0.94	0.96	0.95	1592
other_cyberbullying	0.77	0.82	0.79	1486
age	0.94	0.98	0.96	1577
ethnicity	0.94	0.91	0.92	1563
accuracy			0.90	7766
macro avg	0.90	0.90	0.90	7766
weighted avg	0.90	0.90	0.90	7766

Fig 4.3.3 –2 Raport de clasificare Multinomial Naive Bayes CountVectorizer

TF-IDF, prin faptul că atribuie o greutate mai mare cuvintelor rare și penalizează termenii frecvenți, oferă un avantaj în captarea diferențelor mai subtile dintre clase, ceea ce se reflectă într-un scor mai bun pentru clasa *other_cyberbullying*. Acest lucru sugerează că vectorizarea reușește să evidențieze cuvintele cu relevanță contextuală, care deși apar mai rar, pot fi esențiale pentru o clasificare corectă. Pe de altă parte, CountVectorizer, care se bazează pe frecvența brută a termenilor, pare să funcționeze mai bine în cazul claselor în care apar cuvinte-cheie frecvent repetate, ceea ce a oferit un avantaj pentru o mare parte din clase, mai puțin *other_cyberbullying*, de unde putem trage concluzia că această clasă nu conține cuvinte specifice care să o identifice.

Aceste diferențe pot fi observate și prin matricile de confuzie ale celor două modele rezultate, unde se observă că modelul care utilizează CountVectorizer realizează mai frecvent confuzie pentru clasa *other_cyberbullying*, în special cu eticheta *gender*.

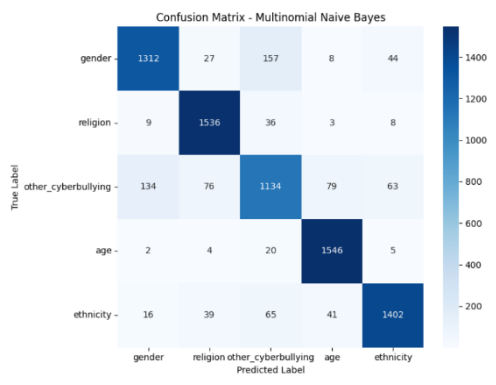


Fig 4.3.3 –3 Matricea de confuzie Multinomial Naive Bayes TF-IDF

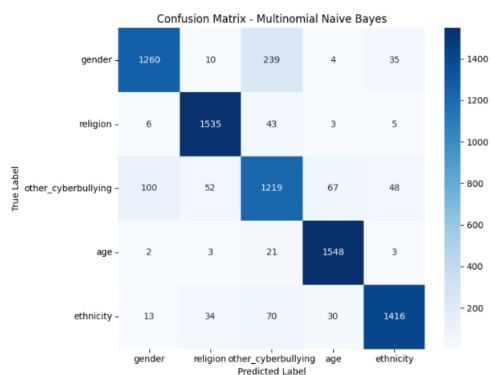


Fig 4.3.3 –4 Matricea de confuzie Multinomial Naive Bayes CountVectorizer

Rezultatele obținute sugerează că atât TF-IDF, cât și CountVectorizer reprezintă metode de vectorizare adecvate pentru modelul Multinomial Naive Bayes, oferind performanțe ridicate și apropiate ca acuratețe. Cu toate acestea, comportamentul modelului variază ușor în funcție de vectorizarea aleasă, ceea ce evidențiază modul distinct în care fiecare tehnică reușește să surprindă informația din setul de date. Diferențele dintre cele două metode reflectă echilibrul dintre relevanța contextuală, oferită de TF-IDF, și intensitatea frecvenței, captată de CountVectorizer. Alegerea metodei optime poate depinde, așadar, de specificul datelor și de tipul de clasificare urmărit.

Tabel 3 Multinomial Naive Bayes

Nr.	Metodă vectorizare	Acuratețe	Precizie	Regăsire	Scor F1
1	TF-IDF	89%	89%	89%	89%
2	CountVectorizer	90%	90%	90%	90%

4.3.4. Support Vector Machine

Modelul SVM a fost configurat cu un kernel liniar, potrivit pentru datele noastre, unde separarea între clase este aproximativ liniară. Această alegere permite o procesare mai rapidă și o interpretare mai ușoară a rezultatului, menținând în același timp o performanță bună. Acest model este recunoscut pentru abilitatea sa de a maximiza marginea de separare dintre clase, ceea ce îl face robust și capabil să

generalizeze bine pe date noi. Astfel, modelul oferă un echilibru solid între precizie și eficiență.

Rezultatele obținute cu modelul SVM arată o performanță foarte bună, în special când se folosește vectorizarea TF-IDF, unde acuratețea atinge 93%, iar scorul F1 este și el ridicat, indicând un echilibru bun între precizie și regăsire pe toate clasele. Aceasta sugerează că modelul SVM, cu kernel liniar, se potrivește bine pentru datele reprezentate prin TF-IDF, unde cuvintele cu o importanță mai mare în contextul lor sunt bine evidențiate.

	precision	recall	f1-score	support
gender	0.92	0.88	0.90	1548
religion	0.99	0.96	0.97	1592
other_cyberbullying	0.80	0.92	0.86	1486
age	0.99	0.98	0.98	1577
ethnicity	0.99	0.94	0.96	1563
accuracy			0.93	7766
macro avg	0.94	0.93	0.93	7766
weighted avg	0.94	0.93	0.94	7766

Accuracy: 0.935
Precision: 0.939
Recall: 0.935
F1 Score: 0.936

Fig 4.3.4 –1 Raport de clasificare SVM TF-IDF

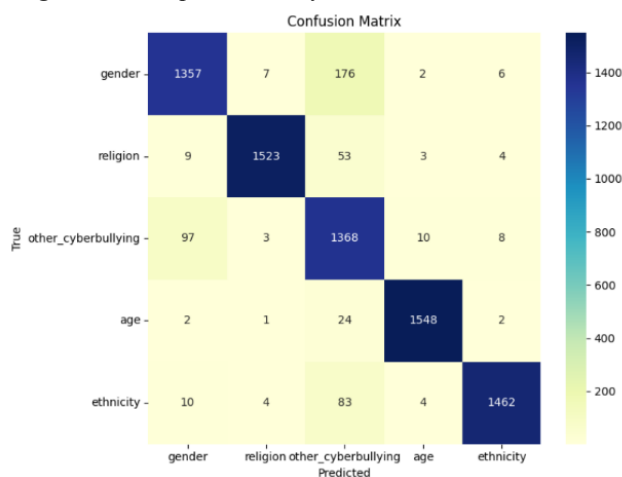


Fig 4.3.4 –2 Matricea de confuzie SVM TF-IDF

Pentru vectorizarea GloVe, performanța scade, cu o acuratețe în jur de 88%, dar valorile de recall și precizie rămân bune, ceea ce indică faptul că SVM reușește să utilizeze și reprezentările distribuite, deși cu o eficiență ceva mai redusă comparativ cu TF-IDF. Se observă scăderea a performanței pentru clasele *gender* și

other_cyberbullying, ceea ce poate reflecta o mai slabă diferențiere a acestor clase în spațiul GloVe.

	precision	recall	f1-score	support
gender	0.85	0.82	0.83	1552
religion	0.94	0.93	0.94	1592
other_cyberbullying	0.74	0.79	0.76	1500
age	0.96	0.97	0.96	1577
ethnicity	0.93	0.90	0.92	1563
accuracy			0.88	7784
macro avg	0.88	0.88	0.88	7784
weighted avg	0.88	0.88	0.88	7784

Accuracy: 0.883
Precision: 0.885
Recall: 0.883
F1 Score: 0.884

Fig 4.3.4 –3 Raport de clasificare SVM GloVe

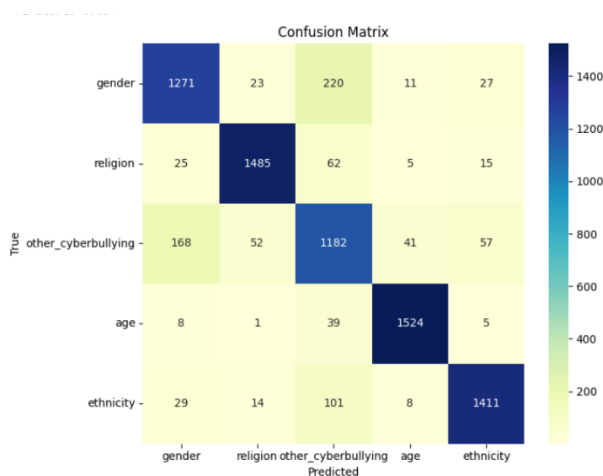


Fig 4.3.4 –4 Matricea de confuzie SVM GloVe

Rezultatele obținute cu Word2Vec sunt apropiate de cele cu TF-IDF, având o acuratețe de 91% și un scor F1 de 90%. Acest lucru reflectă faptul că Word2Vec, prin modul în care leagă cuvintele în funcție de contextul în care apar, reușește să capteze relații semantice mai profunde între termeni.

	precision	recall	f1-score	support
gender	0.89	0.84	0.86	1646
religion	0.95	0.95	0.95	1582
other_cyberbullying	0.77	0.81	0.79	1536
age	0.97	0.96	0.97	1600
ethnicity	0.95	0.96	0.95	1584
accuracy			0.91	7948
macro avg	0.91	0.90	0.90	7948
weighted avg	0.91	0.91	0.91	7948

Accuracy: 0.905
Precision: 0.906
Recall: 0.905
F1 Score: 0.905

Fig 4.3.4 –5 Raport de clasificare SVM Word2Vec

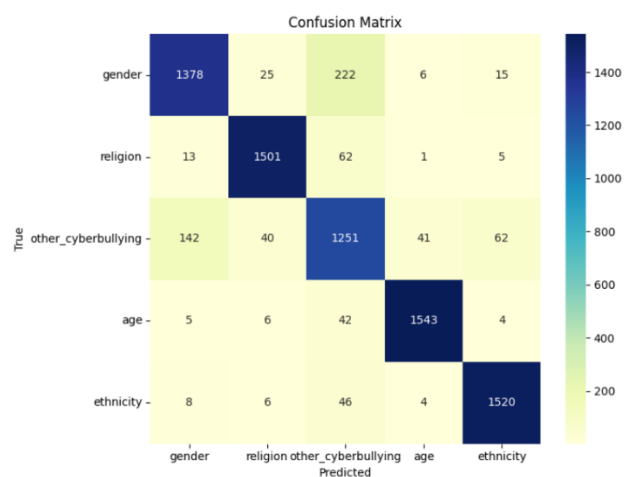


Fig 4.3.4 –6 Matricea de confuzie SVM Word2Vec

SVM se remarcă prin performanțe foarte bune, indiferent de metoda de vectorizare utilizată. Rezultatele obținute confirmă faptul că SVM este o alegere robustă și eficientă pentru această problemă, oferind un echilibru foarte bun între precizie și capacitatea de generalizare.

Tabel 4 Support Vector Machine

Nr.	Metodă vectorizare	Acuratețe	Precizie	Regăsire	Scor F1
1	TF-IDF	93%	93%	93%	93%
2	GloVe	88%	88%	88%	88%
3	Word2Vec	91%	91%	91%	91%

4.3.5. K-Nearest Neighbors

Modelul K-Nearest Neighbors a fost configurat să utilizeze 10 vecini pentru clasificare, ceea ce oferă un echilibru între stabilitate și sensibilitate la variațiile locale ale datelor. Am ales să ponderăm influența fiecărui vecin în funcție de distanța față de punctul de clasificare, astfel încât vecinii mai apropiați să aibă o influență mai mare asupra deciziei finale. Pentru măsurarea distanței, am folosit metrica cosinus, care este potrivită în special pentru date vectorizate și exprimă similaritatea unghiulară între puncte. Această configurare permite modelului să capteze structuri locale relevante și să ofere predicții robuste în probleme de clasificare bazate pe similaritatea vectorială.

Rezultatele obținute pentru modelul KNN, indiferent de metoda de vectorizare folosită sunt foarte apropiate, toate înregistrând o acuratețe de aproximativ 85%. Această consistență sugerează că modelul KNN nu este foarte sensibil la tipul de reprezentare vectorială și funcționează similar în cazul acestor date. KNN se bazează pe calculul distanțelor între vectorii de caracteristici pentru a decide clasa, iar această abordare îl face să fie destul de robust în fața unor diferențe moderate în reprezentarea textului.

	precision	recall	f1-score	support
gender	0.86	0.81	0.83	1548
religion	0.95	0.82	0.88	1592
other_cyberbullying	0.62	0.84	0.71	1486
age	0.95	0.90	0.93	1577
ethnicity	0.94	0.86	0.90	1563
accuracy			0.85	7766
macro avg	0.87	0.85	0.85	7766
weighted avg	0.87	0.85	0.85	7766
Accuracy: 0.846				
Precision: 0.868				
Recall: 0.846				
F1 Score: 0.852				

Fig 4.3.4 –I Raport de clasificare KNN TF-IDF

	precision	recall	f1-score	support
gender	0.82	0.82	0.82	1552
religion	0.94	0.92	0.93	1592
other_cyberbullying	0.80	0.56	0.66	1500
age	0.87	0.98	0.92	1577
ethnicity	0.81	0.95	0.88	1563
accuracy			0.85	7784
macro avg	0.85	0.85	0.84	7784
weighted avg	0.85	0.85	0.84	7784

Accuracy: 0.850
Precision: 0.848
Recall: 0.850
F1 Score: 0.843

Fig 4.3.4 –I Raport de clasificare KNN GloVe

	precision	recall	f1-score	support
gender	0.88	0.82	0.85	1646
religion	0.86	0.97	0.91	1582
other_cyberbullying	0.81	0.48	0.60	1536
age	0.82	0.98	0.90	1600
ethnicity	0.85	0.96	0.90	1584
accuracy			0.85	7948
macro avg	0.84	0.84	0.83	7948
weighted avg	0.84	0.85	0.83	7948

Accuracy: 0.846
Precision: 0.844
Recall: 0.846
F1 Score: 0.834

Fig 4.3.4 –I Raport de clasificare KNN Word2Vec

Faptul că rezultatele sunt atât de apropiate arată că pentru KNN alegerea vectorizării are un impact mai redus decât în cazul altor algoritmi. Modelul profită eficient de structura spațiului vectorial, indiferent dacă vectorii reflectă frecvențe brute sau reprezentări distribuite. Cu toate acestea, performanțele obținute sunt ușor mai scăzute comparativ cu alte modele utilizate, ceea ce sugerează că KNN, deși robust și simplu, poate avea limitări în captarea unor tipare mai complexe din date, în special în probleme de clasificare cu multiple clase și caracteristici subtile.

Tabel 5 K-Nearest Neighbours

Nr.	Metodă vectorizare	Acuratețe	Precizie	Regăsire	Scor F1
1	TF-IDF	85%	86%	84%	85%
2	GloVe	85%	84%	85%	84%
3	Word2Vec	84%	84%	84%	83%

4.3.6. RoBERTa

Pentru a obține reprezentări contextuale eficiente ale textelor, am utilizat modelul preantrenat *RoBERTa-base*. Acesta face parte din familia transformerelor și este cunoscut pentru performanța ridicată în diverse sarcini de procesare a limbajului natural.

În etapa de preprocesare, am aplicat tokenizerul specific modelului RoBERTa, care transformă fiecare propoziție într-o secvență numerică de tokeni, incluzând simbolurile speciale necesare pentru a marca începutul, sfârșitul și alte caracteristici ale textului. După analiza lungimii maxime a secvențelor tokenizate, am fixat o dimensiune standard de 256 de tokeni, valoare care s-a dovedit suficientă pentru majoritatea comentariilor de pe rețelele sociale, ce au în general o lungime limitată.

Clasificatorul constă din două layere liniare separate printr-un strat de dropout și activare *LeakyReLU*. Modelul este antrenat folosind funcția de pierdere *CrossEntropyLoss*, optimizatorul *AdamW* cu o rată de învățare inițială de $2e-5$, iar un scheduler de tip *get_linear_schedule_with_warmup* reduce lin această rată de învățare de la valoarea inițială până la 0 pe parcursul tuturor pașilor de antrenament

Procesul de antrenare al modelului implică parcurgerea iterativă a datelor de antrenament pe parcursul a 3 epoci, pentru care se obține o acuratețe, precizie, regăsire și scor F1, toate de 96%.

Classification Report for ROBERTA :				
	precision	recall	f1-score	support
religion	0.93	0.95	0.94	1583
age	0.98	0.99	0.98	1593
ethnicity	0.92	0.92	0.92	1537
gender	0.99	0.98	0.98	1566
other_cyberbullying	0.99	0.98	0.98	1577
accuracy			0.96	7856
macro avg	0.96	0.96	0.96	7856
weighted avg	0.96	0.96	0.96	7856

Fig 4.3.6 –I Raport de clasificare RoBERTa

NECLASIFICAT

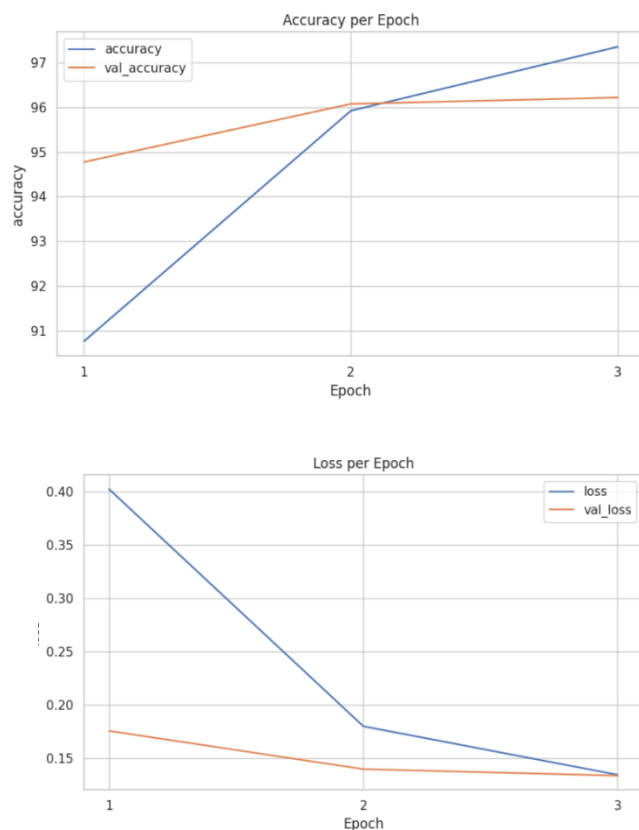


Fig 4.3.6 –2 Curbe de învățare model RoBERTa

În cazul acestui model, se observă confuzia care se crează între clasa *religion* și clasa *ethnicity*:

		RoBERTa Sentiment Analysis Confusion Matrix				
		religion	age	ethnicity	gender	other_cyberbullying
Test	religion	1503	10	66	0	4
	age	5	1572	13	0	3
	ethnicity	82	11	1421	12	11
	gender	6	3	21	1533	3
	other_cyberbullying	5	6	14	0	1552
		religion	age	ethnicity	gender	other_cyberbullying
		Predicted				

Fig 4.3.6 –3 Matricea de confuzie RoBERTa

Tabel 6 RoBERTa-base

Nr.	Model	Acuratețe	Precizie	Regăsire	Scor F1
1	RoBERTa-base	96%	96%	96%	96%

4.3.7. BERT

Arhitectura modelului constă într-un model BERT preantrenat bert-base-cased, urmat de un strat clasificator compus din două layeruri liniare cu o activare ReLU între ele.

Pentru procesul de tokenizare am folosit tokenizerul specific modelului bert-base-cased, care transformă fiecare propoziție într-o secvență numerică de tokeni. Tokenizarea include adăugarea simbolurilor speciale necesare, iar pentru uniformitate și eficiență secvențele sunt ajustate la o lungime fixă de 256 tokeni prin padding și truncation.

Pentru antrenare, am s-au parcurs 3 epoci, toate cu o acuratețe, precizie, regăsire și scor F1 de 93%:

Classification Report for BERT :				
	precision	recall	f1-score	support
religion	0.94	0.96	0.95	1589
age	0.98	0.98	0.98	1577
ethnicity	0.99	0.99	0.99	1548
gender	0.88	0.91	0.90	1521
not bullying	0.86	0.81	0.83	1527
accuracy			0.93	7762
macro avg	0.93	0.93	0.93	7762
weighted avg	0.93	0.93	0.93	7762

Fig 4.3.7 –1 Raport de clasificare BERT-base-uncased

BERT Sentiment Analysis Confusion Matrix						
Test	religion	1367	9	172	4	12
	age	12	1538	26	3	6
	ethnicity	112	19	1290	26	12
	gender	1	1	25	1519	1
	other_cyberbullying	5	6	8	5	1544
	religion	age	ethnicity	gender	other_cyberbullying	

Fig 4.3.7 –2 Matricea de confuzie BERT-base-uncased

Tabel 7 BERT

Nr.	Model	Acuratețe	Precizie	Regăsire	Scor F1
1	BERTa-base-cased	93%	93%	93%	93%

4.3.8. Justificarea alegerii modelului final

În urma experimentelor realizate, toate modelele testate au demonstrat performanțe solide în sarcina de clasificare a textelor. Fiecare model a reușit să capteze, într-o măsură bună, informațiile relevante din date, asigurând o acuratețe satisfăcătoare:

Cu toate acestea, modelul *RoBERTa* s-a evidențiat printr-o acuratețe superioară față de celelalte modele, precum și printr-o stabilitate crescută în timpul antrenamentului, ceea ce s-a tradus printr-o variație redusă a metricilor de performanță între epoci și pe seturile de validare. Această robustețe se datorează în parte capacității sale de a procesa contextualizat informația din text, datorită arhitecturii transformer optimizate și tokenizerului specific, care permite o reprezentare mai fidelă a limbajului natural.

Mai mult, *RoBERTa* a demonstrat o adaptabilitate mai bună la particularitățile datelor, cum ar fi lungimea variabilă și complexitatea comentariilor extrase din rețelele sociale. Aceste aspecte au făcut ca alegerea sa ca model final să fie justificată, garantând astfel performanțe superioare și o generalizare mai bună în aplicații reale.

5. Implementarea aplicației likes2love

În acest capitol este prezentată implementarea aplicației web *likes2love*, destinată analizei interacțiunilor din comentariile postărilor de pe TikTok, în vederea detectării comentariilor specifice hărțuirii. Sistemul combină colectarea în timp real a datelor, preprocesarea avansată a conținutului textual și analiza prin modele de inteligență artificială pentru detectarea tipurilor de hărțuire în mediul online. Implementarea presupune parcurgerea unor etape cheie, începând alegerea postării care va fi supusă analizei, apoi extragerea și stocarea comentariilor, continuând cu prelucrarea acestora pentru uniformizare și culminând cu aplicarea unor modele de clasificare performante și stocarea și returnarea rezultatelor. De asemenea, aplicația

oferă utilizatorilor rapoarte vizuale complexe și istoricul analizelor, într-un mediu ușor de utilizat.

5.1. Definirea cerințelor software ale sistemului

Un element fundamental în dezvoltarea oricărui sistem software constă în stabilirea clară a cerințelor, care garantează că produsul final răspunde așteptărilor utilizatorilor. Aceste cerințe se împart în două categorii principale: cerințele funcționale, ce definesc comportamentele și funcțiile pe care sistemul trebuie să le ofere, și cerințele nefuncționale, care se concentrează pe calități precum fiabilitatea, securitatea, scalabilitatea și eficiența sistemului.

5.1.1. Cerințe funcționale

Aplicația web *likes2love* este proiectată să analizeze și să monitorizeze interacțiunile utilizatorilor din comentariile postărilor de pe TikTok. Pentru a asigura o performanță optimă și un flux coerent, sistemul a fost dezvoltat pe baza unui set clar și bine definit de cerințe funcționale, care stabilesc responsabilitățile și comportamentele esențiale ale aplicației:

- **Pentru fiecare browser de pe care este accesată aplicația, se generează un UUID.** La prima accesare, utilizatorului i se atribuie acest UUID, care este utilizat pentru crearea unui token stocat într-un cookie. În cazul utilizatorilor care revin ulterior, valabilitatea tokenului este prelungită automat cu încă 364 de zile. Totodată, pentru accesările în modul incognito, se generează un UUID nou de fiecare dată, asigurând astfel o identificare temporară și distinctă.
- **Introducerea unui link corespunzător în cadrul analizei.** Link-ul postării pentru care se dorește analiza trebuie să respecte indicațiile oferite în aplicație, altfel, analiza nu va avea succes.
- **Postarea aleasă trebuie să conțină suficiente comentarii, astfel încât analiza să fie robustă.** Postarea ar trebui să aibă un minim de 100 de comentarii pentru a beneficia de o analiză mai completă a datelor.
- **Kafka va fi utilizat pentru orchestrarea fluxurilor de date între componente.** Acest sistem va asigura un management eficient al datelor colectate și prelucrate.

- **Extragerea în timp real.** Comentariile de la postare vor fi extrase în timp real de către microserviciul de scraping, iar utilizatorul poate efectua analize către aceeași postare constant, pentru a observa diferențele în timp.
- **Preprocesarea comentariilor se va face în mod corespunzător.** Fiecare comentariu va fi procesat și se va efectua traducerea acestuia în limba engleză, conversia emoji-urilor, emoticoanelor și a abrevierilor și curățarea datelor de zgomot, pentru o analiză corectă și uniformă.
- **Modelul principal folosit pentru predicția tipului de hărțuire va fi cel cu cea mai mare acuratețe obținută.** Predicția principală, responsabilă pentru identificarea tipului specific de hărțuire în comentarii, va fi realizată de modelul ales pentru performanța sa superioară. Totuși, înainte ca comentariile să fie analizate de acest model, ele vor trece printr-o etapă preliminară de clasificare folosind alte modele, care au rolul de a filtra și identifica conținutul ofensator.
- **Raportul analizei va fi prezentat printr-o serie de grafice sugestive.** Vor fi prezentate grafice care oferă o imagine completă asupra comentariilor din postări, incluzând distribuția acestora pe categorii de pozitivitate, negativitate și neutralitate. De asemenea, se va afișa un grafic ce evidențiază potențialul ofensator al comentariilor, precum și un altul care detaliază tipurile de hărțuire identificate și numărul de comentarii corespunzătoare fiecărei categorii. Pentru o mai bună înțelegere, vor fi oferite exemple relevante de comentarii pentru fiecare tip de hărțuire. În plus, vor fi disponibile grafice de monitorizare a activității comentariilor pe zile, inclusiv un raport special pentru ziua cu cea mai intensă activitate.
- **Informațiile extrase din postare vor fi stocate în baza de date MongoDB.** Informațiile despre numărul de aprecieri, comentarii, distribuiri, salvări ale postării vor fi salvate în baza de date. Pentru comentarii, vor fi stocate atât conținutul text în forma lui brută, numărul de aprecieri și de răspunsuri, data postării comentariilor, cât și forma textului după preprocesare și fiecare etichetă atribuită în timpul analizei
- **Accesarea istoricului analizei.** Aplicația nu dispune de autentificare, iar fiecare utilizator va avea acces la istoric pe baza UUID-ului generat pentru fiecare browser specific.

- **Posibilitatea descărcării în format PDF a raportului.** Toate informațiile din raportul generat în aplicația web vor putea fi generate și în format PDF.

5.1.2. Cerințe nefuncționale

Sistemul trebuie să răspundă următoarelor cerințe nefuncționale:

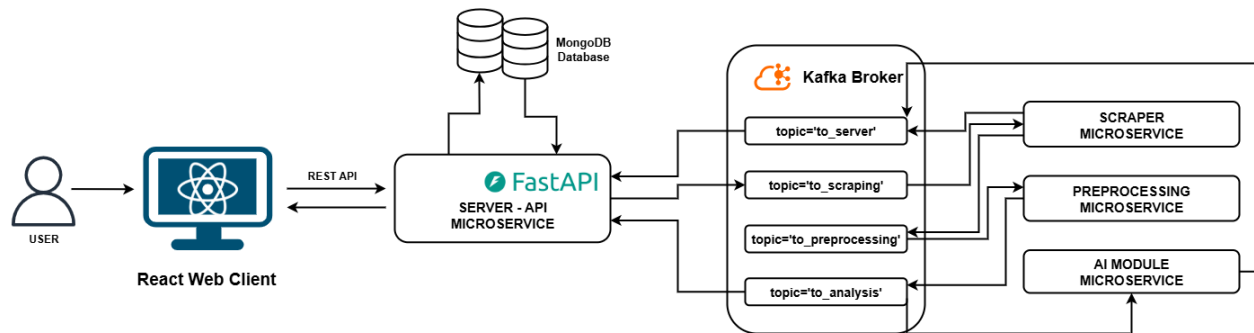
- **Interfață facil de utilizat.** Sistemul trebuie să ofere o interfață ușor de utilizat și intuitivă pentru utilizatori, care să faciliteze navigarea și interpretarea rezultatelor.
- **Acces la internet.** Sistemul are nevoie de o conexiune stabilă la internet pentru a funcționa corect, pentru procesul de extragere a datelor și comunicarea între microservicii.
- **Timp de răspuns rapid.** Sistemul trebuie să întoarcă rezultatele într-un timp redus și cu un minim consum de energie și resurse.
- **Performanță și fiabilitate.** Sistemul trebuie să fie robust și să funcționeze fără întreruperi, minimalizând riscul de căderi sau erori critice. Microserviciile trebuie să aibă mecanisme de monitorizare și auto-recuperare. De asemenea, arhitectura trebuie să permită adăugarea de noi funcționalități sau actualizări, fără a afecta funcționarea întregului sistem.
- **Gestionarea extragerii datelor.** Microserviciul responsabil de extragerea comentariilor trebuie să fie capabil să identifice situațiile în care este detectat ca bot și să reacționeze automat prin schimbarea tokenului `ms_token` sau a altor parametri de autentificare, asigurând astfel continuitatea și fiabilitatea procesului de colectare a datelor.
- **Monitorizare și logare detaliată.** Toate componentele sistemului trebuie să înregistreze evenimente și erori, pentru a facilita depanarea și analizarea performanței.

5.2. Arhitectura sistemului

Aplicația *likes2love* este dezvoltată pe o arhitectură client-server locală, în care clientul constă într-o interfață web simplă realizată în React, iar serverul este compus dintr-o serie de microservicii independente. Interacțiunea dintre client și server se

realizează printr-un API REST construit cu *FastAPI*, iar microserviciile serverului comunică între ele prin *Kafka*, ceea ce asigură o transmitere rapidă și fiabilă a datelor.

Această structură modulară a serverului facilitează întreținerea și extinderea aplicației, permițând modificări de funcționalități fără a afecta întregul sistem. De asemenea, arhitectura asigură eficiență, scalabilitate și oferă o perspectivă clară asupra fluxului de date și a interacțiunilor din cadrul sistemului.



Figură 5.2-1 Arhitectura aplicației

5.2.1. Arhitectura clientului

Aplicația client reprezintă interfața prin care utilizatorul interacționează cu sistemul. Aceasta a fost dezvoltată utilizând React, o bibliotecă JavaScript modernă care permite structurarea logică a interfeței în componente independente, reutilizabile și ușor de întreținut. Navigarea între secțiuni este realizată atât printr-un sistem de rutare internă, cât și externă, ceea ce contribuie la o experiență fluidă și coerentă pentru utilizator.

Pagina de start a aplicației oferă o previzualizare a funcționalităților disponibile, cu un meniu de navigare care conține cele trei secțiuni principale ale aplicației, precum și un buton către o pagină introductivă, care explică care sunt pașii ce trebuie urmați pentru a obține analiza a postării dorite.

Pagina principală a aplicației este structurată în doua containere: cel care permite vizualizarea rapoartelor generate anterior, istoricul analizei, și cel pentru generarea unui nou raport. Istoricul poate fi extins pentru a beneficia de funcționalități de editare a numelui, ștergerea raportului sau exportul său în format PDF, și de modulele de filtrare. Containerul destinat generării de raport este simplu și eficient, conține un buton de pentru introducerea linkului postării și o sugestie de

a consulta instrucțiunile privind formatul corect al linkului pentru o experiență completă. Raportul este afișat automat după finalizarea procesului de analiză de către server.

Pagina despre aplicație oferă informații pe scurt despre modelele utilizate în crearea aplicației și include o secțiune de contact, prin intermediul unui formular de trimitere e-mail, care permite comunicarea cu administratorul aplicației.

Arhitectura clientului este centrată în jurul unei experiențe de utilizare clare, intuitive și axată pe simplitate, cu un design modern și componente vizuale grupate logic. Aceasta a fost gândită pentru a oferi o interfață intuitivă, accesibilă și ușor de utilizat, indiferent de nivelul tehnic al utilizatorului. Alegerea React pentru dezvoltare și integrarea rutării interne și externe contribuie la o experiență fluentă și modernă.

5.2.2. Arhitectura serverului

Serverul reprezintă componenta centrală a sistemului, responsabilă de gestionarea fluxului de date și de coordonarea proceselor din întregul ecosistem aplicațional. Arhitectura serverului este organizată pe baza unui sistem de microservicii, fiecare dintre ele fiind responsabil pentru o etapă specifică a procesului de analiză.

Toate aceste microservicii sunt orchestrate prin Docker Compose, care permite rularea coordonată și automatizată a serviciilor într-un mediu controlat și izolat. Volumele partajate : `shared_logs` și `shared_utils` facilitează comunicarea și reutilizarea codului între microservicii, permițând o structură modulară și ușor de extins.

Brokerul Kafka este elementul central al sistemului de mesagerie, expunând portul 9092 pentru acces extern și porturile interne 29092 și 29093 pentru comunicarea între broker și controller.

În cele ce urmează, vor fi prezentate în detaliu componentele principale ale serverului, precum și rolul și interacțiunile fiecărui microserviciu în cadrul arhitecturii generale.

Serviciul Server API

Serviciul Server API reprezintă punctul central de interacțiune dintre clientul frontend, microserviciile specializate și baza de date MongoDB. Comunicarea dintre client și server este realizată prin intermediul frameworkului FastAPI, care permite expunerea rapidă și eficientă a endpointurilor REST.

La inițializare, aplicația este configurată cu suport CORS, permițând interacțiunea cu interfața web care rulează la adresa `http://localhost:5173`. Acest mecanism asigură accesul controlat și securizat al clientului la resursele serverului. Funcționalitatea serviciului este organizată în jurul unor endpointuri clare, corespunzătoare metodelor HTTP standard: GET, POST, PUT, și DELETE.

Aplicația utilizează un mecanism de autentificare fără cont de utilizator clasic, bazat pe UUID, identificatorul unic de 128 de biți folosit în aplicația web pentru a genera ID-uri unice, fără coliziuni, ideal în sisteme distribuite, și tokenul JWT, stocat în cookie. Acest mecanism asigură recunoașterea utilizator între sesiuni și între cereri consecutive, fără a impune un sistem tradițional de autentificare prin înregistrare și logare pentru utilizatori. Rutele de autentificare și gestionare a sesiunii sunt următoarele:

- *GET /auth/init*: este responsabilă cu inițializarea sesiunii unui client nou sau cu validarea unui token existent. Dacă clientul trimite deja un cookie `auth_token` valid, serverul îl validează și trimite răspunsul către client. Dacă nu există un token sau dacă acesta este invalid sau expirat: se generează un UUID pentru identificarea dispozitivului și se creează un token JWT valabil 364 de zile care încorporează UUID-ul.
- *GET /auth/refresh*: este responsabilă cu reînnoirea tokenului JWT, păstrând același UUID, dar actualizând data de expirare cu încă 364 de zile. Această metoda asigură persistența sesiunii pe termen lung și evită deconectarea implicită atunci când tokenul expiră, fără a schimba identitatea utilizatorului.

Toate rutele expuse necesită autentificarea utilizatorului printr-un token JWT, transmis sub forma unui cookie `:auth_token`. Din acest token este extras identificatorul unic, care este utilizat pentru asocierea cererilor cu datele utilizatorului în baza de date.

- *GET /get-history*: returnează numele și data postărilor analizate de utilizatorul identificat prin UUID.
- *GET /post-info*: returnează raportul detaliat al unei postări din istoricul utilizatorului.
- *POST /get-analysis*: inițierea unei noi analize pentru o postare furnizată de utilizator prin intermediul unui link. Se construiește un payload, format din identificatorul unic, link-ul și data la care a fost inițiată cererea și este trimis prin intermediul producătorului Kafka către extractorul de date prin intermediul topicului “to_server”.
- *GET /analysis-status*: returnează starea unei analize pentru o postare specifică. Dacă în baza de date analiza este completă, se trimite raportul către client. În caz contrar, răspunsul conține statusul processing. Această rută este apelată periodic din interfața frontend, după inițierea analizei prin *POST /get-analysis*, pentru a actualiza progresul analizei și a afișa rezultatul imediat ce devine disponibil.
- *DELETE /delete-post*: permite ștergerea unui raport, identificată prin numele postării, data la care a fost efectuat raportul și identificatorul utilizatorului.
- *PUT /edit-name*: permite redenumirea unei postări analizate. Default, numele raportului este link-ul postării, ulterior, poate fi schimbat cu un nume nou care nu este deja folosit de către utilizator.

Pentru a asigura procesarea asincronă și scalabilă a rezultatelor analizelor, serviciul Backend API integrează un consumator Kafka dedicat. La pornirea aplicației, acest consumator este inițiat pe un thread separat, care ascultă continuu mesajele de pe topicul ‘to_server’.

Mesajele primite provin de la microserviciile specializate în extragerea și analiza datelor. Pe baza câmpului type din fiecare mesaj, consumatorul efectuează operațiuni specifice asupra bazei de date: se înserează postarea cu comentariile în format brut și metadatele corespunzătoare sau se actualizează respectiva postare cu rezultatele analizelor.

Această arhitectură este integrată în microserviciu prin intermediul Dockerfile-ului, care configurează un mediu minimailist pentru rularea aplicației Python bazate pe Uvicorn. Imaginea de bază python:3.10-slim oferă o versiune

ușoară și optimizată a Python 3.10, reducând dimensiunea containerului. Portul 8000 este expus pentru accesul extern la serverul web, iar comanda de start lansează Uvicorn, care pornește aplicația la adresa 0.0.0.0 și portul 8000, făcând astfel serviciul accesibil din afara containerului.

Serviciul de colectare date

Pentru gestionarea procesului de extragere a datelor din platforma Tiktok, acest microserviciu integrează un consumator Kafka dedicat, configurat să asculte mesajele de pe topicul *to_scraper*. La primirea unui mesaj pe acest topic, consumatorul extrage datele necesare și inițializează procesul de scraping prin componenta TiktokScraper.

Componenta TiktokScraper este responsabilă pentru extragerea automata a datelor publice ale postării de pe TikTok, iar aceste date includ: numărul de aprecieri, salvări, distribuiri și comentarii ale postării, lista comentariilor, fiecare cu data creării acestora, cu numărul de aprecieri și răspunsuri. După extragerea acestor date, scraperul trimite datele atât către server, pe topicul *to_scraper*, dar și către topicul *to_preprocessing*, pentru realizarea etapei de curățare a datelor.

Scraperul utilizează biblioteca Python TikTokApi¹ pentru a interacționa cu interfața TikTok. Accesul la datele video și comentarii este posibil prin simularea unor sesiuni de browser real folosind Playwright. Metoda asincronă *await api.create_sessions(...)* este o parte esențială din mecanismul de evitare a protecțiilor anti-bot ale TikTok. Configurarea acesteia se face prin parametrii:

- *ms_token*: tokenii de sesiune, unde este recomandată utilizarea unui token de sesiune al unui cont existent pe platformă pentru a evita blocarea.
- *num_sessions*: numărul de sesiuni paralele de browser care vor fi create. Pentru extragerea metadatelor video, am ales crearea unei singure sesiuni, întrucât procesul este unul simplu, iar pentru extragerea comentariilor, am ales un număr de 5 sesiuni, pentru a accelera colectarea în mod paralel.
- *sleep_after*: un timp de așteptare între cereri menit să reducă presiunea de pe serverul Tiktok și o practică recomandată pentru a evita detectarea automata.

¹ <https://github.com/davidteather/TikTok-API?tab=readme-ov-file>

- *browser*: am utilizat browserul default chromium.
- *headless*: cu valoarea False care permite rularea vizibilă a browserului.

Această rulare vizibilă este posibilă datorită configurației de containerizare. Dockerfile-ul aferent construiește un container Python 3.12 pregătit pentru rularea scraperului TikTok, integrând toate bibliotecile de sistem necesare pentru funcționarea browserului Chromium într-un mediu non-grafic. Este inclus suportul pentru Xvfb (X Virtual Framebuffer), care emulează un display grafic în memorie, permițând rularea Chromium în mod vizibil (*headless=False*), chiar și în lipsa unui ecran fizic. Mediul este complet automatizat: dependențele Python sunt instalate din *requirements.txt*, iar Playwright este configurat împreună cu toate resursele sale. Aplicația este lansată prin comanda *xvfb-run*, care asigură execuția în siguranță a browserului într-un context de containerizare complet izolat, dar compatibil cu cerințele grafice necesare pentru scraping.

Serviciul de preprocesare

Pentru curățarea și pregătirea datelor, acest microserviciu integrează un consumator Kafka dedicat, configurat să asculte mesajele de pe topicul *to_preprocessing*. La primirea primului mesaj pe acest topic, care conține metadatele postării: UUID, data analizei și link-ul postării, consumatorul setează contextul necesar pentru traducere și preprocesare.

După traducere, textul este preprocesat utilizând clasa *PreprocessData*, care aplică operații de curățare, conversie a emoticoanelor și emoji-urilor, eliminarea cuvintelor inutile și adaptarea textului conform arhitecturii modelelor transformers descrisă în capitolul anterior.

În final, datele preprocesate sunt pregătite și transmise către topicul Kafka dedicat analizei, asigurând un flux coerent și optimizat pentru procesarea ulterioară.

Dockerfile-ul pornește de la imaginea oficială Python 3.12, iar pentru funcționarea corectă a preprocesării textului, descarcă pachetele necesare NLTK (*wordnet*, *punkt*, *omw-1.4*, *stopwords*, *punkt_tab*) în directorul */root/nltk_data* și setează variabila de mediu *NLTK_DATA* pentru accesul la aceste resurse.

Serviciul de analiză AI

Acest microserviciu se ocupă de analiza comentariilor text pe baza modelelor de inteligență artificială create. Consumatorul Kafka ascultă mesaje pe topicul *to_analysis*, după ce primește toate loturile de comentarii, le agregă și apoi pornește analiza:

- *GeneralSentimentAnalyzer*: este un modul care utilizează modelul preantrenat *twitter-roberta-base-sentiment** pentru a clasifica comentariile în funcție de sentimente: pozitiv, negativ și neutru. Pentru fiecare postare, returnează o sinteză generală care include procentajul comentariilor din fiecare clasă și media scorurilor de încredere pentru fiecare sentiment. De asemenea, oferă etichetele atribuite individual fiecărui comentariu, pe baza clasei cu cel mai mare scor de încredere pentru acel text.
- *GeneralHateAnalyzer*: este un modul care are scopul de a detecta conținutul ofensator din comentarii. Pentru o detecție cât mai robustă, acest model combină *twitter-roberta-base-offensive** și *twitter-roberta-base-irony** astfel: pentru fiecare comentariu, dacă este clasificat ca ofensiv, primește imediat eticheta OFFENSIVE. Dacă comentariul este clasificat ca nefiind ofensiv, se verifică dacă este ironic și dacă scorul de încredere pentru ironie depășește pragul 0.8, în acest caz, comentariul este tot marcat ca OFFENSIVE. Altfel, comentariul primește eticheta NOT_OFFENSIVE. Modulul captează și mesajele ironice cu potențial ofensator, oferind ca rezultat un rezumat procentual al comentariilor ofensatoare și neofensatoare, precum și o listă cu toate comentariile clasificate ca având conținut ofensator.
- *CyberbullyingAnalysis*: folosește modulul principal al proiectului, dezvoltat prin analiza și interpretarea mai multor algoritmi și modele de învățare automată. Modelul personalizat RoBERTa primește comentariile returnate de modulul anterior, care au fost clasificate ca având conținut ofensator și returnează pentru acestea atât eticheta precisă, cât și probabilitățile aferente fiecărei clasei. La fel ca pentru celelalte module, acesta returnează și un rezumat global care evidențiază distribuția și intensitatea diferitelor tipuri de hărțuire identificate în setul de comentarii analizat.

După procesarea și trimiterea rezultatelor către topicul *to_server*, acestea sunt stocate în baza de date, fiind asociate corect asocierii postării și identificatorului unic și asigurând astfel o gestionare corectă și ușoară a datelor. Ulterior, informațiile sunt

transmise către client, oferind utilizatorului o vizualizare clară și detaliată a analizei comentariilor, contribuind astfel la analiza și monitorizarea eficientă a conținutului online.

Baza de date MongoDB

Persistența datelor este asigurată printr-o bază de date MongoDB, rulată într-un container separat, folosită pentru stocarea rapoartelor de analiză. Conexiunea la baza de date se realizează prin intermediul bibliotecii *pymongo*, folosind un URI securizat stocat în fișierul de mediu `.env`, astfel încât informațiile sensibile să rămână protejate și ușor configurabile pentru diferite medii de rulare.

Instanța MongoDB este orchestrată într-un container Docker, configurat pentru a expune portul 27017 către gazdă, asigurând astfel accesul aplicației la baza de date. Persistența datelor este garantată prin folosirea volumelor Docker dedicate, care păstrează datele pe disc chiar și în cazul repornirii containerului. Pentru menținerea fiabilității, sistemul efectuează healthchecks periodice asupra serviciului MongoDB, asigurând disponibilitatea și funcționarea continuă a bazei de date.

Datele legate de postări sunt stocate în colecția `posts_collection` din baza de date `posts_db`:

```
class Analysis(BaseModel):
    type: str
    result: Dict

class Comment(BaseModel):
    comment: str
    preprocessed_comment: Optional[str]
    likes: int
    posted: str
    lang: Optional[str] = None
    no_replies: Optional[int] = None
    label: Optional[str] = None
    offensive_label: Optional[str] = None
    cyberbullying_label: Optional[str] = None

class Post(BaseModel):
    uuid: str
    post_link: str
    platform: Optional[str] = None
    post_name: Optional[str] = None
    post_likes: Optional[str] = None
    post_saved: Optional[str] = None
    post_distribution: Optional[str] = None
    post_comments: Optional[List[Comment]] = None
    post_date: Optional[datetime.datetime] = None
    post_play: Optional[str] = None
    analysis_date: Optional[datetime.datetime] = None
    comments: Optional[List[Comment]] = []
    analyses: Optional[List[Analysis]] = []
```

Fig. 5.2.2-1 Structura bazei de date

Pentru facilitarea comunicării și afișării, sunt definite funcții de serializare care convertesc obiectele din MongoDB în dicționare serializabile, ușor de transmis către client.

Această arhitectură asigură o gestionare robustă și scalabilă a datelor, fiind esențială pentru integrarea rezultatelor generate de modulele de analiză și pentru interacțiunea cu utilizatorul final.

Volume partajate: /shared_logs și /shared_utils

Volumul */shared_utils* conține componente comune utilizate de toate microserviciile proiectului. În cadrul acestuia, fișierul *logger_config.py* configurează un logger global care scrie atât în fișierul de log, cât și în consola Docker Desktop, asigurând astfel centralizarea și monitorizarea clară a evenimentelor și erorilor în întreaga aplicație. De asemenea, *kafka_producer.py* implementează clienți Kafka Producer care sunt configurați să serializeze mesajele în format JSON, să gestioneze retrimiterile și confirmările de livrare. Funcțiile oferite facilitează trimiterea mesajelor către topic-uri Kafka specifice, urmând un model bine definit de comunicare asincronă între microservicii.

Volumul */shared_logs* este folosit pentru stocarea centralizată a fișierului de jurnalizare, asigurând persistența și accesul facil la informațiile de logging pe durata execuției serviciilor, ceea ce facilitează monitorizarea și depanarea.

Astfel, aceste volume partajate reprezintă un suport infrastructural esențial, oferind atât consistență și vizibilitate în procesul de logare, cât și un mecanism uniform și robust pentru comunicarea între componentele distribuite ale sistemului.

5.3. Analiza Undefined Modeling Language

Pentru o înțelegere clară și structurată a modului în care utilizatorii interacționează cu aplicația *from2love* și a modului în care componentele interne colaborează în timpul procesului de analiză, au fost utilizate două dintre cele mai frecvente tipuri de diagrame din cadrul limbajului de modelare UML: diagrama cazurilor de utilizare și diagrama de secvență.

5.3.1. Diagrama cazurilor de utilizare

Prin intermediul diagramei cazurilor de utilizare sunt evidențiate cerințele funcționale ale sistemului și modul în care acestea sunt accesate în mod direct de către utilizatorii aplicației. Aceasta oferă o viziune de ansamblu asupra funcționalităților principale ale aplicației din perspectiva actorilor implicați. Actorul aplicației este utilizatorul simplu, iar scopul diagramei este să descrie interacțiunile esențiale pe care acesta le are cu sistemul:

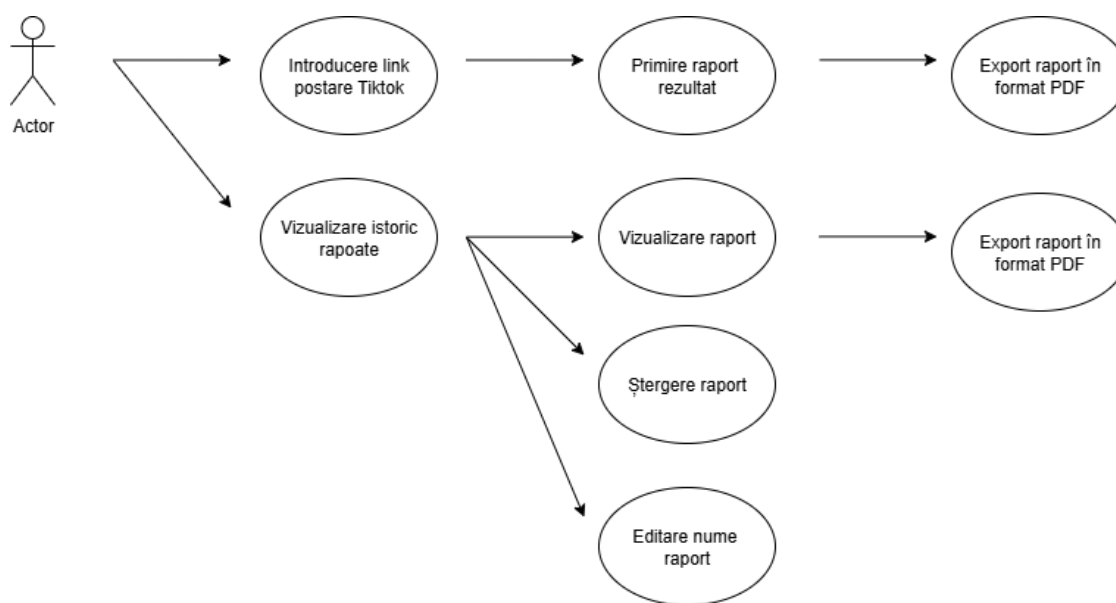


Fig 5.3.1-1 Diagrama cazurilor de utilizare

5.3.2. Diagrama de secvență

Diagrama de secvență completează analiza funcțională prin reprezentarea cronologică interacțiunii utilizatorului și componentele sistemului. Diagrama ajută la înțelegerea fluxului de date și a logicii din spatele funcționalităților aplicației, oferind o imagine clară asupra secvenței de acțiuni declanșate de utilizator.

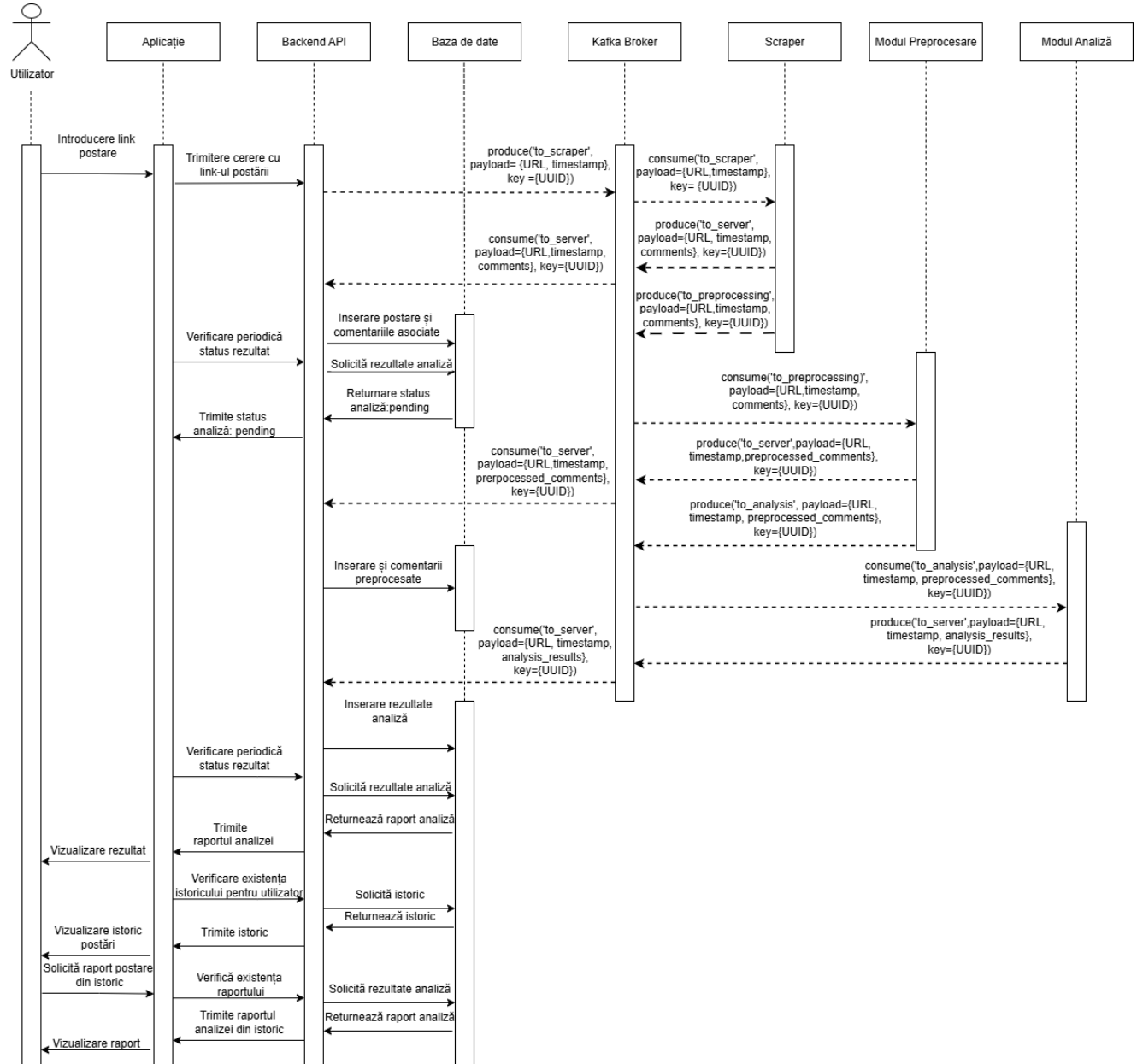


Fig 5.3.1-1 Diagrama de secvență

5.4. Procesul de testare a sistemului

Testarea reprezintă o etapă esențială în ciclul de dezvoltare al proiectului, având ca scop verificarea și validarea corectă a funcționalităților implementate. Prin această procedură, se urmărește asigurarea faptului că sistemul de analiză a sentimentelor răspunde cu succes cerințelor stabilite, oferind un mediu sigur, eficient și intuitiv pentru utilizatori. Vor fi evaluate toate componentele esențiale ale sistemului, subliniindu-se importanța și impactul lor asupra procesului de interpretare a sentimentelor și comportamentelor exprimate în comentariile de pe TikTok.

Pentru o validare riguroasă, testarea va acoperi o serie de funcționalități critice, fiecare fiind evaluată prin criterii clare de succes și posibile scenarii de eroare. Această abordare garantează identificarea timpurie a problemelor și confirmarea stabilității și performanței întregului sistem în condiții reale de utilizare. În continuare, sunt detaliate principalele funcționalități supuse testării, împreună cu rezultatele așteptate și potențialele erori ce pot apărea:

ID	Funcționalitate	Rezultat așteptat	Posibile erori
1	Generarea UUID-ului pentru fiecare sesiune de utilizator, specific browserului de pe care acesta accesează aplicația	UUID-ul este generat corect și persistent pentru 364 de zile, identificând unic browserul utilizatorului	Lipsa generării UUID, generarea de ID-uri duplicate, sesiuni nerecunoscute, erori CORS care împiedică identificarea sau accesul
2	Accesarea istoricului analizelor realizate anterior	Se afișează lista completă a rapoartelor salvate pentru utilizatorul curent, cu date corecte	Rapoarte lipsă, eroare de conectare la baza de date, afișarea incompletă sau eronată a datelor, erori CORS
3	Ștergerea unui raport din istoricul de analize	Raportul selectat este eliminat definitiv din baza de date și nu mai apare în istoricul utilizatorului	Raportul nu este șters, erori la comunicarea cu baza de date, raportul șters incorect, erori CORS
4	Editarea numelui unui raport din	Numele raportului este actualizat corect și	Modificarea nu este salvată, erori de validare a numelui,

	istoricul utilizatorului	modificarea este vizibilă în interfață	inconsistență în afișarea numelui, probleme CORS
5	Extragerea completă a informațiilor prin intermediul microserviciului dedicat	Se obțin toate datele relevante (comentarii, metadata, statistici) pentru postarea analizată	Detectarea ca bot în timpul scraping-ului, blocarea accesului, eroare la conexiunea cu internetul, timeout
6	Preprocesarea corectă a datelor prin intermediul microserviciului dedicat	Datele sunt curățate, normalizate și pregătite corect pentru analiza ulterioară	Preprocesare incompletă, erori de format, pierderi de informații
7	Analiză corectă și completă a datelor	Comentariile sunt clasificate corect pe categorii, iar rezultatele sunt coerente și complete	Eroare de conexiune pentru inițializarea modelelor
8	Inițierea analizei prin introducerea link-ului și apăsarea butonului	Analiza pornește imediat după validarea link-ului și utilizatorul primește un feedback clar	Link invalid, analiza nu pornește, erori CORS care blochează solicitarea
9	Accesul și interogarea bazei de date pentru gestionarea datelor	Interogările către baza de date returnează răspunsuri rapide și corecte pentru cererile utilizatorului	Timp de răspuns lent, erori de conexiune, date lipsă sau corupte
10	Inițializarea consumerilor și producerilor Kafka	Toate microserviciile se conectează corect la broker și ascultă topicurile corespunzătoare, comunicarea se desfășoară fluent între componente.	Timp de răspuns crescut, întreruperi în fluxul de date, conexiuni instabile, imposibilitatea de a consuma sau publica mesaje pe topicuri.

Fig 5.4-1 Planul de testare a aplicației

5.5. Scenariu de utilizare

În contextul amplificării fenomenului de hărțuire pe platformele de social media, aplicația *from2love* oferă o soluție avansată pentru analiza comentariilor postărilor publicate pe TikTok, cu scopul de a identifica în mod automat conținutul ofensator și potențial abuziv. Scopul acestui scenariu este de a ilustra modul în care aplicația asigură detecția rapidă și precisă a comportamentelor toxice, facilitând intervenția responsabililor și protejarea utilizatorilor vulnerabili.

În urma preocupărilor tot mai mari privind impactul hărțuirii online, un student la psihologie alege să studieze dinamica interacțiunilor toxice din comentariile de pe TikTok pentru lucrarea sa de diplomă. Pentru aceasta, el utilizează aplicația *from2love*, un instrument accesibil oricărui tip de utilizator, specializat în analiza și detecția automată a fenomenului de hărțuire online.

La accesarea aplicației web, utilizatorul este întâmpinat de o pagină principală intuitivă și bine structurată, care oferă un tablou clar și acces rapid la funcționalitățile esențiale. Interfața este concepută pentru a facilita navigarea simplă și eficientă, prezentând un meniu organizat ce permite accesul facil la pagina principală, pagina destinată analizei și pagina cu informații despre aplicație. Designul curat și prietenos sprijină o experiență fluidă, adaptată atât utilizatorilor începători, cât și celor avansați.

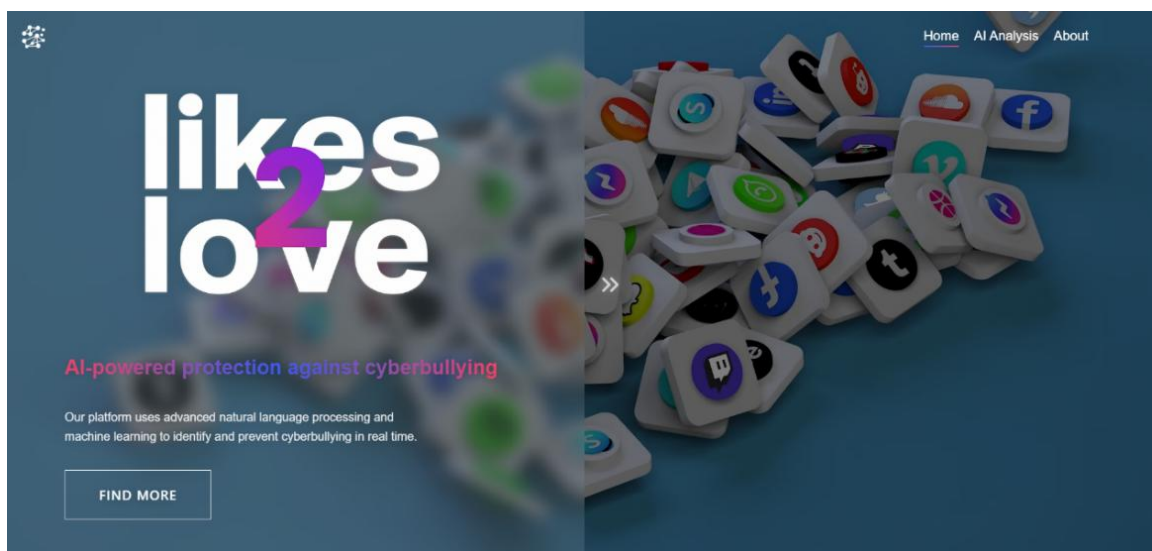


Fig 5.5-1 Pagina principală a aplicației

În cadrul paginii */find-more*, utilizatorul găsește informații detaliate despre modul în care funcționează procesul de analiză, precum și pașii esențiali care trebuie urmați pentru a obține rezultate corecte și relevante. Această secțiune oferă explicații concise și accesibile, menite să ghideze utilizatorul pas cu pas, sporind astfel înțelegerea și eficiența utilizării aplicației. La final, un buton proeminent care invită utilizatorul să demareze imediat procesul de analiză.

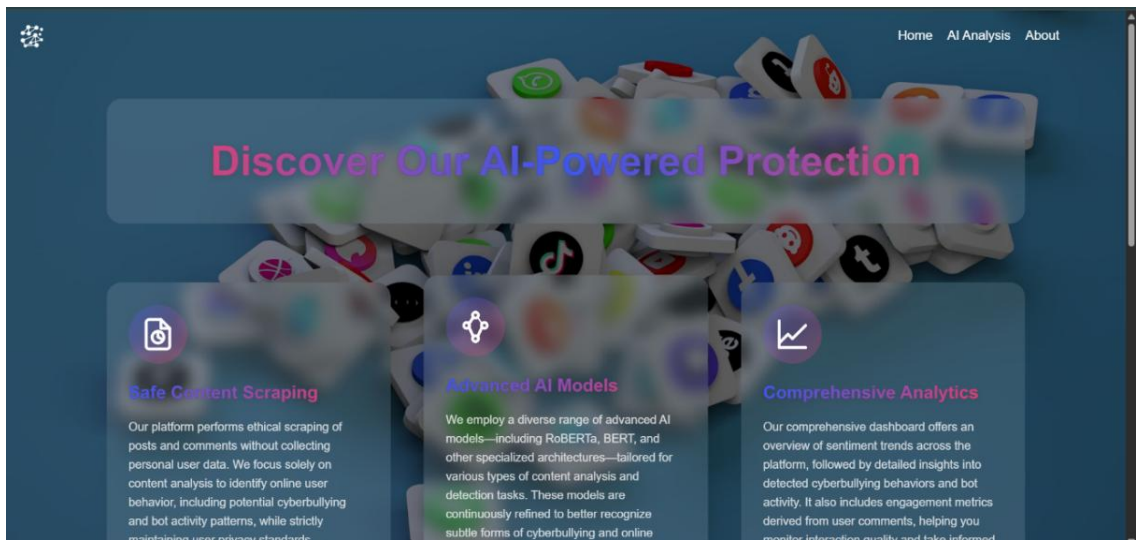


Fig 5.2-2 Pagină */find-more*

Pagina de analiză este structurată pentru a facilita navigarea și utilizarea eficientă a aplicației. În partea stângă, utilizatorul găsește o listă organizată cu istoricul analizelor efectuate anterior, care oferă acces rapid la rapoartele salvate și posibilitatea de a revizita rezultatele anterioare. Containerul din partea dreaptă este dedicat zonei de input, unde utilizatorul poate introduce link-ul TikTok pe care dorește să îl analizeze. Sub acest buton de input, există și un ghid despre cum trebuie ales un anumit link pentru analiză, pe care studentul îl accesează pentru a se asigura că parcurge toți pașii cum trebuie.

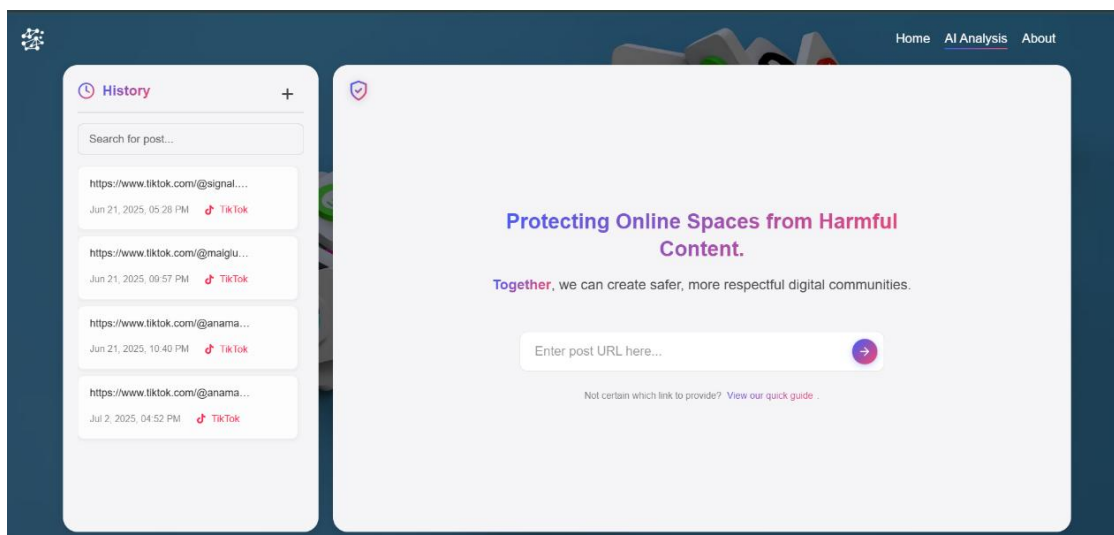


Fig 5.5-3 Pagina destinată analizei

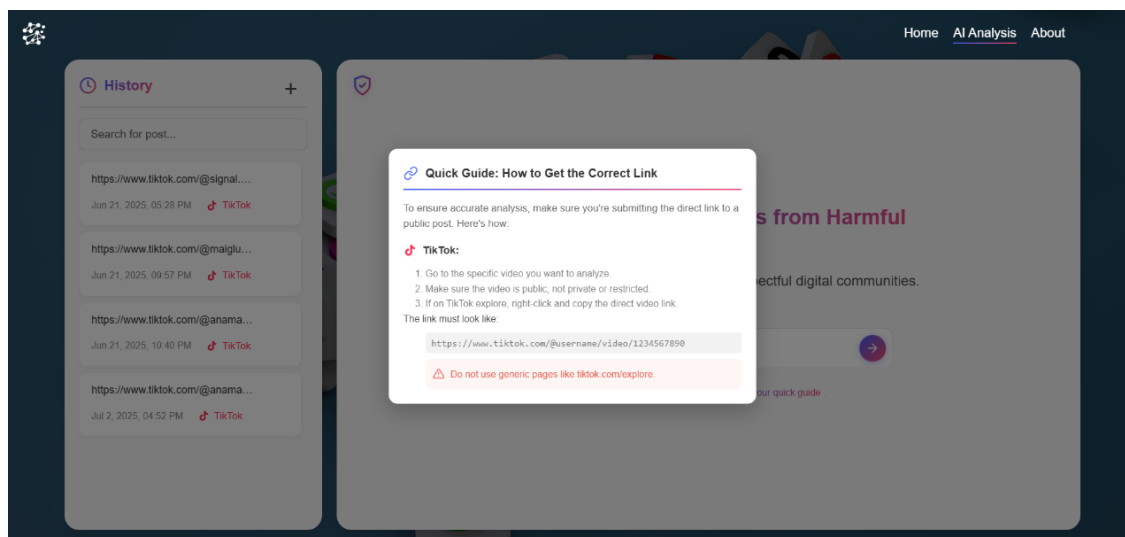


Fig. 5.5-4 Container informativ pentru utilizatori

Utilizatorul observă de asemenea că acest meniu poate fi extins, pentru a beneficia de mai multe funcționalități: filtrare, editare nume, ștergerea raportului.

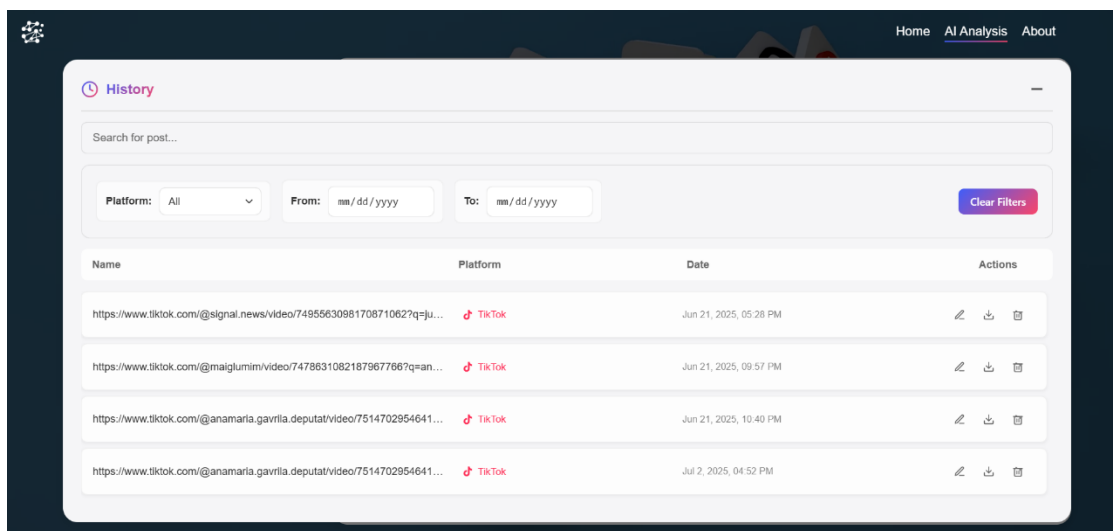


Fig 5.5-6 Istoric în variantă extinsă

După inițierea analizei, utilizatorul așteaptă procesarea datelor, timp în care aplicația afișează un mesaj sau indicator de încărcare pentru a semnaliza progresul. Odată ce analiza este finalizată, în containerul din partea dreaptă apare un raport detaliat și sugestiv, care sintetizează rezultatele obținute.

Primele informații pe care utilizatorul le observă sunt cele generale despre postare: numărul de aprecieri, comentarii, distribuiți și salvări, date care pot fi consultate și direct pe platforma TikTok. Un aspect nou și relevant este însă numărul de vizualizări al postării, o informație care nu este afișată public în cadrul rețelei sociale și este extrasă exclusiv din metadatele postării prin intermediul scraperului. În partea dreaptă sus a paginii, utilizatorul găsește butoanele „New Analysis” pentru a iniția o nouă analiză și „Export PDF” pentru a salva raportul generat în format PDF, facilitând astfel gestionarea și partajarea rezultatelor.

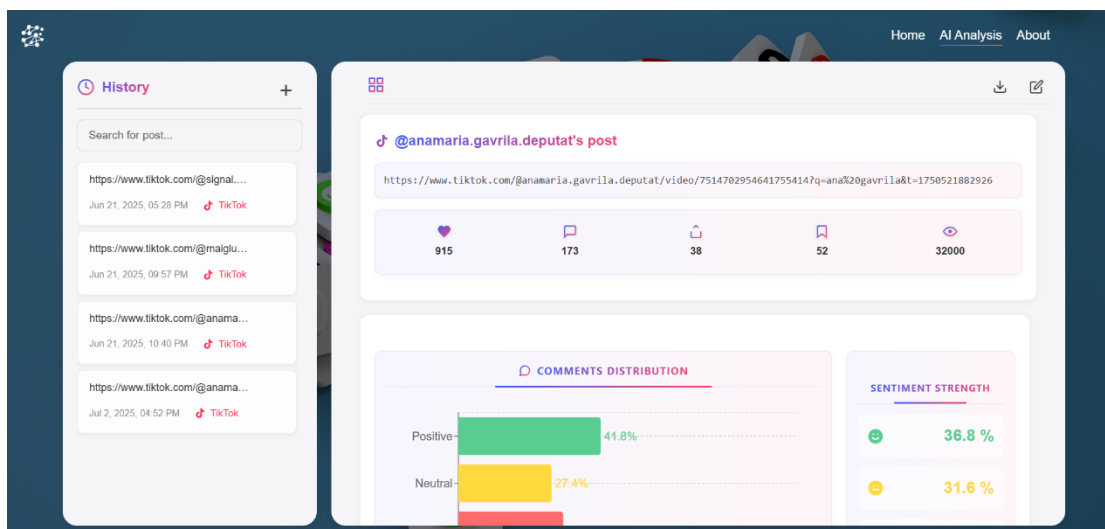


Fig 5.5-7 Vizualizare raport generat

Utilizatorul observă că distribuția etichetelor sentimentelor în comentariile analizate este de asemenea echilibrată, cu aproximativ 40% comentarii pozitive, 30% neutre și 29% negative. În partea dreapta, *Sentiment Strength* ilustrează media ponderată a probabilităților de apariție a fiecărei clase de sentiment în toate comentariile analizate. Astfel, utilizatorul constată că intensitatea sentimentelor exprimate nu este foarte puternică, nici pentru comentariile pozitive, nici pentru cele negative. Aceste grafice oferă o imagine clară și cuprinzătoare asupra tonului general al interacțiunilor din cadrul postării analizate.

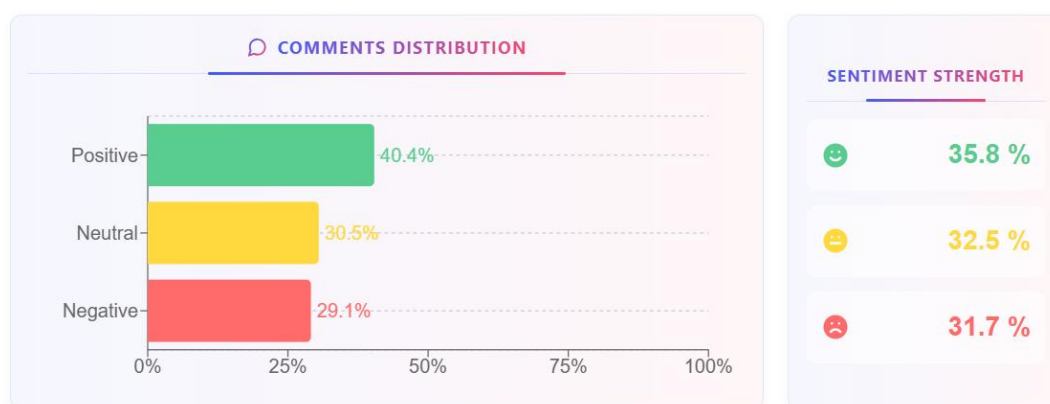


Fig 5.5-8 Grafic destinat primului model

Graficul *Content Safety* evidențiază proporția comentariilor considerate ofensatoare în cadrul postării analizate. În acest caz, utilizatorul observă că aproximativ 14% dintre comentarii conțin conținut potențial abuziv sau nepotrivit. Această informație ajută la identificarea nivelului de toxicitate din discuții, oferind un indiciu clar asupra necesității unor intervenții sau analize suplimentare. Totodată, utilizatorul observă că numărul total de comentarii reflectat în acest grafic este inferior celui menționat în metadatele postării. Acest decalaj semnalează faptul că o parte dintre comentarii au fost cel mai probabil eliminate anterior de către sistemele de moderare automată ale platformei, fiind considerate inadecvate sau în afara politicilor comunității.

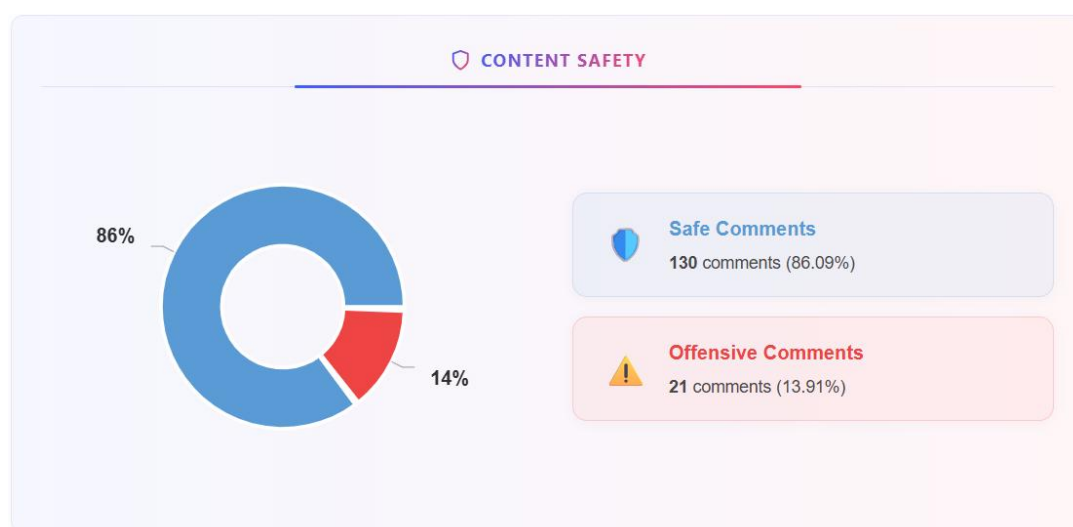


Fig 5.5-9 Piechart distribuție comentarii ofensive

Cyberbullying Distribution oferă o perspectivă detaliată asupra naturii comentariilor ofensatoare din postarea analizată. În această secțiune, utilizatorul observă cum sunt distribuite comentariile toxice în funcție de cinci categorii specifice de comportament abuziv. Această reprezentare vizuală îl ajută să înțeleagă nu doar că există comentarii problematice, ci și ce tipuri de comportament abuziv domină conversația, oferindu-i astfel un sprijin clar în demersul său de analiză calitativă pentru lucrarea de cercetare. Din cele 21 de comentarii ofensatoare, 3 dintre ele sunt prezise a fi hărțuire pe baza religiei, iar celelalte 18 nu se încadrează în niciuna dintre cele 4 clase, așadar, sunt etichetate ca fiind alt tip de hărțuire.

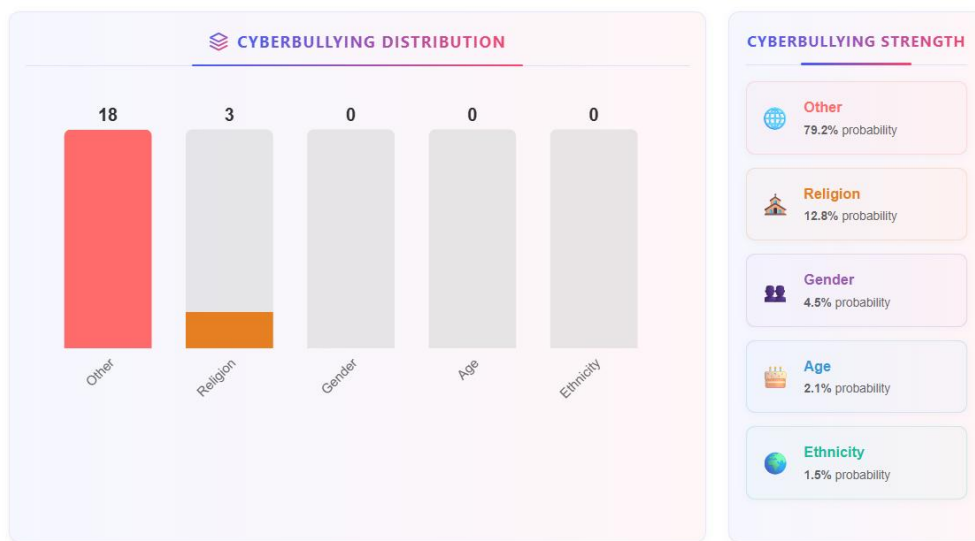


Fig 5.5-10 Grafic clasificator principal

În completarea graficului, utilizatorul observă că aplicația from2love oferă și exemple concrete de comentarii încadrate în fiecare dintre cele cinci categorii de comportament abuziv. Această funcționalitate este esențială din perspectiva utilizatorului, deoarece îi permite să evalueze acuratețea clasificării automate. Prin analiza acestor exemple, el poate valida dacă etichetele atribuite sunt relevante și corect interpretate în context, contribuind astfel la încrederea în sistem și la aprofundarea propriei înțelegeri asupra fenomenului.



Fig 5.5-11 Exemple comentarii pe clase

În continuare, utilizatorul analizează graficul *Engagement Over Time*, care ilustrează evoluția interacțiunilor pe parcursul a șapte zile, începând cu prima zi în care postarea a primit comentarii. Acest grafic oferă o imagine clară asupra ritmului și intensității angajamentului din partea publicului, evidențiind perioadele cu activitate crescută.

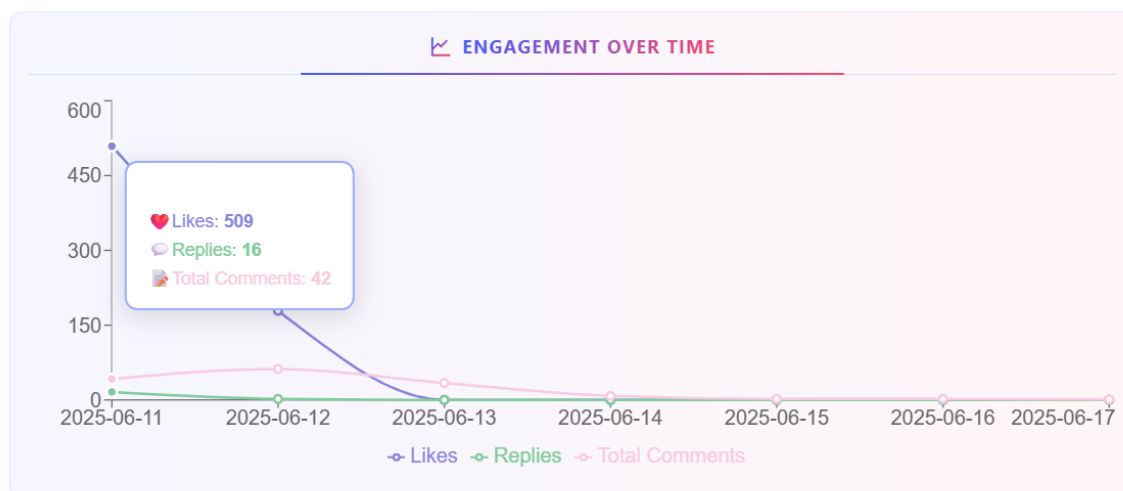


Fig 5.5-12 Grafic engagement pe zile

Utilizatorul identifică ziua cu cel mai mare nivel de engagement, pentru care aplicația generează un grafic detaliat. Aici sunt analizate comentariile din acea zi, alături de distribuția etichetelor, oferind o perspectivă despre tipurile de reacții generate pe parcursul acestei zile.

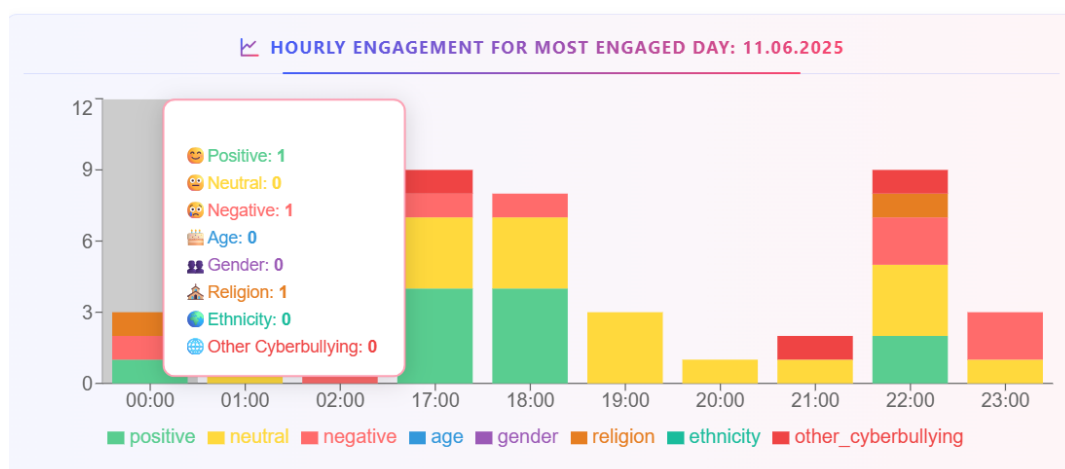


Fig 5.5-13 Grafic engagement pe ore

La finalul procesului, utilizatorul consideră că analiza generată de aplicația *from2love* i-a oferit o perspectivă clară, structurată și argumentată asupra interacțiunilor toxice din jurul postării analizate. Informațiile vizuale, scorurile calculate și exemplele afișate contribuie la validarea ipotezelor formulate în cadrul proiectului său de diplomă privind fenomenul de hărțuire pe rețelele sociale. Satisfăcut de rezultatele obținute, utilizatorul folosește funcționalitatea de export pentru a genera raportul PDF, pe care îl va include ca material de suport în lucrarea sa academică.

5.6. Dificultăți întâmpinate în procesul de implementare

În procesul de dezvoltare a aplicației *from2love*, au apărut o serie de provocări tehnice și organizaționale care au influențat ritmul și modul de implementare. Aceste dificultăți au fost întâmpinate atât la nivelul integrării componentelor complexe, cât și în gestionarea fluxului de date între microservicii, asigurarea performanței și stabilității sistemului, precum și în adaptarea modelelor de inteligență artificială la specificul datelor și cerințelor proiectului. În continuare, vom detalia principalele obstacole identificate și soluțiile adoptate pentru a le depăși.

Inițial, am implementat scraperul pentru colectarea datelor de pe TikTok utilizând librăriile BeautifulSoup și Selenium. Această abordare, deși flexibilă, s-a dovedit ineficientă datorită timpului îndelungat necesar pentru parcurgerea întregului conținut al unei postări, pentru o postare cu peste 500 de comentarii, extragerea conținutului dura peste 20 de minute. De asemenea, această metoda era susceptibilă la erori cauzate de modificările periodice ale structurii platformei TikTok, ceea ce afecta stabilitatea și acuratețea procesului de scraping automat.

Pentru a îmbunătăți performanța și fiabilitatea, scraperul a fost ulterior înlocuit cu utilizarea bibliotecii TikTokApi, care oferă un acces mai direct și mai optimizat la datele platformei. Totuși, stabilitatea conexiunii la internet și comportamentul browserului intern sunt factori critici în funcționarea corectă a scraperului. Astfel, echilibrul între eficiență, stabilitate și evitarea detecției rămâne o provocare constantă în dezvoltarea și mentenanța scraperului.

În ceea ce privește mecanismul de detecție și traducere, am constatat necesitatea introducerii unor întârzieri controlate între cererile API atunci când

volumul de date este mare, pentru a preveni blocarea temporară cauzată de limitările impuse privind numărul de solicitări simultane.

De asemenea, procesul de preprocesare a textelor a fost tratat cu rigurozitate pentru a evita erori critice, prin verificări atente ale formatului și conținutului datelor, asigurând astfel integritatea și stabilitatea fluxului de lucru.

Aceste detalii, deși aparent minore, au avut un impact semnificativ asupra robusteții și fiabilității întregului sistem, subliniind importanța unei gestionări prudente atât a resurselor externe, cât și a etapelor intermediare din procesul de analiză.

6. Concluzii și direcții viitoare

Această lucrare a avut ca obiectiv principal explorarea celor mai eficiente metode de detecție automată a hărțuirii din mediul online. În acest sens, au fost testate și comparate atât modele de învățare automată tradiționale, cât și arhitecturi avansate de rețele neuronale, cu scopul de a identifica algoritmul care oferă cele mai bune rezultate în ceea ce privește atât acuratețea, cât și aplicabilitatea practică.

Modelele clasice s-au dovedit a fi eficiente în anumite scenarii, însă cele mai bune performanțe au fost obținute prin utilizarea modelelor bazate pe arhitecturi transformer, precum BERT și RoBERTa. Deși modelele clasice au obținut acuratețe, în cea mai mare parte, de peste 90%, modelele RoBERTa și BERT au demonstrat o capacitate superioară de înțelegere a limbajului natural, cu scoruri de peste 95%, și dovedite nu doar prin metricile de testare ale modelelor, ci și prin testarea acestora pe date reale. Acest lucru confirmă faptul că modelele de tip transformer sunt în prezent cele mai potrivite soluții pentru analiza conținutului textual online, în special în contexte sensibile precum identificarea hărțuirii.

Procesul de cercetare a evidențiat, de asemenea, o limitare semnificativă legată de disponibilitatea bazelor de date publice specializate în conținut abuziv, ceea ce a impus o atentă selecție și preprocesare a setului ales în final, precum și utilizarea unor tehnici echilibrate de curățare a datelor. Astfel, s-a urmărit asigurarea unei performanțe stabile și generalizabile a modelelor implementate.

Aplicația web integrează toate componentele necesare pentru analiza automată a comentariilor de pe TikTok, oferind utilizatorului acces facil la un raport detaliat și vizualizări clare, cu funcționalități moderne precum. Arhitectura aplicației se bazează pe un sistem asincron scalabil, folosind tehnologii robuste pentru procesarea datelor în timp real și o experiență fluidă de utilizare.

În ceea ce privește direcțiile de dezvoltare viitoare, o prioritate va fi construirea unei baze de date proprii, etichetate manual, pentru a permite adaptarea cât mai fidelă a modelelor la realitatea mediului online. De asemenea, un pas important îl constituie extinderea analizei asupra conținutului vizual, mai exact, imaginile distribuite în comentarii care, în forma actuală, nu sunt incluse în procesul de detecție. Integrarea acestui tip de conținut ar contribui la obținerea unei înțelegeri mai nuanțate și mai

cuprinzătoare a interacțiunilor online. Totodată, se are în vedere adaptarea platformei și pentru alte rețele de socializare, în vederea construirii unui sistem universal de analiză și monitorizare a sănătății conversaționale în mediul digital.

În concluzie, acest proiect demonstrează că tehnologiile moderne de inteligență artificială pot fi utilizate cu succes în sprijinul combaterii hărțuirii online, oferind atât rezultate performante din punct de vedere tehnic, cât și valoare practică într-un context social din ce în ce mai relevant. Aplicația *from2love* reprezintă un exemplu funcțional de integrare a cercetării științifice în soluții utile, accesibile și scalabile.

NECLASIFICAT

Bibliografie

- [1] D. Maladhy, J. Jeevitha, M. Madhumitha, S. Subashree, *BERT-Based Cyberbullying Detection*, International Research Journal on Advanced Engineering Hub, April 2025, DOI: 10.47392/IRJAEH.2025.0293
- [2] Parth Aggarwal, Rhea Mahajan, *Shielding Social Media: BERT and SVM Unite for Cyberbullying Detection and Classification*, Journal of Information Systems and Informatics, June 2024, DOI: 10.51519/journalisi.v6i2.692
- [3] Lotfi, C., Srinivasan, S., Ertz, M., and Latrous, I. *Web Scraping Techniques and Applications: A Literature Review*, SCRS Conference Proceedings on Intelligent Systems, 2021, pp. 381-394. DOI: 10.52458/978-93-91842-08-6-38
- [4] Fetzer, J.H., *What is Artificial Intelligence?*, Artificial Intelligence: Its Scope and Limits. Studies in Cognitive Systems, vol 4. Springer, Dordrecht, DOI: 10.1007/978-94-009-1900-6_1
- [5] A. Ahad, S.H. Talpur, A.K. Jumani, *Natural Language Processing Challenges and Issues: A Literature Review*, Gazi University Journal of Science 2023: 1522-1536 (2023), DOI: 10.35378/gujs.1032517
- [6] A.G. Philipo, D.S. Sarwatt, J.Ding, M.Daneshmand, *Cyberbullying Detection: Exploring Datasets, Technologies, and Approaches on Social Media Platforms*, DOI:10.48550/arXiv.2407.12154
- [7] F. Alrowais, A. A. Jamjoom, H. Karamti, M. Umer, S. Alsubai, T.-H. Kim și I. Ashraf, *RoBERTaNET: Enhanced RoBERTa Transformer Based Model for Cyberbullying Detection with GloVe Features*, IEEE Access, vol. 12, January 2024, pp. 58950–58959, DOI: 10.1109/ACCESS.2024.3386637
- [8] Kamaluddin, M.I. Rasyid, M.W.K. Abqoriyyah, F.H. Saehu, *Accuracy Analysis of DeepL: Breakthroughs in Machine Translation Technology*. Journal of English Education Forum (JEEF), vol. 4, no. 2, June 2024, pp. 122-126, DOI: 10.29303/jeeef.v4i2.681
- [9] Cambrigde Royal College of Psychiatrists, *Cyberbullying and its impact on young people's emotional health and well-being*, The Psychiatrist , Volume 37 , Issue 5 , May 2013 , pp. 167 – 170, DOI: 10.1192/pb.bp.112.040840

- [10] Lobe B., Velicu A., Staksrud E., Chaudron S., Di Gioia R. *How Children (10-18) Experienced Online Risks during the COVID-19 Lockdown: Spring 2020: Key Findings from Surveying Families in 11 European Countries*, Publications Office of the European Union; Luxembourg, 2021, DOI: 10.2760/562534
- [11] Wissam Antoun, Fady Baly, Hazem Hajj, *AraBERT: Transformer-based Model for Arabic Language Understanding*, DOI: 10.48550/arXiv.2003.00104
- [12] J. Wang, K. Fu, C.T. Lu, *SOSNet: A Graph Convolutional Network Approach to Fine-Grained Cyberbullying Detection*, Proceedings of the 2020 IEEE International Conference on Big Data, December 10-13, 2020, DOI: 10.1109/BigData50022.2020.9378065
- [13] Zampieri, M. Malmasi, S. Nakov, P., Rosenthal, S., Farra, N. and Kumar, R. *Predicting the Type and Target of Offensive Posts in Social Media*. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). pp. 1415-1420, DOI: 10.18653/v1/N19-1144
- [14] Grace Powell, Yasin N. Silva, Deborah Hall, Mara Hamlett, *A Labeled Dataset for Investigating Cyberbullying Content Patterns in Instagram*, Proceedings of the International AAAI Conference on Web and Social Media, 16(1), 1251-1258. DOI: 10.1609/icwsm.v16i1.19376
- [15] Dabade, M. "Sentiment Analysis of Twitter Data by Using Deep Learning and Machine Learning." Turkish Journal of Computer and Mathematics Education (TURCOMAT), vol. 12, no. 6, Apr. 2021, pp. 962-970.